ABSTRACT

Keyword-based applicant tracking systems rank candidates by the overlap between a resume and a job description. They reward vocabulary rather than evidence, they cannot relate similar skills, and they give no explanation of their decisions, while recent audits show that neural resume screeners can inherit demographic bias. This project presents the AI Powered Resume Screening and Ranking System, an explainable, bias-aware and offline-capable screening system that treats every skill as a claim that needs evidence. A resume parser with an optical character recognition fallback produces structured profiles, and a 79-skill ontology with alias normalisation, fuzzy matching and partial credit reports matched, partially matched and missing skills. A seven-factor hybrid ranker combines skill match, semantic similarity, experience, education, projects, certifications and resume quality, and an evidence-weighted skill coverage gives half credit to skills that are listed but not demonstrated.

A blind-screening filter removes names, contact details and gendered words before any evaluation, and a council of four agents (Tech Lead, Culture and Retention, Fair-Hiring Auditor, and Seniority and Compensation) produces a consensus verdict with knock-out gates, red flags, confidence, dissent and human-review triggers. The system also contains a GitHub authenticity inspector, a hybrid dense and BM25 talent search, an interview-question generator and a retrieval-grounded recruiter chatbot. It is implemented as a Python core, a FastAPI service with a relational database, a Streamlit workspace and a React dashboard, and it is deployed as a multi-user web service with server-side accounts (salted PBKDF2 hashing, expiring sessions, verified Google sign-in) and a MongoDB Atlas database. A suite of 78 automated tests passes. On a pilot of twelve synthetic resumes, a screening takes about 30 milliseconds per resume, a keyword-stuffed resume that a keyword baseline scores like a genuine candidate is ranked tenth and rejected, scores did not change in 96 name-substitution runs, and rankings were stable under random weight changes of 20 percent. All data are synthetic, and the limitations of the evaluation are stated explicitly.

TABLE OF CONTENTS

CHAPTER NO.		TITLE	PAGE NO.
1	**INTRODUCTION**		**1**
	1.1	General Overview	2
	1.2	Problem Statement	3
	1.3	Objectives	4
	1.4	Scope	4
	1.5	Motivation	5
	1.6	Methodology	6
	1.7	Applications	7
	1.8	Advantages	7
	1.9	Limitations	8
	1.10	Project Outcomes	9
2	**LITERATURE SURVEY**		**11**
	2.1	Literature Survey Summary	15
	2.2	Research Gaps	16
3	**PRELIMINARIES**		**18**
	3.1	Python and Its Ecosystem	18
	3.2	Resume Parsing and Text Extraction	19
	3.3	Vector Space Model and TF-IDF	20
	3.4	Dense Sentence Embeddings	20
	3.5	Okapi BM25 and Hybrid Retrieval	21
	3.6	Skill Ontology, Fuzzy Matching and Partial Credit	23
	3.7	Explainability and Fairness	25
	3.8	Multi-Agent Deliberation	27
	3.9	Evaluation Metrics	27
4	**REQUIREMENT SPECIFICATIONS**		**29**
	4.1	Functional Requirements	29
	4.2	Non-Functional Requirements	31
	4.3	Hardware Requirements	32
	4.4	Software Requirements	32
	4.5	User Requirements	34

TABLE OF CONTENTS (Contd.)

CHAPTER NO.		TITLE	PAGE NO.
5	**SYSTEM DESIGN**		**35**
	5.1	High-Level Design	35
	5.2	System Architecture	37
	5.3	Data Flow Diagram	38
	5.4	Design of the Four-Agent Council	39
	5.5	Use-Case Diagram	39
	5.6	Sequence Diagram	40
	5.7	Database Design	41
	5.8	Interface Design	45
6	**IMPLEMENTATION**		**48**
	6.1	Overview of Technologies Used	48
	6.2	Implementation Details of Modules	49
	6.3	Difficulties Encountered and Strategies Used to Tackle Them	63
7	**TESTING AND EVALUATION**		**67**
	7.1	Unit Testing	67
	7.2	Integration Testing	70
	7.3	System Testing	71
	7.4	User Acceptance Testing	73
8	**RESULTS**		**74**
	8.1	Snapshots	74
	8.2	Experimental Setup and Research Questions	85
	8.3	Processing Latency (RQ4)	86
	8.4	Ranking Divergence from the Keyword Baseline (RQ1)	87
	8.5	Evidence Mechanism	88
	8.6	Ablation of Evidence Weighting and Gates	89
	8.7	Demographic Robustness: Counterfactual Name Swap (RQ2)	90
	8.8	Sensitivity to the Heuristic Weights (RQ3)	90
	8.9	Talent-Search Retrieval Quality	91
	8.10	Scaled 100-Resume Multi-Role Benchmark	91
	8.11	Adversarial Edge-Case Stress Test (RQ5)	93
	8.12	Discussion of the Research Questions	94
9	**CONCLUSION AND FUTURE ENHANCEMENTS**		**95**
	9.1	Conclusion	95
	9.2	Future Enhancements	96
	BIBLIOGRAPHY		**98**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Hybrid retrieval flow of the talent search: query understanding, dense and BM25 scoring, fusion and per-candidate aggregation	22
3.2	Decision flow of the skill matcher: normalisation, exact match, related-pair match and missing skill	25
4.1	Context diagram of the AI Powered Resume Screening and Ranking System	29
5.1	System workflow diagram of the AI Powered Resume Screening and Ranking System	36
5.2	System architecture of the AI Powered Resume Screening and Ranking System	37
5.3	Level-1 data flow diagram of the system	38
5.4	Orchestration of the four-agent council	39
5.5	Use-case diagram of the system	40
5.6	Sequence diagram of a screening session	41
5.7	Entity-relationship diagram of the database	42
5.8	Sign-in card of the web interface	46
5.9	Workspace of the web interface: active role and candidate intake	46
6.1	Verdict thresholds and knock-out gates of the council	57
8.1	Candidate intake with the core suite of twelve resumes loaded	75
8.2	Screening leaderboard: run telemetry, talent-pool search and the first candidate cards	76
8.3	Leaderboard cards with evidence lines, skill chips and red-flag counts	77
8.4	Compact table view of the leaderboard	78
8.5	Candidate dossier: scorecards of the four council agents and the debate	79
8.6	Head-to-head comparison matrix of two candidates	80
8.7	Interview assistant: about twenty candidate-specific questions with category filters and evaluation rubrics	81
8.8	Recruiter analytics and cohort metrics	82
8.9	Recruiter chatbot answering a red-flag audit question	83
8.10	Recruiter chatbot answering follow-up questions about the candidate discussed last, with clickable suggestions	84
8.11	Leaderboard with blind screening switched on: identities are hidden	85
8.12	Per-stage processing time per resume	86
8.13	Keyword baseline versus proposed hybrid score for the twelve pilot resumes	87
8.14	Component scores of the four demonstration resumes	88
8.15	Scores of the four council agents and the consensus for the demonstration resumes	89
8.16	Council consensus with and without evidence weighting	90
8.17	Verdicts of the 100-resume benchmark by role	92

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Reviewed Literature	15
2.2	Research Gaps and the Response of the Proposed System	16
3.1	Comparison of Sparse and Dense Representations	22
5.1	REST Endpoints of the Service	43
5.2	Collections of the MongoDB Document Store	44
6.1	Education Score Components	52
6.2	Components of the Resume Quality Score	53
6.3	Adjustments of the Culture and Retention Agent	55
7.1	Representative Unit and Component Test Cases	67
7.2	Automated Test Results by Test Area	68
7.3	System Test Results of the Demonstration Resumes	71
8.1	Per-Resume Processing Latency (12-Resume Pool, TF-IDF, Single Thread)	86
8.2	Keyword Baseline and Proposed Hybrid Scores of the Four Test Cases	87
8.3	Council Evidence Metrics and Flags of the Four Test Cases	88
8.4	Ablation: Effect of Evidence Weighting and Red-Flag Gates on the Pilot Pool	89
8.5	Name-Swap Experiment: Maximum Score Change over All Substitutions (Points)	90
8.6	Monte Carlo Weight-Perturbation Analysis (±20 Percent)	91
8.7	Retrieval Quality (8 Queries, 12 Resumes)	91
8.8	100-Resume Benchmark by Role (S = Strong, L = Lean or Hire, R = Reject)	92
8.9	Confusion Matrix: Generation Label Versus System Verdict	92
8.10	Adversarial Probes: Verdicts Before and After the New Gates (Consensus in Parentheses)	93

LIST OF ABBREVIATIONS

SL. NO.	ABBREVIATION	FULL FORM
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	ATS	Applicant Tracking System
4	BERT	Bidirectional Encoder Representations from Transformers
5	BM25	Best Matching 25 (Okapi BM25)
6	CGPA	Cumulative Grade Point Average
7	CORS	Cross-Origin Resource Sharing
8	CPU	Central Processing Unit
9	CSS	Cascading Style Sheets
10	DFD	Data Flow Diagram
11	DOCX	Office Open XML Document
12	ER	Entity-Relationship
13	ESCO	European Skills, Competences, Qualifications and Occupations
14	EU	European Union
15	FA*IR	Fair Top-k Ranking Algorithm
16	GDPR	General Data Protection Regulation
17	GUI	Graphical User Interface
18	HIRE	Hire verdict of the council
19	HTML	Hypertext Markup Language
20	HTTP	Hypertext Transfer Protocol
21	IDF	Inverse Document Frequency
22	INR	Indian Rupee
23	JD	Job Description
24	JSON	JavaScript Object Notation
25	JSX	JavaScript XML
26	LLM	Large Language Model
27	LPA	Lakhs Per Annum

LIST OF ABBREVIATIONS (Contd.)

SL. NO.	ABBREVIATION	FULL FORM
28	MiniLM	Miniature Language Model
29	ML	Machine Learning
30	nDCG	Normalised Discounted Cumulative Gain
31	NLP	Natural Language Processing
32	OCR	Optical Character Recognition
33	ORM	Object-Relational Mapping
34	PBKDF2	Password-Based Key Derivation Function 2
35	PDF	Portable Document Format
36	PK / FK	Primary Key / Foreign Key
37	RAG	Retrieval-Augmented Generation
38	REST	Representational State Transfer
39	RQ	Research Question
40	SBERT	Sentence-BERT
41	SHA-256	Secure Hash Algorithm, 256-bit
42	SMTP	Simple Mail Transfer Protocol
43	SQL	Structured Query Language
44	SRE	Site Reliability Engineering
45	TC	Test Case
46	TF	Term Frequency
47	TF-IDF	Term Frequency–Inverse Document Frequency
48	UAT	User Acceptance Testing
49	UI	User Interface
50	URL	Uniform Resource Locator
51	USD	United States Dollar
52	VKIT	Vivekananda Institute of Technology
53	VTU	Visvesvaraya Technological University
54	XAI	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

Recruitment is one of the most information-intensive activities of an organisation. A single advertised vacancy can attract hundreds of applications, and every application must be read, interpreted and compared against the requirements of the role before a shortlist can be prepared. Automated screening software, usually called an Applicant Tracking System (ATS), has therefore become the first gatekeeper in many hiring pipelines. Studies of artificial intelligence in human-resources management stress both the efficiency gains and the managerial challenges of such tools, and research on automated interviews shows that candidates accept automated decisions less readily when the stakes are high. Because the screening stage decides which people are never seen by a human recruiter, the way it ranks candidates has direct consequences for the people applying and for the organisations hiring.

Most commercial and academic screening tools rank candidates by lexical overlap with the job description. Such a keyword approach is fast and easy to implement, but it rewards vocabulary rather than evidence: a resume that lists every fashionable tool can outrank one that demonstrates fewer tools with measurable results. At the other end of the spectrum, learned screeners based on neural networks and large language models can capture meaning yet they are opaque and have been shown to inherit demographic bias from their training data. Regulation is moving in the same direction: the European Union Artificial Intelligence Act places systems that filter or evaluate job applicants among high-risk applications, which brings obligations of transparency and human oversight.

The AI Powered Resume Screening and Ranking System developed in this project, referred to as the system or Nexus ATS, is the second phase of a university recruitment project. The first phase was a keyword-matching ATS. The second phase replaces the keyword score with an explainable, evidence-based and bias-aware decision process. It parses resumes, normalises skills through an auditable skill ontology, scores each candidate on seven weighted signals, optionally hides identity before any scoring takes place, and lets a council of four specialised software agents.

1.1 General Overview

Hiring is a decision made under limited information. A resume is a self-reported document, and the claims it makes (skills, years of experience, projects, certifications) are not verified at the screening stage. A recruiter reading a resume quickly looks for supporting evidence: where was the skill used, for how long, and with what result? A keyword-based ATS cannot do this. It can only count how many words of the job description also occur in the resume. As a consequence it cannot tell that FastAPI and Flask are closely related, it cannot tell a skill that was demonstrated in a project from a skill that was merely listed, and it cannot notice that the dates of a resume are impossible.

The system presented in this report treats a skill as a **claim that needs evidence**. A required skill that appears in the experience or project text of a resume earns full credit; a skill that appears only in the skills list earns half credit. This simple rule is the basis of the evidence-weighted coverage that drives both the ranking and the final verdict. It allows the system to separate a genuine candidate from a keyword-stuffed one, which a plain keyword score cannot do.

The system is implemented in Python for the scoring core and in React for the recruiter interface. The Python core contains the resume parser (with an optical character recognition fallback for scanned documents), the job-description parser, the skill ontology, the skill-gap analyser, the hybrid ranker, the resume quality analyser, the explainable-AI module, the blind-screening filter, the four-agent council, the GitHub authenticity inspector, the hybrid talent-search engine and the interview-question generator. The same core is exposed through a FastAPI web service backed by a relational database, and through a Streamlit application. In the deployed system the web interface calls the API, which stores user accounts, sessions and the candidate pools in a MongoDB Atlas database. The React interface, deployed as a web site, offers a guided workflow for the recruiter: choose a role, load candidates, run the screening engine and review the results on a leaderboard, a candidate dossier, a side-by-side comparison, an interview assistant, an analytics page and a recruiter chatbot.

Two design principles guided the whole project. The first is explainability: every number shown to the recruiter can be traced to a rule, and every verdict is accompanied by the features that produced it. The second is graceful degradation: the system runs completely offline, using TF-IDF vectors when the dense language model.

1.2 Problem Statement

Keyword-based applicant tracking systems rank candidates by counting how many words of a job description appear in a resume. This approach has three weaknesses. First, it rewards keyword stuffing and cannot distinguish a skill that is demonstrated in a project from one that is only listed. Second, it cannot reason about related skills, so a candidate who knows Flask is scored as having no knowledge of FastAPI. Third, it produces a single number with no explanation, so a rejected candidate and the recruiter who rejected him or her have no way of knowing why the decision was taken.

Learned screening models address the first two weaknesses but introduce a new one. Studies of resume screening with embedding models and large language models have found that the same resume receives different scores when only the name of the candidate is changed, which echoes the classical field experiments on labour-market discrimination. A screening tool that cannot explain its ranking, cannot demonstrate that it ignores protected attributes and cannot hand uncertain cases to a person is therefore both ethically and legally fragile.

The problem addressed by this project is the design and implementation of a recruitment screening system that (i) ranks candidates by evidence rather than by vocabulary, (ii) understands related skills and synonyms, (iii) is invariant to the name and contact details of the candidate, (iv) explains every score and verdict, (v) reports its own uncertainty and routes doubtful cases to a human reviewer, and (vi) runs without network access so that its behaviour can be reproduced and tested.

The system must also keep the roles of its components separate. Detecting that a resume mentions a skill, crediting a related skill, discounting an unevidenced claim, flagging an inconsistent date and issuing a hiring verdict are different operations, and each of them must be visible to the recruiter on its own rather than being merged into one opaque score.

1.3 Objectives

The main objectives of the project are:

1. To extract structured information (contact details, skills, education, experience, projects and certifications) from resumes in PDF, DOCX and plain-text formats, with an OCR fallback for scanned documents.
2. To build an auditable skill ontology with alias normalisation, fuzzy matching and partial credit for related skills, and to report matched, partially matched and missing skills for every candidate.
3. To implement a seven-factor hybrid ranking engine that combines skill match, semantic similarity, experience, education, projects, certifications and resume quality into one explainable score.
4. To treat skills as claims and to discount skills that are listed but not demonstrated, through an evidence-weighted skill coverage measure.
5. To implement bias-aware blind screening that removes names, contact details, locations, profile links and gendered words before any agent evaluates a resume, and to verify name invariance through a counterfactual name-swap test.
6. To implement a four-agent council (Tech Lead, Culture and Retention, Fair-Hiring Auditor, and Seniority and Compensation) that produces a consensus verdict with knock-out gates, red flags, confidence, dissent and human-review triggers.
7. To provide a recruiter interface with a leaderboard, candidate dossier, comparison matrix, interview-question generator, analytics page and a retrieval-grounded recruiter chatbot, and a REST API with server-side authentication and Google sign-in for integration.
8. To evaluate the system on synthetic resume pools, to measure its processing time, and to state the limitations of the evaluation honestly.

1.4 Scope

The AI Powered Resume Screening and Ranking System covers the complete screening stage of a hiring pipeline: from the moment a recruiter selects a role and loads resumes to the moment a ranked, explained shortlist is available for human review. The

scope includes resume and job-description parsing, skill normalisation and matching, multi-signal scoring, bias-aware anonymisation, multi-agent review, GitHub-based verification of claimed programming languages, natural-language search over the candidate pool, interview-question generation, a recruiter dashboard and a REST service with a persistent screening history, user accounts and a managed document database.

The system is designed for technical roles. The skill ontology contains 79 canonical skills grouped into eight categories (programming languages, artificial intelligence and machine learning, web and backend frameworks, data science and analytics, cloud and DevOps, databases and storage, frontend technologies and software engineering) and eight role families are pre-configured: machine learning engineer, cloud DevOps and site-reliability lead, full-stack AI engineer, data-platform engineer, backend architect, AI-safety auditor, business-intelligence analyst and Python backend developer. A recruiter can also create a custom role by editing the competency list.

The system is a decision-support tool. It does not decide to hire or reject a candidate on its own: it ranks, explains and flags, and a person takes the decision. The project is an engineering prototype evaluated on synthetic data. Evaluation on real applicant pools with recruiter judgements as ground truth, large-scale multi-user deployment, validation of certificates with their issuers and measurement of group-fairness metrics on demographically labelled data are outside the scope of the current implementation.

1.5 Motivation

The motivation for this project comes from three observations. The first is practical. Recruiters receive far more applications than they can read, and the first filter decides who is read at all. A filter that rewards keyword stuffing penalises honest candidates, and a filter that cannot explain itself leaves the recruiter unable to defend a decision. The second observation is ethical. It has been documented that an AI recruiting tool built at a large technology company was abandoned after it penalised resumes associated with women, that identical resumes with different names receive different numbers of call-backs, and that the same pattern appears in modern language-model screeners. A system that treats fairness as a design requirement rather than as an afterthought is therefore needed. The third observation is regulatory: recruitment systems are classified as high-risk under the European Union Artificial Intelligence Act, and personal data in resumes is protected by data-protection law.

The project also offers an opportunity to combine several areas of computer science in a single working system: natural-language processing for parsing and semantic similarity, information retrieval for talent search, rule-based reasoning for the skill ontology and the red-flag gates, multi-agent evaluation for the council, web engineering for the dashboard and API, and software testing for the verification of the whole pipeline. Developing these components as separate, testable modules also gives the team practical experience of the engineering practices expected of a production system.

1.6 Methodology

The development of the system followed an iterative, module-by-module process. Each stage produced a tested component before the next stage was started. The main stages are described below, and the end-to-end workflow is shown later in Figure 5.1.

- **Requirement analysis:** the weaknesses of the Phase 1 keyword ATS were analysed and converted into functional and non-functional requirements.
- **Literature survey:** research on person-job fit, semantic retrieval, algorithmic hiring bias, explainable AI and multi-agent evaluation was reviewed to identify the gaps that the system should address.
- **Data preparation:** a pilot pool of twelve resumes (eight resumes shipped with the project and four purpose-written test cases), a 50-candidate demonstration pool and a 100-resume multi-role synthetic benchmark were prepared. All resumes are synthetic.
- **Parsing and ontology:** the resume parser, the job-description parser and the skill ontology with alias normalisation, fuzzy matching and related-skill rules were implemented.
- **Scoring and explanation:** the seven-factor hybrid ranker, the skill-gap analyser, the ATS quality analyser and the explainable-AI module were implemented.
- **Fairness and review:** the blind-screening filter, the four-agent council with consensus, gates and red flags, and the GitHub authenticity inspector were implemented.
- **Interfaces and persistence:** the React dashboard, the Streamlit application, the FastAPI service and the SQLAlchemy data model were developed, and for the

deployed system a MongoDB store for accounts, sessions and candidate pools with server-side authentication was added.

- **Verification and evaluation:** a suite of 78 automated tests (49 on the Python side and 29 on the web-interface side) was written, and experiments on processing time, ranking agreement with the keyword baseline, evidence ablation, name invariance, weight sensitivity, retrieval quality and adversarial resumes were run.

1.7 Applications

Although the prototype was built for technical roles, the approach applies wherever a large number of documents must be compared against a written set of requirements and the decision has to be explained. The main applications are listed below.

- **Corporate recruitment:** shortlisting candidates for engineering, data and analytics vacancies with an explanation for every ranking position.
- **Campus placement:** ranking student resumes against the requirements of visiting companies, with skill-gap reports that tell students what to learn.
- **Recruitment agencies:** searching a large pool of stored resumes with natural-language questions and receiving evidence snippets rather than names only.
- **Fair-hiring audits:** running the same pool in standard and blind modes and comparing the outcomes, which supports internal audits of screening practice.
- **Resume improvement:** the ATS quality analyser produces actionable suggestions that a candidate can use to improve a resume.
- **Interview preparation:** the interview assistant generates about twenty tailored technical, system-design, gap-probing, red-flag and behavioural questions per candidate, each with a note on what to look for, for the interview panel.

1.8 Advantages

- **Evidence-based ranking:** a skill that is demonstrated counts for more than a skill that is only listed, which separates genuine candidates from keyword-stuffed ones.
- **Explainability:** every score, verdict and red flag comes with a stated reason, and the cross-examination between the agents is stored and can be replayed.

- **Fairness by design:** blind screening removes direct identifiers before any agent sees the resume, an auditor agent can reduce the weight of a biased evaluator, and name invariance is tested explicitly.
- **Uncertainty handling:** the council reports a confidence value, lists dissenting agents and requests human review for borderline or poorly documented cases instead of issuing a confident verdict.
- **Offline operation:** all scoring components run locally without network access; dense embeddings, language-model narratives and the GitHub inspector are optional and degrade gracefully.
- **Speed:** a complete screening takes about 30 milliseconds per resume on a laptop CPU in the offline configuration.
- **Modularity:** the scoring core is independent of the user interface, so the same algorithms serve the React dashboard, the Streamlit application and the REST API.
- **Reproducibility:** the decision logic is deterministic, and the thresholds and weights are configurable.

1.9 Limitations

- All evaluation resumes are synthetic and were written with knowledge of the system's rules. The results therefore demonstrate consistency and mechanism, not accuracy on real hiring outcomes.
- The weights, thresholds and gates are chosen by the developers and are not learned from recruiter decisions. They were found to be stable under random changes of 20 percent, but they have not been calibrated against real data.
- The skill ontology contains 79 skills. Specialised vocabularies outside these skills are not recognised except through the free-text fallback.
- A skill is considered evidenced when it appears outside the skills block of the resume. This is a proxy for demonstrated experience and can be defeated by an adversary who repeats skills in several places.
- Blind screening removes direct identifiers. Proxy variables such as the name of an institution or a location are flagged but not corrected, and no group-fairness metric was computed because the pools carry no demographic labels.

- Certifications are counted but are not validated against the issuing body.
- The GitHub inspector can only judge public repositories. A missing or thin public profile is reported as insufficient data and is not treated as evidence of dishonesty.
- Compensation bands produced by the seniority agent are indicative planning ranges and are not market data.
- The deployed system runs on free cloud tiers that sleep when idle (the first request after a pause can take about 50 seconds) and it has not been load-tested.
- The 100-resume benchmark reuses 50 candidate names for different profiles, so a name alone does not identify a candidate in that pool; the chatbot asks which person is meant.

1.10 Project Outcomes

The project produced a working recruitment screening system with the following outcomes.

- A Python core library of about ten thousand lines containing every scoring algorithm, together with a FastAPI service, a SQLAlchemy data model with an Alembic migration, a Streamlit application and a React dashboard, with a MongoDB document store for accounts, sessions and candidate pools.
- An auditable skill ontology of 79 canonical skills with 353 alias entries, eight categories and fifteen related-skill pairs.
- A seven-factor ranker and a four-agent council that rank twelve pilot resumes in about 30 milliseconds each and place the strong test candidate first with a score of 88.1 percent, while rejecting the keyword-stuffed test resume that a keyword baseline scores at 47.1 percent.
- Name invariance: the score and the verdict did not change in 96 name-substitution runs, and conclusions were stable under random changes of the weights (mean Kendall rank correlation 0.969 over 200 draws).
- A suite of 78 automated tests (49 in Python and 29 in JavaScript) covering the parser, the ontology, the ranker, the council, the GitHub inspector, the talent search, the chatbot and the interview generator of the web interface, the

database, the authentication flow, the MongoDB-backed account and candidate API and the verification of Google sign-in tokens, all of which pass.

- A live deployment on free cloud tiers (the web interface on Vercel, the API on Render and the data in MongoDB Atlas) with server-side accounts, sessions and Google sign-in, so that the system can be demonstrated and evaluated by other users.

CHAPTER 2

LITERATURE SURVEY

A detailed literature survey was conducted to understand existing research on automated resume screening and candidate ranking. The reviewed works cover person–job fit and resume understanding, the extraction of skills from resumes and job postings, semantic and hybrid retrieval of text, and the use of developer profiles as evidence of skills, all of which relate to the proposed system. Each reviewed paper is described with its authors, its contribution and the limitation that is relevant to this project.

1. Matching GitHub Developer Profiles to Job Advertisements (2015)

Authors*: C. Hauff and G. Gousios*

The authors matched the public GitHub profiles of developers with job advertisements by extracting concepts from both and measuring their similarity. In that work, repository evidence is used to recommend developers; the present system uses it differently, to audit the languages that a candidate claims on a resume.

2. Enhancing person-job fit for talent recruitment: An ability-aware neural network approach (2018)

Authors*: C. Qin, H. Zhu, T. Xu, C. Zhu, L. Jiang, E. Chen, and H. Xiong*

The authors proposed an ability-aware neural network for person-job fit. Job requirements and the experiences of a job seeker are encoded at word level with recurrent networks, and a hierarchical attention mechanism measures how important each requirement is and how much each past experience contributes to each requirement. The attention weights make the matching partly interpretable. The model, however, is trained on historical hiring outcomes, and it does not check whether an ability claimed in a resume is backed by evidence in the rest of the document.

3. Person-Job Fit: Adapting the Right Talent for the Right Job with Joint Representation Learning (2018)

Authors*: C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li*

This work learns a joint representation of job postings and resumes with a convolutional neural network, so that a matching score can be computed directly from the text of both documents. The approach shows that semantic representations improve on

keyword matching, but the output is a score without a human-readable explanation, and it requires a large collection of labelled matches.

4. Towards Effective and Interpretable Person-Job Fitting (2019)

Authors*: R. Le, W. Hu, Y. Song, T. Zhang, D. Zhao, and R. Yan*

The authors addressed the lack of interpretability of neural person-job fitting models. Their model aligns the individual requirements of a job with the experiences of a candidate and allows the contribution of each pair to the prediction to be inspected. The study confirms that interpretability is a recognised requirement of recruitment models, although the explanation is limited to attention over text and does not cover skill gaps or bias.

5. End-to-End Resume Parsing and Finding Candidates for a Job Description using BERT (2019)

Authors*: V. Bhatia, P. Rawat, A. Kumar, and R. R. Shah*

This paper combined a resume parser with BERT-based sentence-pair classification to rank candidates for a job description, and reported 73 percent accuracy for candidate suitability. It demonstrates an end-to-end pipeline from parsing to matching. The result is a single similarity score; there is no skill-gap breakdown, no handling of uncertainty and no check for bias.

6. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks (2019)

Authors*: N. Reimers and I. Gurevych*

Sentence-BERT adds siamese and triplet network structures to BERT so that sentences are mapped to fixed-size vectors that can be compared with cosine similarity. The authors report that finding the most similar pair in a collection of ten thousand sentences, which takes about 65 hours with a standard BERT cross-encoder, takes about five seconds with Sentence-BERT. This efficiency makes sentence embeddings practical for matching resumes with job descriptions.

7. Retrieving Skills from Job Descriptions: A Language Model Based Extreme Multi-label Classification Framework (2020)

Authors*: A. Bhola, K. Halder, A. Prasad, and M.-Y. Kan*

The authors cast the retrieval of skills from job descriptions as an extreme multi-label classification problem solved with a language-model based framework. The method handles a large skill vocabulary, but it is probabilistic and requires training data, whereas

the present system uses an explicit, auditable ontology so that every credited skill can be traced to a rule.

8. Dense Passage Retrieval for Open-Domain Question Answering (2020)

Authors*: V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih*

Dense passage retrieval encodes questions and passages into dense vectors and retrieves by inner product. On open-domain question answering it outperformed a strong BM25 system by 9 to 19 percentage points in top-20 passage retrieval accuracy. Dense retrieval therefore handles paraphrase well, while sparse methods remain strong for rare exact terms, which motivates the hybrid search of the present system.

9. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)

Authors*: P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela*

Retrieval-augmented generation combines a pre-trained sequence-to-sequence model with a non-parametric memory, a dense index of documents, so that generated text is grounded in retrieved passages. The idea underlies the retrieval-grounded recruiter chatbot of this project, which answers from retrieved and scored resume evidence instead of free generation.

10. Skills prediction based on multi-label resume classification using CNN with model predictions explanation (2021)

Authors*: K. F. F. Jiechieu and N. Tsopze*

The authors framed the prediction of skills from a resume as a multi-label classification problem solved with a convolutional neural network, and added an explanation of the model predictions. This is useful for inferring skills that are not stated explicitly, but the inferred skills are probabilistic, whereas a screening decision must be traceable to explicit evidence.

11. Job Recommender Systems: A Review (2021)

Authors*: C. de Ruijt and S. Bhulai*

This review classifies job recommender systems into content-based, collaborative and hybrid approaches and discusses their data, evaluation and ethical problems. It shows that most recommenders optimise matching accuracy and give little attention to explanation or to the verification of claims.

12. JobBERT: Understanding Job Titles through Skills (2021)

Authors*: J.-J. Decorte, J. Van Hautte, T. Demeester, and C. Develder*

JobBERT learns representations of job titles by using the skills mentioned in job postings as supervision, which helps to normalise the many ways in which the same role is named. It shows the value of skills as a bridge between titles and requirements.

13. SkillSpan: Hard and Soft Skill Extraction from English Job Postings (2022)

Authors*: M. Zhang, K. N. Jensen, S. D. Sonniks, and B. Plank*

SkillSpan provides an annotated collection of English job postings in which hard and soft skills are marked as spans, and evaluates transformer models on the extraction task. It shows that skill extraction is a hard task even for strong models and that annotated data is scarce.

14. A Comprehensive Survey of Artificial Intelligence Techniques for Talent Analytics (2023)

Authors*: C. Qin, L. Zhang, Y. Cheng, R. Zha, D. Shen, Q. Zhang, X. Chen, Y. Sun, C. Zhu, H. Zhu, and H. Xiong*

This survey reviews artificial-intelligence techniques across the life cycle of talent management, including recruitment, matching and ranking. It records the shift from handcrafted features to deep models and discusses open challenges such as fairness and interpretability, which motivate the design of the present system.

15. Large Language Models as Batteries-Included Zero-Shot ESCO Skills Matchers (2023)

Authors*: B. Clavié and G. Soulié*

The authors showed that a large language model can map job-posting text to the skills of the ESCO taxonomy without task-specific training, by generating synthetic data and re-ranking candidates, and reported R-precision at ten of 61.0 and 68.9 on two evaluation sets. The approach is powerful but probabilistic and dependent on an external model, so a generative model could assert a skill that is not in the text.

2.1 Literature Survey Summary

Table 2.1: Summary of Reviewed Literature

Sl. No.	Author	Title	Year	Key Focus
1	C. Hauff and G. Gousios	Matching GitHub Developer Profiles to Job Advertisements	2015	Matching public GitHub profiles with job advertisements.
2	C. Qin, H. Zhu, T. Xu, C. Zhu, L. Jiang, E. Chen, and H. Xiong	Enhancing Person-Job Fit for Talent Recruitment: An Ability-Aware Neural Network Approach	2018	Ability-aware neural person–job fit with hierarchical attention.
3	C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li	Person-Job Fit: Adapting the Right Talent for the Right Job with Joint Representation Learning	2018	CNN-based joint representation of job postings and resumes.
4	R. Le, W. Hu, Y. Song, T. Zhang, D. Zhao, and R. Yan	Towards Effective and Interpretable Person-Job Fitting	2019	Interpretable alignment of job requirements with candidate experience.
5	V. Bhatia, P. Rawat, A. Kumar, and R. R. Shah	End-to-End Resume Parsing and Finding Candidates for a Job Description using BERT	2019	Resume parsing with BERT-based candidate ranking.
6	N. Reimers and I. Gurevych	Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	2019	Fast sentence embeddings compared by cosine similarity.
7	A. Bhola, K. Halder, A. Prasad, and M.-Y. Kan	Retrieving Skills from Job Descriptions: A Language Model Based Extreme Multi-label Classification Framework	2020	Extreme multi-label retrieval of skills from job descriptions.
8	V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih	Dense Passage Retrieval for Open-Domain Question Answering	2020	Dense retrieval that outperforms BM25 on paraphrased queries.
9	P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	2020	Generation grounded in retrieved passages.
10	K. F. F. Jiechieu and N. Tsopze	Skills Prediction Based on Multi-Label Resume Classification Using CNN with Model Predictions Explanation	2021	Multi-label CNN prediction of skills from resumes with explanations.
11	C. de Ruijt and S. Bhulai	Job Recommender Systems: A Review	2021	Review of content-based, collaborative and hybrid job recommenders.
12	J.-J. Decorte, J. Van Hautte, T. Demeester, and C. Develder	JobBERT: Understanding Job Titles through Skills	2021	Skill-supervised representations for normalising job titles.

Sl. No.	Author	Title	Year	Key Focus
13	M. Zhang, K. N. Jensen, S. D. Sonniks, and B. Plank	SkillSpan: Hard and Soft Skill Extraction from English Job Postings	2022	Annotated dataset and models for hard and soft skill extraction.
14	C. Qin, L. Zhang, Y. Cheng, R. Zha, D. Shen, Q. Zhang, X. Chen, Y. Sun, C. Zhu, H. Zhu, and H. Xiong	A Comprehensive Survey of Artificial Intelligence Techniques for Talent Analytics	2023	Survey of AI across recruitment, matching and talent management.
15	B. Clavié and G. Soulié	Large Language Models as Batteries-Included Zero-Shot ESCO Skills Matchers	2023	Zero-shot LLM mapping of job text to ESCO skills.

Table 2.1 presents a summary of the reviewed research papers, including their authors, titles, publication years and key focus areas. The survey shows that semantic and neural matching improves on keyword overlap but usually produces a score without a reason, that skill extraction and normalisation are difficult problems that benefit from an explicit vocabulary, that sparse and dense retrieval have complementary strengths, and that public repositories can provide evidence for skill claims. These observations guided the evidence-weighted hybrid ranker, the auditable skill ontology, the hybrid talent search and the GitHub authenticity inspector of the proposed system.

2.2 Research Gaps

The reviewed literature shows that existing matching systems are either accurate but opaque, or address one part of the problem, such as skill extraction or retrieval, in isolation. No reviewed system combines evidence-aware skill matching, understanding of related skills, verification of claims, blind screening and an explanation of every verdict. Table 2.2 lists the gaps that were identified and the way in which the proposed system responds to each of them.

Table 2.2: Research Gaps and the Response of the Proposed System

Research gap	Response of the proposed system
Neural person–job fit and BERT-based matching give a score without a reason and do not check whether a skill claim is supported by evidence.	Seven-factor hybrid score with a visible breakdown, and evidence-weighted skill coverage that gives full credit only to skills demonstrated in experience or projects.
Learned skill extraction and LLM-based skill matching are probabilistic, need training data or an external service, and do not credit related skills in an auditable way.	Skill ontology with aliases, fuzzy matching and an explicit table of related-skill pairs that earn half credit, reported as partial matches.

Research gap	Response of the proposed system
Dense retrieval handles paraphrase but can miss rare tool names, while sparse retrieval misses paraphrase.	Hybrid talent search that fuses dense (or TF-IDF) and BM25 scores, with evidence snippets for every result.
Retrieval-augmented generation grounds text in documents but has not been applied to explaining screening results to a recruiter.	Retrieval-grounded recruiter chatbot that answers only from the screened pool and the active role.
GitHub profiles have been used to recommend developers, not to audit the skills claimed on a resume.	GitHub authenticity inspector that verifies claimed languages and reports insufficient data instead of guessing.
The reviewed matching systems do not address identity bias, uncertainty or inconsistent employment timelines.	Blind screening, a four-agent council with confidence and human-review triggers, and explicit gates for inverted dates and concurrent employment.

CHAPTER 3

PRELIMINARIES

This chapter presents the fundamental concepts, technologies and techniques that are needed to understand the design and implementation of the AI Powered Resume Screening and Ranking System. It covers the programming ecosystem, the extraction of text from resumes, the vector representations used to compare documents (TF-IDF, BM25 and dense sentence embeddings), the skill ontology with fuzzy matching, the notions of explainability and fairness that guide the design, multi-agent deliberation, and the metrics used to evaluate the system.

3.1 Python and Its Ecosystem

Python is a high-level, interpreted, general-purpose programming language that is widely used for text processing, data analysis and machine learning. In this project Python implements the entire scoring core: parsing, ontology matching, ranking, the council, the GitHub inspector, the talent search and the interview generator. Its strengths for this work are a readable syntax, a rich standard library (regular expressions, sequence matching, dataclasses and HTTP clients), and a mature scientific ecosystem.

The libraries that matter most are scikit-learn, which provides the TF-IDF vectoriser and the cosine-similarity function; NumPy and pandas, which hold the score arrays and tabular analytics; the Hugging Face Transformers ecosystem from which the sentence-embedding model is loaded when it is installed; PyMuPDF and python-docx, which extract text from PDF and DOCX files and pytesseract, a wrapper around the Tesseract optical character recognition engine. Matplotlib was used to draw the charts of the evaluation chapter. The modular nature of Python allows the parsing, scoring and decision logic to be written as independent classes that can be tested one by one. The recruiter-facing parts rely on web technologies. A REST (Representational State Transfer) service exposes resources such as jobs, resumes and screenings through uniform HTTP operations; FastAPI implements this service in Python, and the React library renders the browser interface. The advantages of Python for the proposed system include:

- simple and readable syntax that shortens development and review;
- extensive libraries for text processing, retrieval and machine learning;

- support for modular, testable code with the pytest framework;
- easy integration with web frameworks and relational databases; and
- portability across Windows and Linux environments.

3.2 Resume Parsing and Text Extraction

A resume is an unstructured document. Before any scoring is possible, the text must be extracted and organised into fields. Documents in PDF format either contain a text layer, which can be read directly, or consist of scanned page images, which must be converted to text by optical character recognition. Tesseract recognises characters in an image in several stages: it finds text lines and words, recognises each word in two passes with an adaptive classifier, and applies linguistic analysis to the result. In the proposed system a page is sent to OCR only when its text layer contains almost no text, which keeps the common case fast.

After extraction, the text is divided into sections (skills, education, experience, projects, certifications) by recognising header patterns such as *Technical Skills* or *Work Experience*. Contact details are found with regular expressions. Employment dates are collected as intervals with a resolution of one month. If a date is written as a month and a year, it is converted to a month index m = 12y + mon, so that durations and overlaps become simple integer arithmetic. Total experience is the length of the union of all valid intervals, so that two overlapping roles are not counted twice:

$$E = \frac{1}{12}\left|\bigcup_{i=1}^{n} [s_i, e_i]\right| \tag{3.1}$$

where:

E total years of experience;

s_i, e_i start and end month indices of the i-th valid role;

n number of roles with a valid interval.

An interval with $e_i < s_i$, or one that starts in the future, is not valid. It is excluded from the experience total and reported as an inconsistency, because it signals either an error or an attempt to mislead. This simple temporal reasoning is invisible to text-similarity methods, which treat a date as just another token.

3.3 Vector Space Model and TF-IDF

The vector space model represents a document as a vector in which every dimension corresponds to a term. The importance of a term in a document is estimated by TF-IDF (term frequency, inverse document frequency) weighting. The idea, introduced as term specificity, is that a term which occurs often in a document is important for it, but a term that occurs in almost every document is a poor discriminator and should be down-weighted. The weight of a term t in a document d is:

$$w(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t) \tag{3.2}$$

$$\mathrm{idf}(t) = \ln \frac{1+N}{1+\mathrm{df}(t)} + 1 \tag{3.3}$$

where:

tf(t, d) number of occurrences of term t in document d;

df(t) number of documents that contain t;

N total number of documents.

Equation (3.3) is the smoothed form of idf used by scikit-learn, where the added constants avoid division by zero. The vectoriser used in this project removes English stop words, uses unigrams and bigrams, and keeps at most 3000 features. Two documents are compared by the cosine of the angle between their vectors:

$$\cos(u,v) = \frac{u \cdot v}{\|u\|\|v\|} \tag{3.4}$$

Cosine similarity ignores the length of the documents and takes values between 0 and 1 for non-negative TF-IDF vectors. TF-IDF is fast, needs no training and is fully reproducible, which is why it is the offline fallback for the semantic component of the ranker. Its weakness is that it is purely lexical: the sentences *built* REST *services* and developed HTTP API share no word, so their TF-IDF similarity is close to zero although their meaning is the same.

3.4 Dense Sentence Embeddings

Dense embeddings map text to vectors of a few hundred real numbers such that texts with similar meaning lie close together. Early word embeddings such as word2vec and GloVe gave every word one vector. Transformer models produce context-dependent representations, and BERT and its refinements became the standard encoders for language understanding. A plain BERT model compares two texts by feeding both

through the network at once, which is accurate but slow when many pairs must be compared. Sentence-BERT removes this cost by encoding each text once into a fixed-size vector, so that comparison reduces to a cosine similarity, Equation (3.4). The compact model all-MiniLM-L6-v2 used in this project is a distilled transformer that produces vectors with 384 dimensions and runs comfortably on a laptop CPU.

When the sentence-transformers library is installed the semantic score of a candidate is the cosine similarity between the embedding of the resume and the embedding of the job description. When it is not installed, the TF-IDF cosine is used instead. The interface reports which backend is active so that the recruiter always knows how the semantic part of a score was obtained.

Dense models also have drawbacks that matter in hiring. They can over-match generic text, they are harder to explain than lexical scores, and they inherit social bias from their training data: the associations learnt by word embeddings reproduce human-like stereotypes, and recent audits of embedding-based resume retrieval found strong preferences related to the name of the candidate. For this reason the semantic similarity carries only 20 percent of the overall score in the proposed ranker, and the system removes the name before any text is embedded when blind screening is on. Cross-encoder re-rankers are more accurate than bi-encoders for fine ranking, but they need a pair-wise pass for every candidate and are listed as future work.

3.5 Okapi BM25 and Hybrid Retrieval

BM25 is a probabilistic ranking function that improves on TF-IDF by saturating the effect of term frequency and by normalising for document length. For a query q and a document d it is defined as:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{idf}_{\mathrm{BM}}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d)+k_1\left(1-b+b\frac{|d|}{\mathrm{avgdl}}\right)} \tag{3.5}$$

$$\mathrm{idf}_{\mathrm{BM}}(t) = \ln\left(1+\frac{N-n_t+0.5}{n_t+0.5}\right) \tag{3.6}$$

where:

- $f(t, d)$ frequency of term t in document d;
- $|d|$, $avgdl$ length of d and average document length in the collection;
- k_1, b tuning parameters; this project uses $k_1 = 1.5$ and $b = 0.75$;
- n_t number of documents that contain t.

Dense retrievers outperform sparse ones when the query and the document use different words for the same idea. In one study, dense passage retrieval beat a strong BM25 system by 9 to 19 percentage points in top-20 accuracy. Sparse methods remain better at rare, exact terms such as the name of a tool, for example *Kubernetes* or *PyTorch*. Because a recruiter query often mixes concepts and tool names, the proposed talent search fuses both kinds of evidence at the level of a resume chunk:

$$s(q,c) = \alpha s_{\text{dense}}(q,c) + (1-\alpha)s_{\text{BM25}}(q,c), \alpha = 0.55 \quad (3.7)$$

Here c is a chunk of a resume (a section or part of a section), and both scores are normalised to the range 0 to 1 before they are combined. An alternative way of fusing rankings, reciprocal rank fusion, adds 1/(k + rank) over the lists to be combined; score fusion was preferred here because both scores are available and calibrated per chunk. Retrieval-augmented systems ground their answers in retrieved text, and the recruiter chatbot of this project follows the same principle: it composes answers from retrieved and scored resume evidence and not from free text generation. Figure 3.1 shows the flow of a talent-search query.

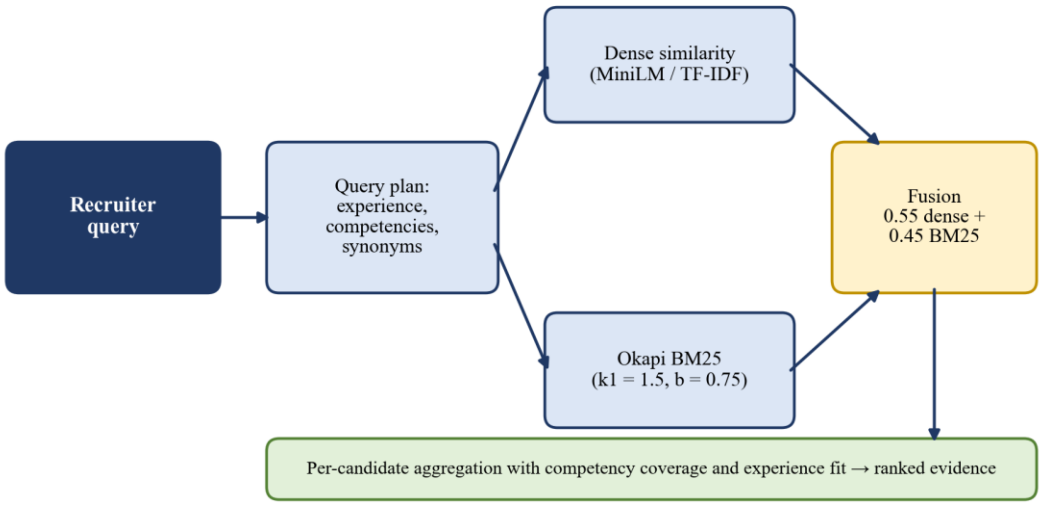


Figure 3.1: Hybrid retrieval flow of the talent search: query understanding, dense and BM25 scoring, fusion and per-candidate aggregation.

Table 3.1: Comparison of Sparse and Dense Representations

Property	TF-IDF / BM25 (sparse)	Sentence embeddings (dense)
Matches	Exact and stemmed terms	Meaning, paraphrases
Rare tool names	Strong	Can be weak
Paraphrase	Weak	Strong
Training	None	Pre-trained model needed

Property	TF-IDF / BM25 (sparse)	Sentence embeddings (dense)
Explainability	High, term by term	Low
Bias risk from training data	Low	Present
Offline use	Always	Needs the model files

3.6 Skill Ontology, Fuzzy Matching and Partial Credit

A skill ontology is a controlled vocabulary in which every skill has one canonical name and a set of aliases. Aliases capture the many ways in which the same skill is written, for example *sklearn*, *scikit learn* and *Scikit-Learn*, or *k8s* and *Kubernetes*. Normalising every skill mentioned in a resume or a job description to its canonical name is a prerequisite for comparing them. Large taxonomies such as ESCO have thousands of skills and can be matched with language models, but such matching is probabilistic. The ontology of this project is deliberately small (79 skills) and every credit given by it can be traced to an explicit rule.

The normalisation of a skill string proceeds in three steps: an exact alias lookup, a lookup after stripping punctuation, and finally a fuzzy match against all aliases. The fuzzy step uses the similarity ratio of the sequence matcher of Python's standard library, which is based on the longest common contiguous blocks of the two strings (the Ratcliff-Obershelp idea):

$$\text{ratio}(a, b) = \frac{2M}{|a|+|b|} \tag{3.8}$$

where:

M number of characters in the matching blocks of the strings a and b;

$|a|, |b|$ lengths of the two strings.

A string is mapped to an alias only when the ratio exceeds 0.88, so that a small spelling error is tolerated but unrelated skills are not merged. Edit distance in the sense of Levenshtein counts the minimum number of insertions, deletions and substitutions needed to turn one string into another; the sequence-matching ratio used here is a related, similarity-based measure that is faster to compute for short strings.

After normalisation every required skill is classified as *matched* if the candidate has the same canonical skill, *partially matched* if the candidate has a skill that is related to it through an explicit table of fifteen pairs (for example FastAPI and Flask, Docker and

Kubernetes, or Machine Learning and Deep Learning), and *missing* otherwise. A matched skill earns full credit and a partially matched skill earns half credit. The skill coverage of a candidate is therefore:

$$C_{\text{skill}} = \frac{n_m + 0.5 n_p}{n_r} \tag{3.9}$$

where:

- n_m number of matched required skills;
- n_p number of partially matched required skills;
- n_r total number of required skills.

A skill that is claimed is not necessarily a skill that is possessed. To reflect this, the proposed system distinguishes the required skills that appear anywhere in the resume (set L) from those that also appear outside the skills block, in experience or project text (set E). Evidence-weighted coverage gives half credit to a skill that is only listed:

$$c_{\text{ev}} = \frac{|E| + 0.5(|L| - |E|)}{|R|} \tag{3.10}$$

where:

- R set of required skills of the job;
- $L \subseteq R$ required skills found anywhere in the resume;
- $E \subseteq L$ required skills that are also evidenced in experience or project text.

The ratio of listed skills that have no supporting evidence, (|L| − |E|)/|L|, is used to detect keyword stuffing. Figure 3.2 summarises the decision flow of the skill matcher.

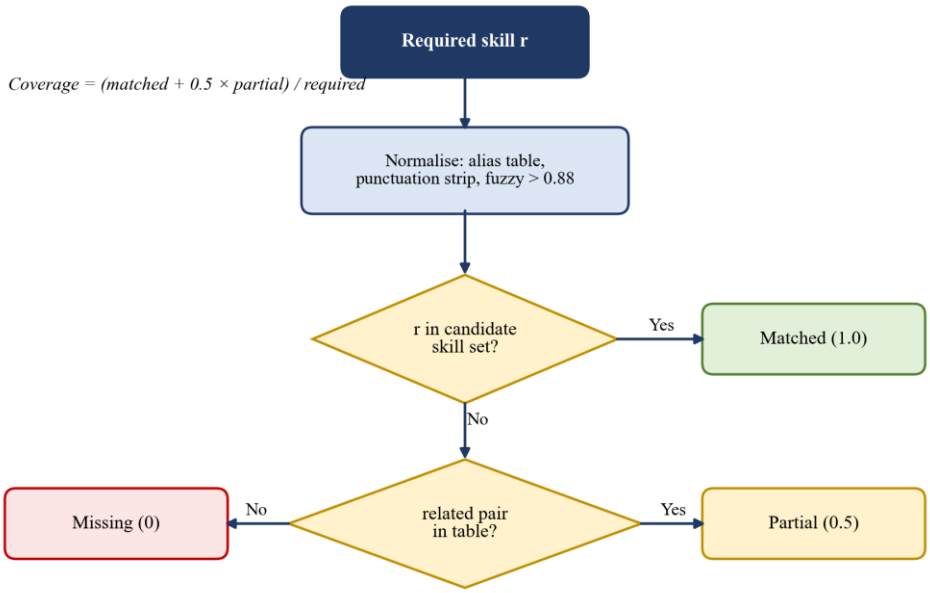


Figure 3.2: Decision flow of the skill matcher: normalisation, exact match, related-pair match and missing skill.

3.7 Explainability and Fairness

Explainable AI

Explainable artificial intelligence (XAI) is concerned with making the output of an automated system understandable to the people who use it or are affected by it. Research in the social sciences shows that people expect explanations to be contrastive, selective and tied to causes that they can act upon. Methods for explaining a black-box model after the fact, such as LIME and SHAP, approximate the behaviour of a complex model around one prediction and are surveyed in the literature. A different position holds that for high-stakes decisions an interpretable model should be used from the start instead of explaining a black box, and that the quality of an explanation has to be evaluated rigorously. The proposed system follows the second position: it uses transparent rules and weighted sums whose parts can be shown to the user, and it produces explanations (reasons, strengths, warnings, skill gaps and red flags) as part of the computation instead of reconstructing them afterwards.

Fairness

Fairness in algorithmic decision-making has several, partly conflicting definitions. Individual fairness requires that similar individuals are treated similarly. Group criteria such as equality of opportunity require equal error rates across protected groups, and it is known that several natural group criteria cannot all hold at once when base rates differ. Surveys list many sources of bias, from the data to the model to the way results are used. For rankings, FA*IR tests whether a protected group is sufficiently represented in every prefix of a top-k list. Applied to hiring, the literature shows that fairness cannot be reduced to a statistical test, and that legal notions of discrimination, vendor practice and technical metrics differ.

The proposed system uses three complementary measures. First, blind screening removes direct identifiers (name, e-mail, phone, location, profile links and gendered words) before any agent evaluates the resume. Second, a Fair-Hiring Auditor agent computes a merit-only score from job-related evidence and can reduce the consensus weight of an agent whose reasoning shows bias. Third, a counterfactual name-swap test checks that the score does not change when only the name is replaced, following the design of the classical audit studies. The invariance is measured as:

$$\Delta_{\text{score}} = max_{n \in \mathcal{N}} |S(x_n) - S(x)| \tag{3.11}$$

where:

- x original resume;
- x_n the same resume with the name replaced by n in the name field and in the text;
- $S(\cdot)$ score or council consensus of a resume.

Good practice in documentation, such as model cards and datasheets for datasets, and internal algorithmic auditing, recommends that the intended use, the data and the limitations of a system are recorded. Finally, the regulatory background includes the EU Artificial Intelligence Act, which treats recruitment systems as high-risk, and the General Data Protection Regulation, which governs the processing of the personal data found in resumes.

3.8 Multi-Agent Deliberation

A multi-agent system divides a complex judgement among several specialised components, each of which examines the problem from one point of view. With language models, it has been shown that letting several model instances propose answers and then criticise each other improves factuality and reasoning, that teams of referees judge text closer to human annotators than a single referee, and that agent frameworks can coordinate several roles to solve tasks. Two further ideas are often combined with such systems: interleaving reasoning with actions and sampling several reasoning paths and taking the consistent answer. A caution comes from the study of language models used as judges, which can be biased by the position of an answer or by its length.

The council of this project borrows the structure of these systems, which is independent evaluation followed by cross-examination and synthesis, but not their non-determinism. Each agent is a transparent rule-based evaluator that can run offline. An optional language-model service may enrich the text of an agent and may move its score by at most 10 points, but the final verdict is always computed locally, so that it remains auditable. A fourth agent, the Fair-Hiring Auditor, reviews the other three. This design keeps the benefit of multiple viewpoints without making the decision depend on a generative model.

3.9 Evaluation Metrics

The agreement between two rankings of the same candidates, for example the ranking of the keyword baseline and the ranking of the proposed ranker, is measured with Kendall's rank-correlation coefficient:

$$\tau = \frac{n_c - n_d}{\binom{n}{2}} \tag{3.12}$$

where:

- n_c number of concordant pairs, which are ordered the same way in both rankings;
- n_d number of discordant pairs;
- n number of ranked items.

A value of 1 means identical rankings and -1 means reversed rankings. To test whether the conclusions depend on the expert-chosen weights, each weight is multiplied

by a random factor and renormalised, and the resulting ranking is compared with the default ranking:

$$\tilde{w}_i = \frac{w_i u_i}{\sum_k w_k u_k}, u_i \sim \mathcal{U}[0.8, 1.2] \tag{3.13}$$

The quality of a search result list is measured with precision and recall at k, and with the normalised discounted cumulative gain:

$$\mathrm{P@}k = \frac{|\mathrm{Rel} \cap \mathrm{Top}_k|}{k}, \mathrm{R@}k = \frac{|\mathrm{Rel} \cap \mathrm{Top}_k|}{|\mathrm{Rel}|} \tag{3.14}$$

$$\mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}, \mathrm{DCG@}k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{3.15}$$

where:

Rel set of relevant items;

Top_k the first k items returned;

rel_i graded relevance of the item at position i;

IDCG@k DCG@k of the ideal ordering.

The agreement between the verdict of the system and a reference label is shown in a confusion matrix, whose entry in row i and column j counts the items of reference class i that received verdict j. Processing time is reported per resume and per stage. As Manning, Raghavan and Schütze point out, the choice of metric must follow the use of the system, which is why a ranking metric, a name-invariance measure and a verdict-agreement matrix are all reported here.

CHAPTER 4

REQUIREMENT SPECIFICATIONS

This chapter describes the fundamental requirements, functional and non-functional, of the AI Powered Resume Screening and Ranking System. It also specifies the hardware, software and user requirements that are necessary for the development and operation of the system. The requirements were derived from the weaknesses of the Phase 1 keyword ATS, from the gaps identified in the literature survey, and from the needs of a recruiter who must be able to explain every decision.

4.1 Functional Requirements

Figure 4.1 shows the context of the system: the recruiter supplies a role, a job description and resumes, and the system returns a ranked and explained shortlist together with supporting outputs.

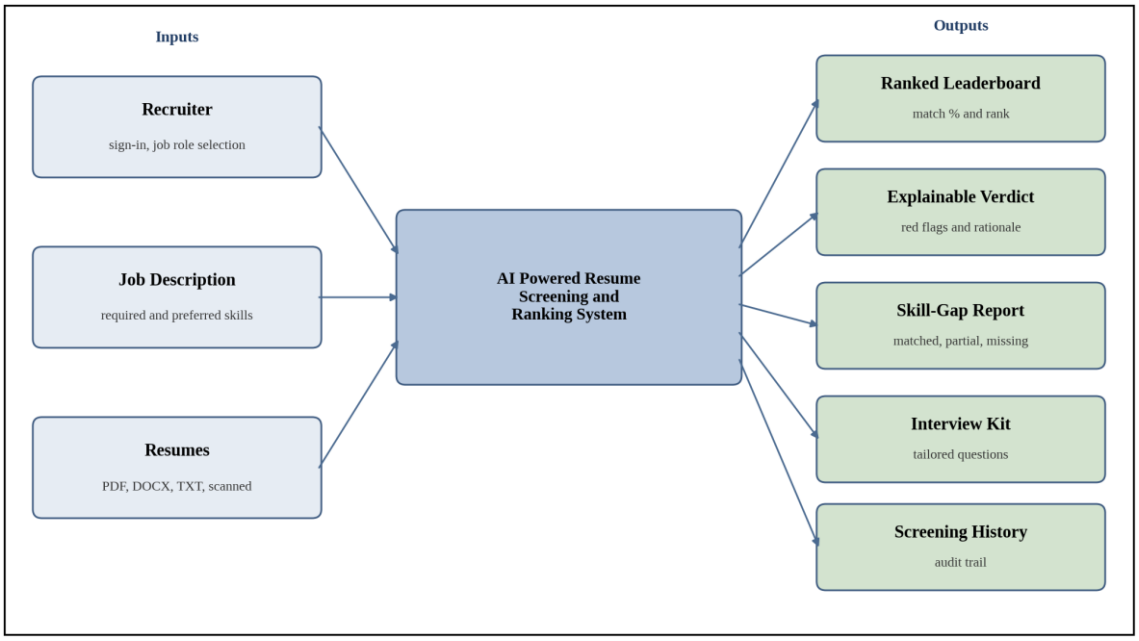


Figure 4.1: Context diagram of the AI Powered Resume Screening and Ranking System.

The system shall provide the following functions:

1. **Resume input:** accept resumes in PDF, DOCX and plain-text formats, and apply OCR to scanned PDF pages.
2. **Resume parsing:** extract contact details, profile links, skills, education, experience, projects and certifications from every resume.

3. **Job-description handling:** parse a job description into required skills, preferred skills, minimum experience and preferred degrees, and allow the recruiter to select a pre-configured role or to create a custom role.
4. **Skill normalisation and gap analysis:** normalise skills through the ontology and report matched, partially matched (related skill, half credit, with the reason) and missing skills for every candidate.
5. **Hybrid ranking:** compute the seven-factor hybrid score and rank all candidates of a pool in descending order of score.
6. **Evidence weighting:** give half credit to a skill that appears only in the skills list, and report the unevidenced-skill ratio.
7. **Timeline checks:** exclude inverted or future date ranges from the experience total and report them as inconsistencies, and raise a high-severity flag when two full-time on-site roles in different countries overlap by six months or more.
8. **Blind screening:** remove names, contact details, locations, profile links and gendered words before any agent evaluates a resume.
9. **Council review:** evaluate every candidate with four agents and return a verdict, a confidence value, red flags, dissent and human-review triggers; apply knock-out gates for low evidence coverage, high-severity flags and missing education; record the cross-examination so that it can be replayed; and show an inconsistency, a red flag and a verdict separately.
10. **GitHub verification:** verify the programming languages claimed on a resume against public GitHub repositories when a profile is available, and report insufficient data instead of guessing.
11. **Recruiter tools:** provide a leaderboard, candidate dossier, side-by-side comparison, analytics page and report export, and generate a tailored interview kit of at least 15 questions for every candidate.
12. **Recruiter chatbot:** answer natural-language questions about the pool with evidence, resolve follow-up questions, page long lists, ask which candidate is meant when a name is ambiguous, and draft outreach messages.
13. **Accounts and access:** require sign-in before the workspace is shown, keep accounts and sessions on the server, lock an account for 15 minutes after five

failed sign-ins without revealing whether an e-mail address is registered, and verify Google sign-in on the server (Gmail accounts only).

14. **API and persistence:** expose jobs, resumes, screenings and audit logs through a REST API, store screening sessions with the recruiter decision, and serve the candidate pools of 12, 50 and 100 resumes from a database.
15. **Fault tolerance:** continue the screening when an optional component (embedding model, language model or network service) fails.

4.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes that the system must satisfy:

- **Usability:** the recruiter workflow is organised as three guided steps (choose a role, load candidates, review results); every score is shown with its breakdown and every verdict with its reasons; matched, partial and missing skills are visually distinguished; and failed operations produce understandable messages.
- **Reliability:** the remaining resumes are still screened when one resume cannot be parsed; optional components fall back to local alternatives (TF-IDF for embeddings, rule-based text for the narrative, a structured report for GitHub failures); duplicate resumes are detected through their SHA-256 hash; and the decision logic is deterministic.
- **Performance:** a complete screening (parsing, ranking and council review) should take less than 100 milliseconds per resume on a laptop CPU in the offline configuration (about 30 milliseconds was measured), and the search index is cached per resume pool.
- **Security and privacy:** passwords are stored as salted PBKDF2-HMAC-SHA256 digests; session tokens are random, stored only as SHA-256 digests and expire after seven days; secrets are read from environment variables; and personal data is redacted before any text is sent to an optional language-model service.
- **Fairness and explainability:** name, gender, age, nationality, location, institution prestige and employment gaps do not enter any score; invariance to the name of

the candidate is verified by a test; and every automated verdict is explained and open to human review.

- **Portability and design constraints:** the Python core runs on Windows and Linux with Python 3.11 and is independent of the user interface; the system works without network access; weights and thresholds are configurable; and the dense model is optional so that the API fits the 512 MB memory limit of a free hosting tier.
- **Maintainability:** the code is divided into independently testable modules for parsing, ontology, ranking, council, inspection, search and interface; an automated test suite is run before every release; and database changes are managed by migration scripts.

4.3 Hardware Requirements

The minimum recommended hardware for developing and running the system is listed below.

- **Processor:** dual-core processor or higher; the evaluation was carried out on a laptop CPU in a single process.
- **Memory (RAM):** 4 GB minimum; 8 GB or more is recommended when the dense embedding model is installed.
- **Storage:** at least 5 GB of free space for the source code, the dependencies, the sample resume pools and the optional model files.
- **Display:** a standard monitor or laptop display of at least 1366 by 768 pixels.
- **Input devices:** keyboard and mouse or touch-pad.
- **Network:** not required for the offline configuration; an internet connection is needed only for the optional GitHub inspector, the language-model service, the Google sign-in, the managed database and the deployed web site.

The actual hardware requirements depend on the size of the resume pool and on whether the optional sentence-embedding model is installed.

4.4 Software Requirements

The software required to develop and run the system is summarised below.

- **Operating system:** Windows 10 or later, or Ubuntu Linux.

- **Programming languages:** Python 3.11 for the core and the API; JavaScript (JSX) for the web interface.
- **Web browser:** a modern browser such as Google Chrome or Microsoft Edge.
- **Code editor:** Visual Studio Code or another suitable editor.
- **Version control:** Git, with the source code hosted in a GitHub repository.
- **Hosting services (optional):** Vercel for the web interface, Render for the API, MongoDB Atlas (free M0 tier) for the database, and Streamlit Community Cloud for the Streamlit application.

The main libraries and frameworks used are listed below.

- **Machine learning:** scikit-learn, NumPy and pandas for TF-IDF, cosine similarity and array and table processing.
- **Dense embeddings (optional):** sentence-transformers with the all-MiniLM-L6-v2 model for semantic similarity.
- **Document parsing:** PyMuPDF, python-docx, pytesseract and Pillow for text extraction from PDF and DOCX files and OCR of scanned pages.
- **Web API:** FastAPI, Uvicorn and pydantic-settings for the REST service with automatic documentation.
- **Databases:** SQLAlchemy 2.0 with Alembic migrations (SQLite locally, PostgreSQL through psycopg), and MongoDB Atlas with PyMongo for accounts, sessions and candidate pools.
- **User interfaces:** Streamlit with Plotly for the recruiter workspace; React 19 with Vite, Tailwind CSS 4, lucide-react, pdfjs-dist and mammoth for the web dashboard.
- **Language model (optional):** google-generativeai for narrative enrichment of agent output.
- **Testing:** pytest and mongomock for the Python core and the API, and the Node.js test runner for the web-interface logic.

The availability of these packages must be verified before the system is run. The lean API requirements file omits the dense embedding model, so that the service fits in the memory of a free-tier host.

4.5 User Requirements

The user requirements describe the basic operations and information that a recruiter expects from the system.

Screening Workflow

- The recruiter should be able to sign up, to sign in with a Gmail address and a password or with a Google account, and to see the three-step workspace.
- The recruiter should be able to select one of the eight pre-configured roles or create a custom role and edit its competencies.
- The recruiter should be able to upload resumes or load a ready-made pool of 12, 50 or 100 resumes, and to start the screening engine with one action.
- The recruiter should be able to see the ranked candidates, the score breakdown, the verdict and the red flags.

Review and Decision

- The recruiter should be able to open a candidate dossier, read the scorecards of the four agents and replay the debate.
- The recruiter should be able to compare two candidates side by side and to ask the chatbot questions about the pool.
- The recruiter should be able to turn blind screening on or off and to export a report of the session.
- The recruiter should be able to record a shortlist or reject decision, which is stored with a time stamp.

Configuration, Logs and Error Information

- The recruiter should be able to adjust the weights of the ranker through the configuration of the service.
- The recruiter should be able to review the screening history and the audit log of recruiter actions.
- The system should give meaningful error messages when a file cannot be read, and should distinguish between a missing optional component and a failure of the core.

CHAPTER 5

SYSTEM DESIGN

System design defines the architecture, the components, the interfaces and the data of the system, and the way in which they interact to satisfy the specified requirements. It gives a structured representation of the proposed system and explains how its modules work together. This chapter presents the high-level design (workflow, architecture and data flow), the design of the four-agent council, the use-case diagram, the sequence diagram, the database design and the interface design of the AI Powered Resume Screening and Ranking System.

5.1 High-Level Design

The high-level design describes the overall workflow of the system, from the choice of a role by the recruiter to the decision on a candidate. The recruiter selects one of the pre-configured roles or pastes a job description, which the job-description parser converts into required skills, preferred skills, a minimum number of years of experience and preferred degrees. The recruiter then uploads resumes or loads a ready-made pool. Each resume is parsed into structured fields; scanned pages are sent to OCR.

If blind screening is enabled, the parsed profile is anonymised: the name is replaced by a neutral identifier such as *Candidate #001*, contact details, location and profile links are masked, and gendered words are neutralised. Only this anonymised copy is passed on. The skill ontology then normalises the skills and classifies each required skill as matched, partially matched or missing. The hybrid ranker computes seven scores and their weighted sum, and the four-agent council reviews every candidate, applies the knock-out gates and issues a verdict with red flags, a confidence value and, where needed, a request for human review. The explanations, the skill-gap analysis, the interview questions and the leaderboard are generated from these results and the screening history is saved. The recruiter reviews the shortlist and records a decision. The complete flow is shown in Figure 5.1.

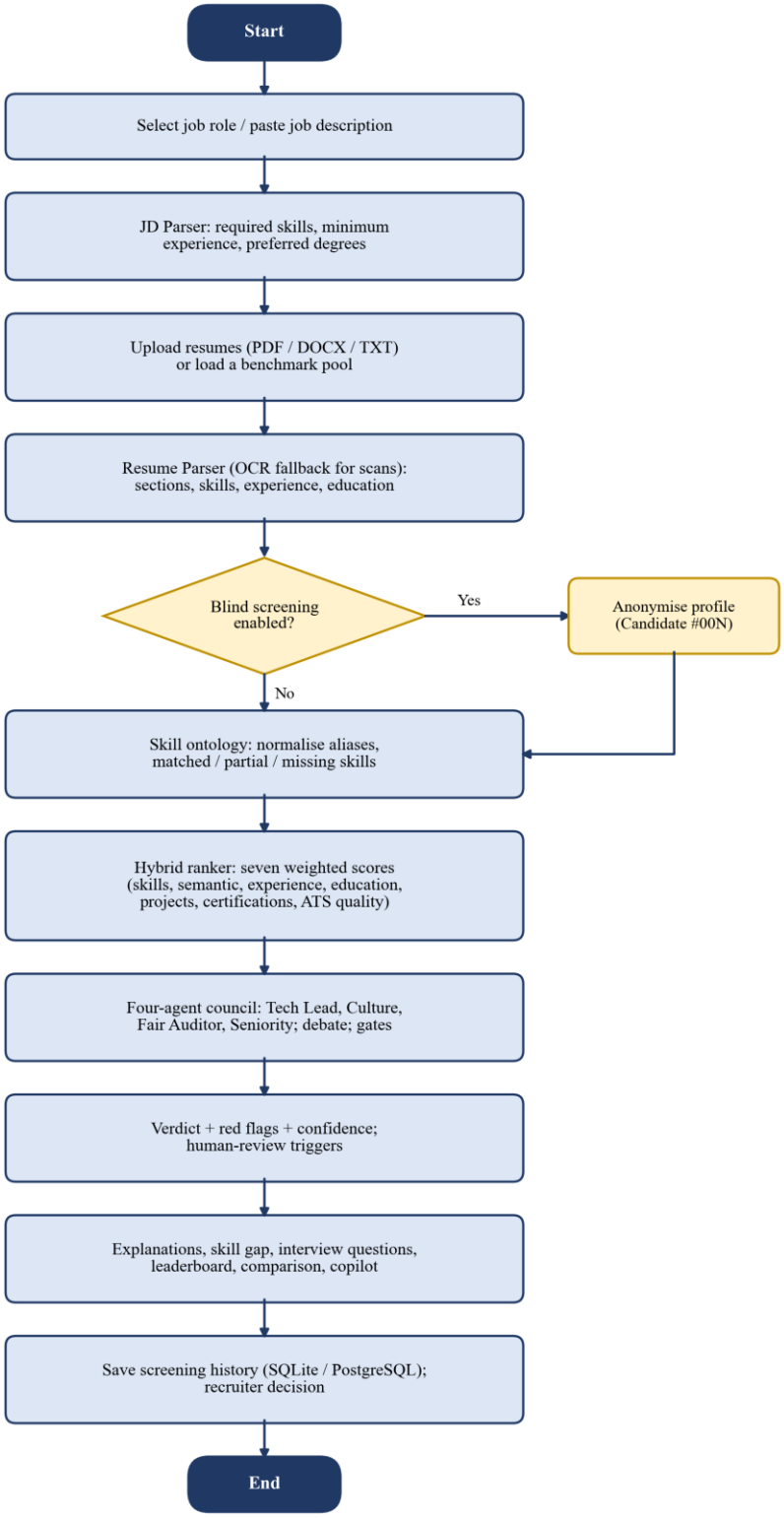


Figure 5.1: System workflow diagram of the AI Powered Resume Screening and Ranking System.

Figure 5.1 shows the sequence of stages from the selection of a role to the recorded decision, and the optional anonymisation branch.

5.2 System Architecture

The system follows a layered architecture. The presentation layer consists of the React web application (Nexus ATS) and the Streamlit workspace. The service layer is a FastAPI REST service that exposes jobs, resumes, screenings and audit logs, and that orchestrates the parsing, ranking, explanation and anonymisation components through service classes. The intelligence engine is a framework-free Python library in which every scoring algorithm is implemented: the resume parser, the skill ontology, the hybrid ranker, the council, the talent search, the GitHub inspector and the interview generator. The data layer comprises a relational database (SQLite for local use) for jobs, resumes, screening sessions and the audit log, a MongoDB Atlas document store for the user accounts, the sessions and the candidate pools of the deployed system, and the role definitions. The layers are shown in Figure 5.2.

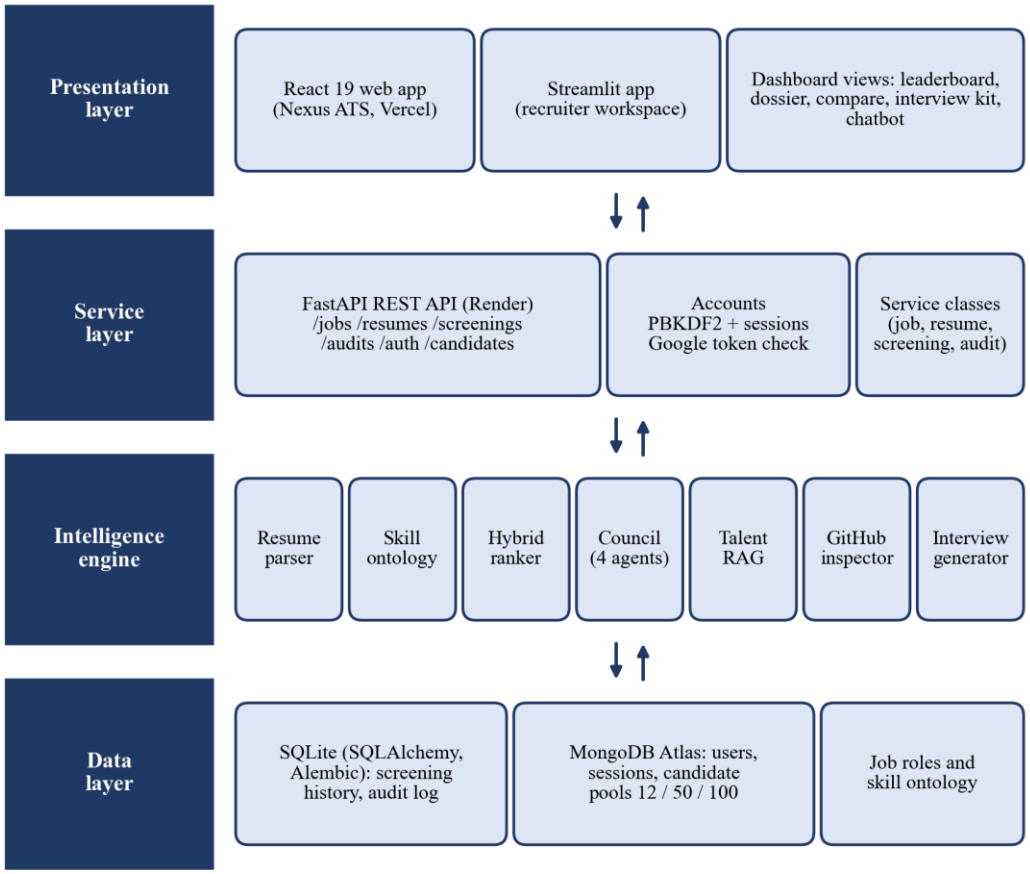


Figure 5.2: System architecture of the AI Powered Resume Screening and Ranking System.

The React application can run in a stand-alone mode with bundled sample candidates, so that the system can be demonstrated without any server, or it can be connected to the API through environment variables. Its data layer is organised as replaceable service modules for authentication, candidates, GitHub and e-mail, which are selected by configuration, so that a persistent store or an external service can be attached without changing the components. In the deployed configuration the application is connected to the API through three configuration values (the API address, the authentication provider and the candidate provider): accounts and sessions are then created and checked by the API, and the candidate pools are read from the database.

5.3 Data Flow Diagram

The data flow of the system is shown in Figure 5.3 as a level-1 diagram. The recruiter supplies a job description and resumes to the parsing process (1.0), which reads the job roles from the data store D1 and the resumes from D2. The parsed profiles pass to the optional anonymisation process (2.0), whose output, the blind profiles, enters the skill-matching and ranking process (3.0). The ranked scores go to the council review process (4.0), which writes the screening history to D3 and returns the verdict, red flags and explanations to the recruiter. Before any of these processes can be used, the recruiter signs in, and the accounts and sessions are read from and written to the data store D4. In the deployed system D2 and D4 are collections of the MongoDB database.

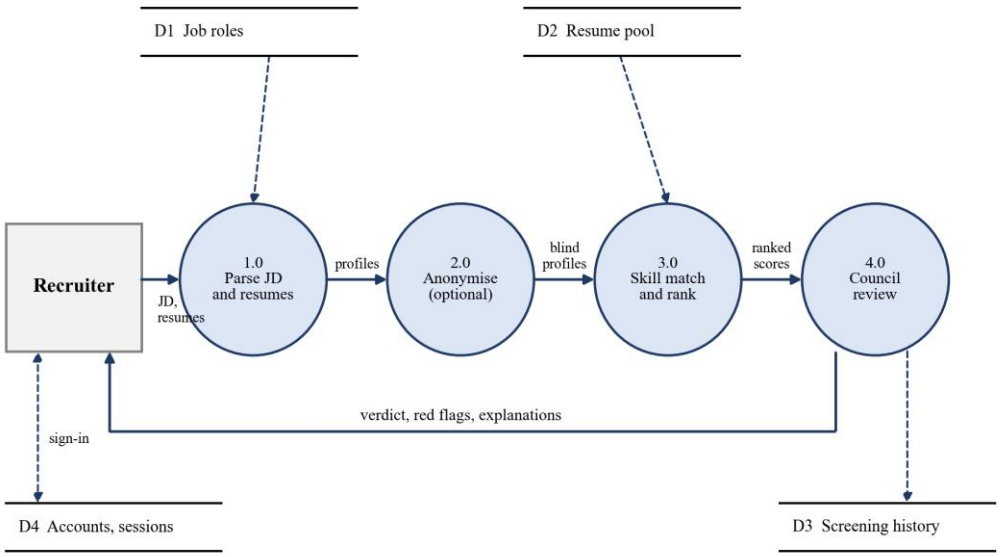


Figure 5.3: Level-1 data flow diagram of the system.

5.4 Design of the Four-Agent Council

The council evaluates a candidate in rounds. In the first round the Tech Lead, the Culture and Retention agent and the Seniority and Compensation agent score the candidate independently from a common set of features. An optional enrichment step may let a language model improve the text of these scorecards, but it can move a score by at most 10 points. In the second round the Fair-Hiring Auditor reviews everything that has been written so far, including any model-written text, scans it for biased language, and computes a merit-only score. In the third round the agents cross-examine each other in a deterministic, data-driven transcript. Finally an executive consensus is synthesised: a weighted score, a verdict, red flags, confidence and review triggers. The sequence is shown in Figure 5.4.

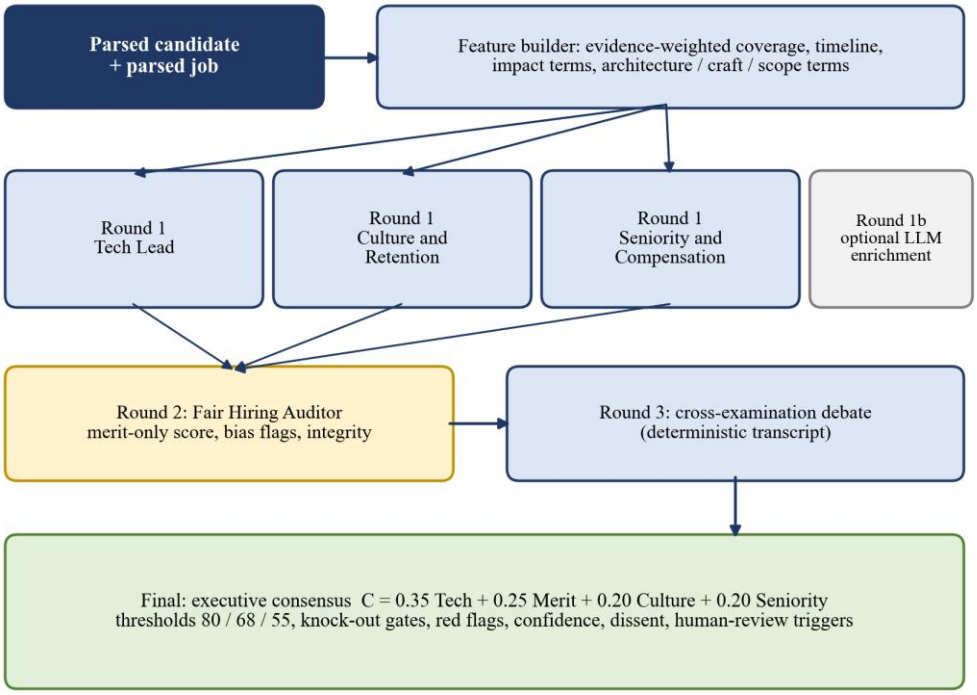


Figure 5.4: Orchestration of the four-agent council.

5.5 Use-Case Diagram

A use-case diagram represents the interaction between the actors of a system and the functions that the system provides. The primary actor of the system is the recruiter, who signs up, signs in with a password or a Google account, selects or creates a role, loads resumes, runs the screening engine and reviews the results. The review includes the leaderboard, the candidate dossier with the council debate, the side-by-side comparison,

the interview questions, the analytics page and the chatbot. The recruiter can also turn blind screening on and off and can record a shortlist or reject decision. The GitHub service and the language-model provider are optional external actors that support the inspection of a dossier and the answers of the copilot. The use cases are shown in Figure 5.5.

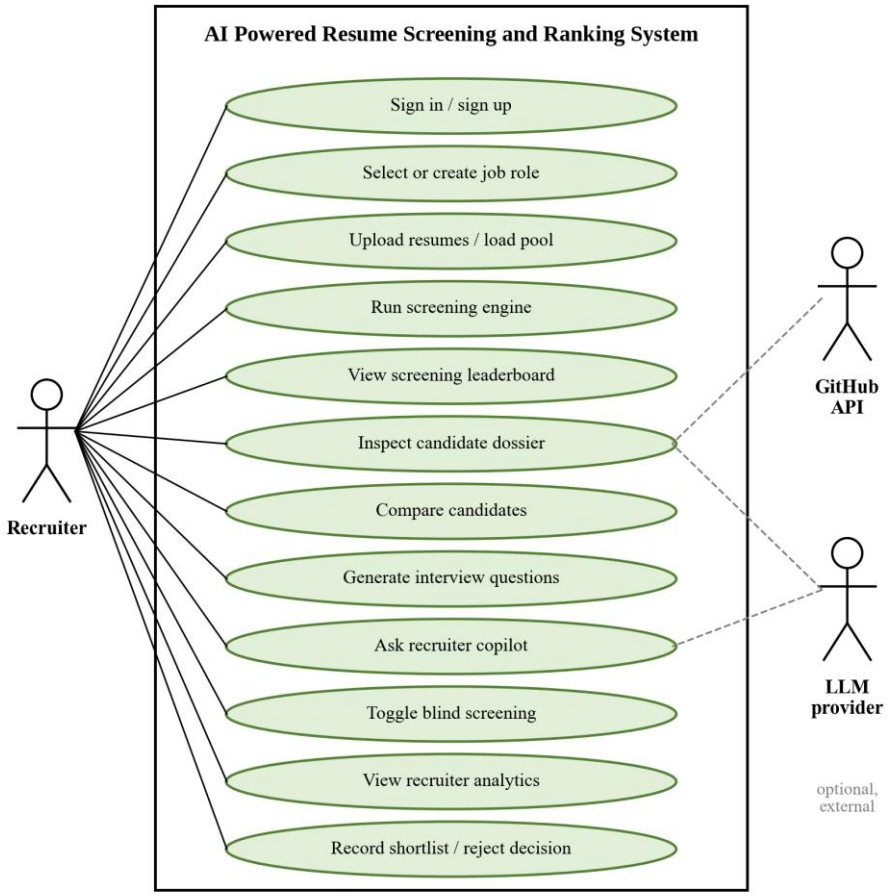


Figure 5.5: Use-case diagram of the system.

Figure 5.5 shows that every function is initiated by the recruiter; the external actors are optional and the system remains usable without them.

5.6 Sequence Diagram

A sequence diagram shows the order in which the components interact to complete an operation. In the screening sequence the recruiter chooses a role and uploads resumes through the web interface. The interface asks the resume parser to parse every file and receives structured profiles. It then asks the ranker, which uses the ontology, to score and rank the profiles, and receives the ranked list with the score breakdown. For each candidate that the recruiter opens, the interface asks the council to evaluate the candidate,

and receives the verdict, the flags and the confidence. The session and the results are saved in the database. The recruiter reviews the leaderboard and the dossier and records a decision, which is stored together with an audit-log entry. The sequence is shown in Figure 5.6.

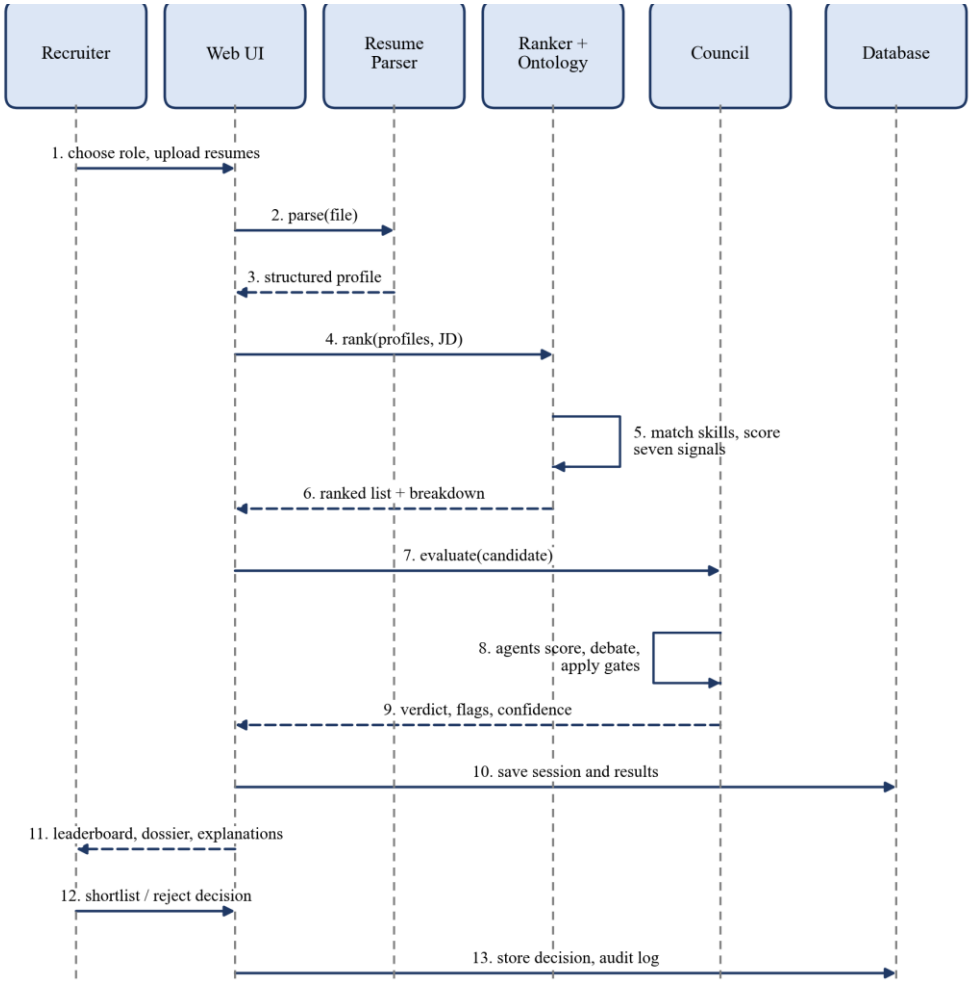


Figure 5.6: Sequence diagram of a screening session.

The diagram is a conceptual representation. Actual function calls and communication paths may differ between the Streamlit application, the React interface and the API, but the order of the stages is the same.

5.7 Database Design

The system uses two stores. The screening data of the REST service is stored in a relational schema that is defined with SQLAlchemy and created by an Alembic migration, and the user accounts, the sessions and the candidate pools of the deployed system are stored in a MongoDB document store. Five tables are used. The *jobs* table stores the role definition: title, department, minimum experience, the required and

preferred skills and the preferred degrees as JSON lists, and the description text. The *resumes* table stores the file name, the SHA-256 hash of the file, which is unique and is used to reject duplicates, the raw text, the parsed data as JSON and the main contact fields. The *screening_sessions* table represents one batch run of a job and stores the recruiter, the weights that were used and a time stamp. The *screening_results* table stores, for each candidate of a session, the overall score, the rank, the score breakdown, the skill-gap result and the candidate data as JSON, and the decision (shortlist or reject) with its time stamp. The *audit_logs* table is an append-only record of actions with the actor and a JSON description of the details. Figure 5.7 shows the entities and their relationships.

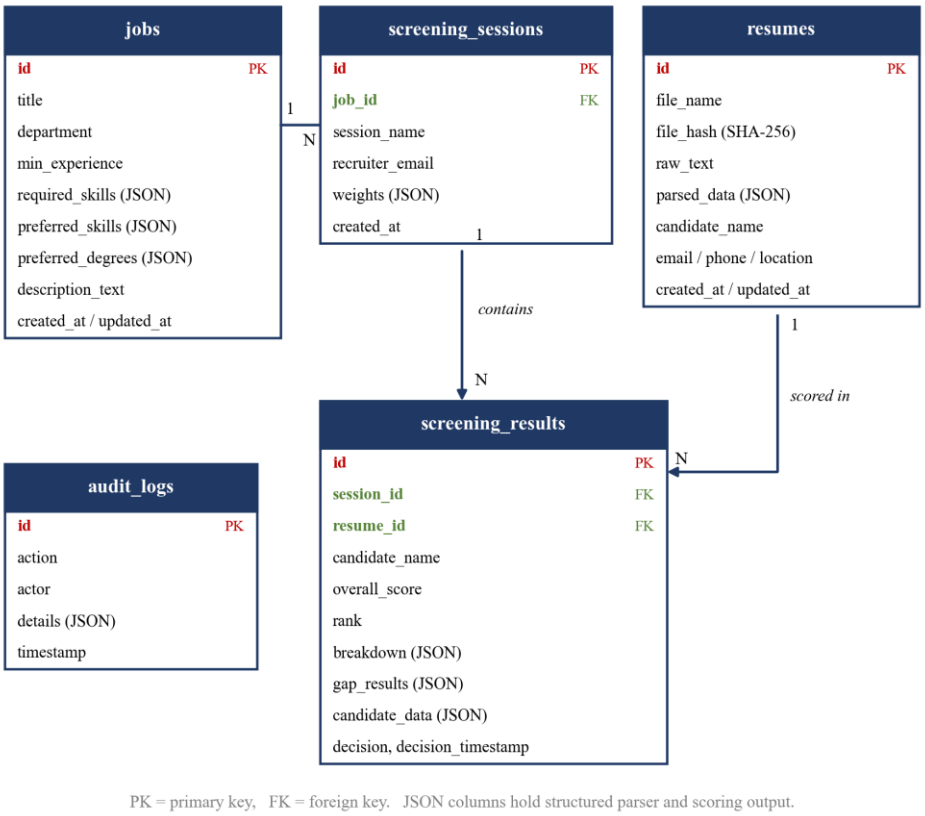


Figure 5.7: Entity-relationship diagram of the database.

A job has many screening sessions, a session contains many results, and a resume can be scored in many results, which allows the same candidate to be screened for several jobs. Deleting a job deletes its sessions and results through cascading rules, while deleting a resume leaves the results in place with the resume reference set to null.

REST Interface

The service is versioned with the prefix /api/v1 and is documented automatically at /docs; the account endpoints (/auth) and the candidate endpoint (/candidates) are mounted without the version prefix, because the web interface calls them directly. Table 5.1 lists the endpoints.

Table 5.1: REST Endpoints of the Service

Method	Endpoint	Purpose
POST	/api/v1/jobs	Create a job with required and preferred skills
GET	/api/v1/jobs, /api/v1/jobs/{job_id}	List jobs or read one job
PATCH, DELETE	/api/v1/jobs/{job_id}	Update or delete a job
POST	/api/v1/resumes/upload	Upload a resume, parse it and reject duplicates by SHA-256
GET	/api/v1/resumes, /api/v1/resumes/{resume_id}	List resumes or read one resume
POST	/api/v1/screenings/run	Run a screening of selected resumes against a job, optionally in blind mode
GET	/api/v1/screenings, /api/v1/screenings/{session_id}	List sessions or read a session with its ranked results
PATCH	/api/v1/screenings/results/{result_id}/decision	Record a shortlist or reject decision
GET, POST	/api/v1/audits	Read or write the audit log
POST	/auth/signup	Create an account (Gmail address, name, company, role and password)
POST	/auth/signin	Check the password and return a session token
POST	/auth/google	Verify a Google ID token on the server and return a session token
GET	/auth/me	Return the signed-in user
POST	/auth/signout	End the session
GET	/candidates?pool=	Return the candidate pool core12, benchmark50 or benchmark100
GET	/health, /	Health check and service information

Document Store (MongoDB)

The accounts, the sessions and the candidate pools of the deployed system are stored in a MongoDB Atlas database, which was chosen because these records are independent documents with a flexible structure and because the free tier is persistent. The database holds three collections, which are created together with their indexes when the API starts. Table 5.2 lists them.

Table 5.2: Collections of the MongoDB Document Store

Collection	Content	Keys and indexes
users	name, e-mail (lower case), company, role, password salt, hash and iteration count, failed-sign-in counter, lock-until time, Google identifier and picture, creation time	e-mail is unique; an account created with Google has no password hash
sessions	SHA-256 digest of the session token, user reference, creation time and expiry time	token digest is unique; a time-to-live index removes a session when it expires (seven days)
candidates	one document per candidate with scores, verdict, skills, red flags, GitHub summary and council scorecards, the pool name and its order	the pair (pool, candidate id) is unique; pools are core12 (12), benchmark50 (50) and benchmark100 (100)

When the API starts it creates the indexes and fills any candidate pool that is still empty from the bundled seed files, so that a restart never duplicates data. If the database address is not configured, the account and candidate endpoints answer with a service-unavailable error and the rest of the API keeps working.

Authentication and Sessions

The authentication design protects the accounts against the usual attacks on a sign-in form. It is implemented by the account endpoints of the API and is summarised below.

- **Sign-up:** the API checks the name, the e-mail address (Gmail only), the company, the role and the password length (at least eight characters), stores the e-mail in lower case, and rejects a second account with the same address.
- **Password storage:** the password is hashed with PBKDF2-HMAC-SHA256 using 210,000 iterations and a random 16-byte salt, and is compared in constant time.
- **Sign-in:** an unknown e-mail address and a wrong password produce the same message, and a hash is computed in both cases so that the response time does not

reveal which addresses exist. After five wrong passwords the account is locked for 15 minutes.

- **Sessions:** a successful sign-in returns a random token. Only its SHA-256 digest is stored, so a copy of the database cannot be used to sign in, and the session expires after seven days or at sign-out.
- **Google sign-in:** the browser obtains a Google ID token and the API verifies it with Google: the audience must be the client identifier of the project, the issuer must be Google, the token must not have expired, and the e-mail must be verified and must end with gmail.com. A new user is stored without a password; if the same address already has a password account, the Google identity is linked to it.
- **Interface protection:** the credential fields of the forms block copy, cut, paste and drop but accept the autofill and the strong-password suggestion of a password manager, and the sign-in form starts empty after a logout.

5.8 Interface Design

The interface design describes how the recruiter interacts with the system. The web interface is organised as a left navigation with seven pages (Job Management, Screening Leaderboard, Candidate DeepDive, Candidate Comparison, Interview Assistant, Recruiter Analytics and Recruiter Chatbot), a top bar with a search field, the blind-screening switch and the signed-in user, and a content area. Access is controlled by a single sign-in and sign-up card, shown in Figure 5.8. The card accepts a Gmail address and a password or a Google account; its credential fields block copy and paste but accept the autofill of a password manager, and it is empty again after a logout.

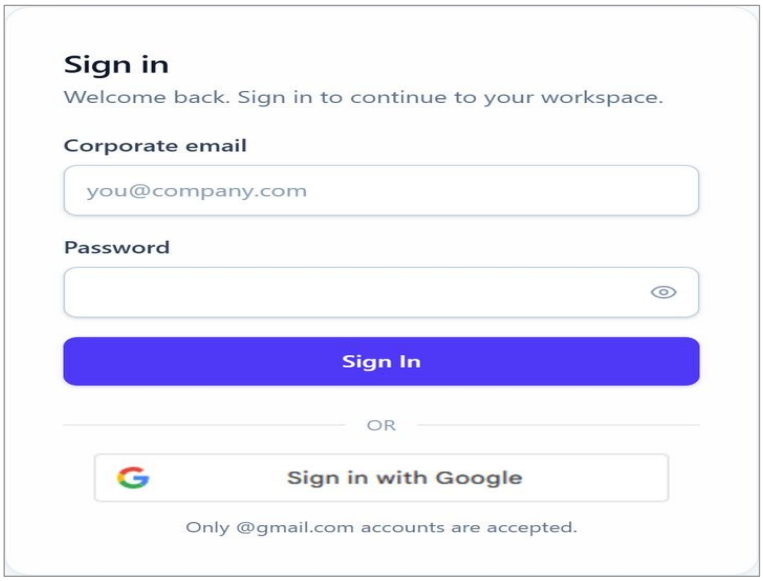


Figure 5.8: Sign-in card of the web interface.

The home page follows the principle of progressive disclosure: it is organised as three guided steps, an active-role summary with the full competency editor behind a disclosure control, candidate intake by upload or by one-click benchmark pools, and a launchpad for the next actions. All controls of a flat layout remain available, but their visibility is staged so that a new user is not overwhelmed. Figure 5.9 shows the workspace.

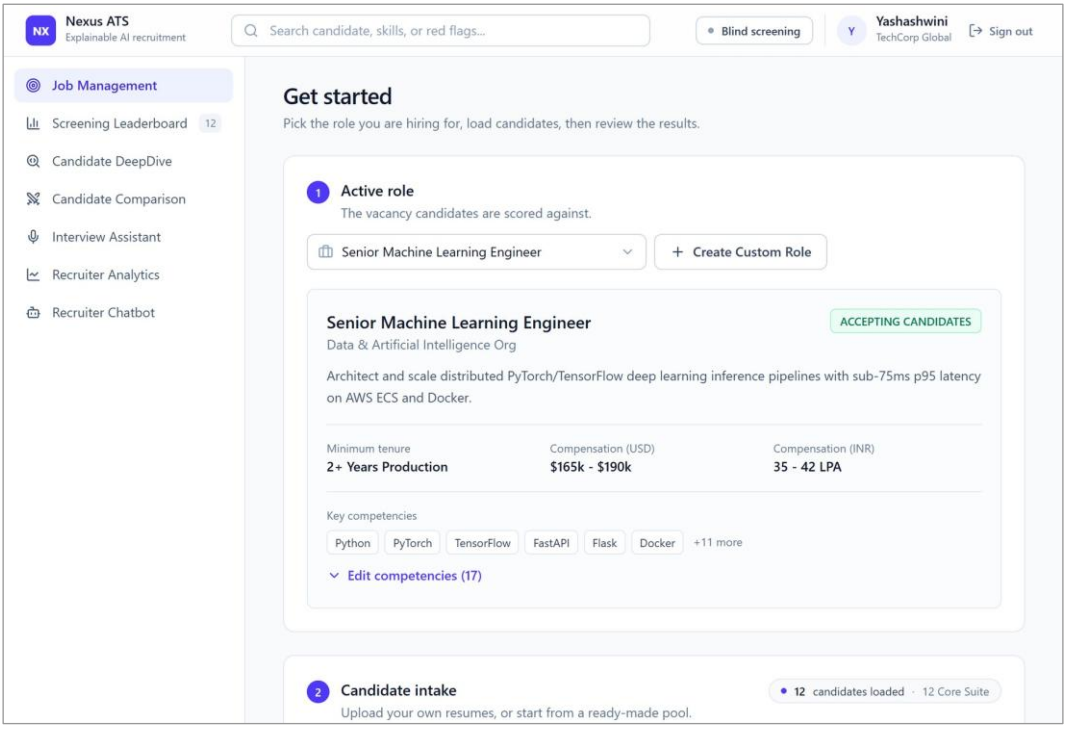


Figure 5.9: Workspace of the web interface: active role and candidate intake.

The leaderboard presents the telemetry of the run (candidates screened, top match, model backend and active role), a search bar for natural-language questions with prompt shortcuts, and ranked candidate cards with a score bar, an evidence excerpt, matched and missing skills, the number of red flags and the verdict. Cards can be switched to a compact table. The dossier of a candidate shows the four council scorecards, the estimated level and the compensation band, and a replayable debate.

The interview assistant shows a kit of about twenty questions for the selected candidate, each with a category badge and a note on what the interviewer should look for. The kit can be filtered by category and copied as text. The recruiter chatbot keeps the context of the conversation, so that a question such as *why?* or *should I hire her?* refers to the candidate that was discussed last, shows long lists five at a time, and offers clickable follow-up questions below each answer. The whole conversation can be exported as a Markdown file.

The interface distinguishes between a detected inconsistency, a red flag and a verdict, and displays each of them separately, so that a recruiter can see which fact led to which decision.

CHAPTER 6

IMPLEMENTATION

This chapter describes the technologies, the modules and the procedures involved in the implementation of the AI Powered Resume Screening and Ranking System. The system is implemented in Python for the scoring core and the API and in JavaScript for the web interface. Every module of the core is described with the formulas that it computes and with short excerpts of the source code. The chapter also discusses the difficulties met during development and the strategies used to overcome them, and ends with pseudocode of the overall screening pipeline.

6.1 Overview of Technologies Used

The system combines programming languages, libraries and hosting services. Each technology supports a particular responsibility, including document parsing, text analysis, ranking, persistence and presentation.

1. **Python 3.11:** the primary language of the core library. It implements the parser, ontology, ranker, council, inspector, search and generator, and the API.
2. **scikit-learn, NumPy and pandas:** scikit-learn provides the TF-IDF vectoriser and cosine similarity; NumPy holds score arrays and the BM25 index; pandas supports tabular analytics.
3. **sentence-transformers (optional):** loads the compact model all-MiniLM-L6-v2 to compute 384-dimensional embeddings. When it is not installed, the system falls back to TF-IDF.
4. **PyMuPDF, python-docx and pytesseract:** extract text from PDF and DOCX files and perform OCR on scanned pages.
5. **FastAPI, Uvicorn and Pydantic:** implement the REST API with automatic documentation, request validation and typed schemas.
6. **SQLAlchemy, Alembic and SQLite:** the object-relational model, schema migrations and the local database.
7. **MongoDB Atlas and PyMongo:** the managed document database that stores the accounts, the sessions and the candidate pools of the deployed system.

8. **Streamlit and Plotly:** the first recruiter workspace and its interactive charts.
9. **React 19, Vite and Tailwind CSS 4:** the web interface (Nexus ATS). The libraries pdfjs-dist and mammoth extract text from PDF and DOCX files inside the browser, and lucide-react provides the icons.
10. **Google Identity Services:** optional sign-in with Google, restricted to Gmail accounts; the ID token is verified on the server.
11. **pytest:** the automated test suite.
12. **Git, GitHub, Vercel, Render and MongoDB Atlas:** version control and the hosting of the web interface, of the API and of the database.

These technologies are used together to build the screening pipeline: documents are parsed in Python, scored by the ranker and the council, stored through SQLAlchemy or MongoDB and presented through React or Streamlit.

6.2 Implementation Details of Modules

The system is divided into modules with a single responsibility each. The modules are described in the order in which they are used in a screening.

Module 1: Resume Parser

The resume parser converts a file into a structured profile. For PDF files, PyMuPDF reads the text layer page by page. If the text of a page is shorter than 30 characters, the page is regarded as scanned, rendered at 200 dots per inch and passed to Tesseract OCR; a document that yields almost no text at all is processed entirely by OCR. DOCX files are read with python-docx, and plain text files are decoded directly. The text is then cleaned and segmented into sections by header patterns.

The parser extracts the following information:

- name, e-mail address, telephone number, location, and LinkedIn and GitHub links;
- education: degree, branch, institution, CGPA or marks and year;
- experience: total years (the union of valid month intervals, Equation 3.1), detected job titles and the timeline of roles;
- projects: title, description and the skills used; and
- certifications and the list of skills, normalised through the ontology.

Dates are read at month resolution. A role with an end date before its start date, or with a start date in the future, is excluded from the experience total and recorded as an inconsistency. Valid overlapping roles are merged so that the same month is never counted twice. The parser returns a dictionary that contains these fields together with the cleaned raw text and the sections, which is the input of every later module.

Module 2: Job-Description Parser

The job-description parser converts the text of a vacancy into evaluation criteria. The job title is taken from a hint or from one of the first lines that contains a role word such as *engineer*, *developer* or *analyst*. All skills of the text are extracted by phrase-boundary matching against the ontology. If the text contains a section introduced by words such as *preferred*, *good to have* or *nice to have*, the skills of this section become preferred skills and the others become required skills. The minimum experience is read from patterns such as *3+ years*, and the preferred degrees from patterns such as *B.E.*, *M.Tech* or *Ph.D.*. The pre-configured roles of the interface use the same structure, with 17 required skills for the machine-learning role used in the evaluation.

Module 3: Skill Ontology and Skill-Gap Analyser

The ontology holds 79 canonical skills in eight categories, with 353 entries in its alias lookup table (canonical names and aliases). Every skill is stored with its aliases and its category. At start-up, one regular expression per alias is compiled, with the longest aliases first, so that *machine learning* is found before *learning*. Short aliases of up to three characters, such as *C*, *Go* or *ML*, use strict boundaries so that they are not found inside other words. The function that matches the required skills of a job with the skills of a candidate is shown in Listing 6.1.

Listing 6.1: Core of the skill matcher (abridged from skill_ontology.py)

```
def match_skills(jd_skills, resume_skills):
    norm_jd = [normalize_skill(s) or s.strip() for s in jd_skills]
    set_resume = {normalize_skill(s) or s.strip() for s in resume_skills}
    matched, partially_matched, missing = [], [], []
    for req_skill in norm_jd:
        if req_skill in set_resume:
            matched.append(req_skill)
        else:
            for res_skill in set_resume:
                if (req_skill, res_skill) in related_pairs or \
                   (res_skill, req_skill) in related_pairs:
                    partially_matched.append({
                        "required_skill": req_skill,
                        "related_candidate_skill": res_skill})
                    break
            else:
                missing.append(req_skill)
```

```
    raw = (len(matched) * 1.0 + len(partially_matched) * 0.5) / len(norm_jd)
    match_percentage = round(min(1.0, raw) * 100, 1)
```

The result of the matcher feeds the skill-gap analyser, which groups the skills by category and computes a coverage percentage for each category with the same credit rule (1.0 for a match, 0.5 for a partial match). The category scores are displayed in the dossier of a candidate, so that the recruiter can see, for instance, that a candidate is strong in programming languages but weak in cloud and DevOps.

Module 4: Hybrid Ranker

The hybrid ranker computes seven scores in the range 0 to 100 and combines them as a weighted sum. If a user supplies custom weights, they are renormalised so that their sum is one.

$$S = \frac{\sum_{i=1}^{7} w_i s_i}{\sum_{i=1}^{7} w_i}, w = (0.35,0.20,0.15,0.10,0.10,0.05,0.05) \quad (6.1)$$

where:

- s_1 technical skill score S_{tech} (the partial-credit match percentage, w = 0.35);
- s_2 semantic job-description score S_{sem} (w = 0.20);
- s_3 experience score S_{exp} (w = 0.15);
- s_4 education score S_{edu} (w = 0.10);
- s_5 project score S_{proj} (w = 0.10);
- s_6 certification score S_{cert} (w = 0.05);
- s_7 resume quality score S_{ats} (w = 0.05).

Semantic score. When the dense model is available, the score is the cosine similarity of the embeddings of the resume and the job description, multiplied by 100 and limited to the range 0 to 100. When it is not available, the TF-IDF cosine similarity is used, with unigrams and bigrams, English stop words and at most 3000 features. TF-IDF cosine values of two documents usually lie between 0.3 and 0.7, so the value is multiplied by 150 to bring it to a comparable scale:

$$S_{\text{sem}} = min(100, max(0,100 \cdot \cos_{\text{dense}})) \text{or} S_{\text{sem}} = min(100,150 \cdot \cos_{\text{tfidf}}) \quad (6.2)$$

Experience score. The score compares the years of experience y with the minimum y_{min} of the job. When the minimum is met, the score is 85 plus 5 points for every surplus year,

up to 15 points; otherwise it is proportional to the shortfall, with a floor of 25. If the job states no minimum, a fixed value is used:

$$S_{\text{exp}} = \begin{cases} min(100,85 + 5(y - y_{min})), & y \geq y_{min} > 0 \\ max(25,80y/y_{min}), & 0 < y < y_{min} \\ 90 \text{ if } y > 0, \text{ otherwise } 80, & y_{min} = 0 \end{cases} \quad (6.3)$$

Education score. A base score is assigned from the highest parsed degree and is increased by a bonus for a related branch and for a high CGPA, with the sum limited to 100 (Table 6.1).

Table 6.1: Education Score Components

Item	Condition	Points
Base score	Ph.D.	100
	Master's degree (M.E., M.Tech, M.S.)	95
	Bachelor's degree (B.E., B.Tech)	88
	BCA or MCA	85
	Secondary school	45
	Other or unrecognised	75
Bonus	Related branch (computer, information, AI, data science, electronics)	+5
	CGPA of 8.5 or more (or 85 percent or more)	+7
	CGPA of 7.5 or more (or 75 percent or more)	+3

Project and certification scores. The project score rewards the number of projects, with up to 60 points, and the share of the required skills that are used in the projects, with up to 40 points. A candidate without projects receives 40. The certification score depends on the number of certifications and is 40 when there is none:

$$S_{\text{proj}} = min(100, max(20, min(60,30 + 15n_p) + 40a)), S_{\text{cert}} = min(100,60 + 15n_c) \quad (6.4)$$

where:

- n_p number of projects;
- a share of the required skills that appear in the skills used by the projects;
- n_c number of certifications.

Certifications are counted and not validated against an issuer, which is a limitation. The implementation of the weighted sum is shown in Listing 6.2.

Listing 6.2: Weighted sum of the seven scores (hybrid_ranker.py)

```
w = self.weights
total_weight = sum(w.values())
norm_w = {k: v / total_weight for k, v in w.items()}

overall_score = (
    skill_score    * norm_w["technical_skills"] +
    semantic_score * norm_w["semantic_jd"] +
    exp_score      * norm_w["experience"] +
    edu_score      * norm_w["education"] +
    proj_score     * norm_w["projects"] +
    cert_score     * norm_w["certifications"] +
    quality_score  * norm_w["resume_quality"]
)
overall_score = round(max(0.0, min(100.0, overall_score)), 1)
```

Module 5: Resume Quality and ATS Analyser

The quality analyser measures how well a resume can be read by an automated system and how convincing its content is. It returns a score from 10 to 100, a rating (Excellent for 85 or more, Good for 70 or more, Fair for 50 or more, otherwise Needs Work), lists of good points and improvements, and personalised suggestions. The score is the sum of the components of Table 6.2, from which up to 10 points are deducted for impossible or non-chronological date ranges.

Table 6.2: Components of the Resume Quality Score

Component	Maximum	How points are given
Contact hygiene	20	E-mail 6, telephone 6, LinkedIn link 4, GitHub or portfolio link 4
Section structure	25	Skills 7, education 6, projects 6, experience 6 (4 for a fresher)
Action verbs	20	20 for five or more verbs such as built, deployed or optimised; 12 for two to four; otherwise 5
Quantified impact	20	20 for four or more measurable results (percentages, users, latency); 12 for one to three; otherwise 4
Length and readability	15	15 for 200 to 800 words; 8 if longer; 6 if shorter
Date integrity	-10	5 points deducted for every inverted or impossible date range, at most 10

Module 6: Explainable-AI Module

The explanation module turns the numbers of the ranker into a statement that a recruiter can read. It assigns one of four recommendation tiers by the overall score: *Strongly Recommended* for 80 or more, *Recommended* for 65 or more, *HR Review* for 50 or more and *Not Recommended* below 50. It then lists the reasons (for example, the number of required skills matched and the coverage percentage, the years of experience against the requirement, the number of projects), the strengths (for instance, strong competency in the primary skills when coverage is 75 percent or more), the warnings (for

instance, missing required skills or an experience shortfall) and an executive summary. Partially matched skills are explained by naming the related skill of the candidate.

Module 7: Blind-Screening Filter

The blind-screening filter creates an anonymised copy of a profile. The name is replaced by a neutral identifier of the form *Candidate #001*; the e-mail address, the telephone number, the location and the profile links are replaced by masks; every occurrence of the real name in the text is replaced by the identifier; and gendered pronouns and honorifics are neutralised (for example *he* and *she* become *they*, and *Mr.* and *Ms.* are removed). The council receives only this copy, so the removal takes place at the data layer before any score is computed. Direct identifiers are removed; proxy variables such as the name of an institution are not removed but are reported by the auditor. Listing 6.3 shows the core of the filter.

Listing 6.3: Core of the blind-screening filter (abridged from bias_filter.py)

```
blind = copy.deepcopy(candidate_data)
masked_id = f"Candidate #{candidate_index:03d}"
blind["candidate_name"] = masked_id
blind["email"] = "[redacted]@candidate.id"
blind["phone"] = "+[redacted]"
blind["location"] = "Location Redacted (Fair Hiring)"
raw_text = re.sub(re.escape(real_name), masked_id, raw_text, flags=re.IGNORECASE)
for pattern, replacement in GENDER_PRONOUNS:      # he/she -> they, mr/ms -> ''
    raw_text = re.sub(pattern, replacement, raw_text, flags=re.IGNORECASE)
```

Module 8: Four-Agent Council

The council is the decision module of the system. A common feature builder first reads the candidate profile and the job description and computes the quantities on which all agents rely: the evidence-weighted coverage c_{ev} of Equation 3.10, the unevidenced-skill ratio, a month-aware timeline of roles, the number of quantified results, and the number of architecture, craftsmanship, scope and deployment terms. A resume skill counts as evidenced when its alias occurs in the text after the skills block has been removed.

Tech Lead agent. The agent estimates technical depth as a weighted sum of six normalised signals:

$$a_{\text{tech}} = 100(0.40c_{\text{ev}} + 0.20d + 0.10k + 0.10i + 0.10m + 0.10b) \qquad (6.5)$$

where:

- c_{ev} evidence-weighted required-skill coverage (0.65 if the job has no parsed skills);
- d architecture depth, $\min(1, n_{arch}/6)$;
- k craftsmanship, $\min(1, n_{craft}/4)$;
- i quantified impact, $\min(1, n_{impact}/3)$;
- m repository maturity, $0.5 \min(1, n_{rich}/3) + 0.5 \min(1, n_{shipped}/2)$, where n_{rich} counts projects with a description of at least 80 characters and $n_{shipped}$ counts projects that mention deployment; when a GitHub report exists, m is replaced by $0.7\,m + 0.3\,g/100$ with g the authenticity score;
- b breadth of tools, $\min(1, n_{skills}/12)$.

The agent also raises concerns, for example when more than 25 skills are listed (keyword stuffing) or when 60 percent or more of the listed skills have no supporting evidence.

Culture and Retention agent. The agent looks at the career timeline. It starts from 68 points and applies the adjustments of Table 6.3. Employment gaps are reported but never lower the score, because gaps have many legitimate causes. If no dated full-time role is found, the score is a neutral 62 with low confidence, and if only one full-time role exists, the score is capped at 78 because progression cannot be assessed.

Table 6.3: Adjustments of the Culture and Retention Agent

Signal	Adjustment
Average tenure of 30 months or more	+14
Average tenure of 18 to 29 months	+8
Average tenure below 12 months	-12
Each completed role shorter than 10 months	-8 (at most -24)
Each step up in title	+9 (at most +18)
Each move to a lower title band	-4
Current role of 24 months or more	+4
No title progression over four or more years with two or more roles	-3

Fair-Hiring Auditor. The auditor first scans the text written by the other agents for biased language and records a flag with a severity for every match. It then computes a merit-only score from job-related evidence, which has three parts:

$$a_{\text{merit}} = 50(0.8c_{\text{ev}} + 0.2c_{\text{pref}}) + 25min(1, \frac{y}{y_{min}}) + 25(0.5min(1, \frac{n_{\text{impact}}}{3}) + 0.5min(1, \frac{n_{\text{proj}}}{3})) \quad (6.6)$$

where:

c_{pref} coverage of the preferred skills;

y, y_{min} years of experience of the candidate and minimum required;

n_{impact} number of quantified results;

n_{proj} number of projects with a description of at least 60 characters.

If the mean score of the Tech Lead and the Culture agent differs from the merit score by more than 18 points, a *merit divergence* flag is raised, because a gap of this size suggests that non-merit factors may be influencing the subjective scores. The integrity of the evaluation is then rated:

$$I = max(0, 100 - \sum_f \pi(f)), \pi(\text{high}) = 25, \pi(\text{medium}) = 12, \pi(\text{low}) = 5 \quad (6.7)$$

An agent that was flagged with high or medium severity is called *neutralised*, and its weight in the consensus is halved. Protected attributes that are visible in the resume (for example date of birth or marital status) are reported, with advice to redact them, but are never used in scoring.

Seniority and Compensation agent. The agent estimates a level from L3 to L6 from the full-time experience: below 2 years is L3, below 5 years is L4, below 9 years is L5, and 9 years or more is L6. The level is raised by one band when there are at least four scope signals (for example *led*, *architected*, *mentored*) and a lead-level title, lowered by one band when the tenure is long but the ownership signals are weak, and capped at L5 unless both org-level scope and a lead or staff title are present. The target level of the job is derived from its title (principal or staff is L6, senior or lead is L5, junior or associate is L3) or from the required years. The level-fit score depends on the difference d between the estimated level and the target level:

$$a_{\text{level}} = f(d), f(0) = 100, f(+1) = 82, f(+2) = 60, f(+3) = 40, f(-1) = 70, f(-2) = 42, f(-3) = 20 \quad (6.8)$$

An indicative compensation band is attached to every level (for example L4 has 120,000 to 175,000 US dollars and 14 to 28 lakh rupees per annum). The suggested position in the band is $p = 0.25 + 0.45\,c + 0.30\,\min(1, n_{scope}/6)$, limited to the range 0 to 1,

where c is the skill coverage. The bands are indicative planning values and must be calibrated before any offer is made.

Cross-examination and consensus. After the scorecards, the agents cross-examine each other in a deterministic transcript that is generated from the data and is shown in the interface as a replayable debate. The executive consensus is the weighted sum of the four scores:

$$C = 0.35a_{\text{tech}} + 0.25a_{\text{merit}} + 0.20a_{\text{culture}} + 0.20a_{\text{level}} \quad (6.9)$$

with the weights of neutralised agents halved and renormalised. The consensus is mapped to a verdict: STRONG_HIRE for C of 80 or more, HIRE for 68 or more, LEAN_HIRE for 55 or more, and REJECT otherwise. Knock-out gates are then applied, because a high average must not hide a fatal flaw. They are shown in Figure 6.1 and in Listing 6.4.

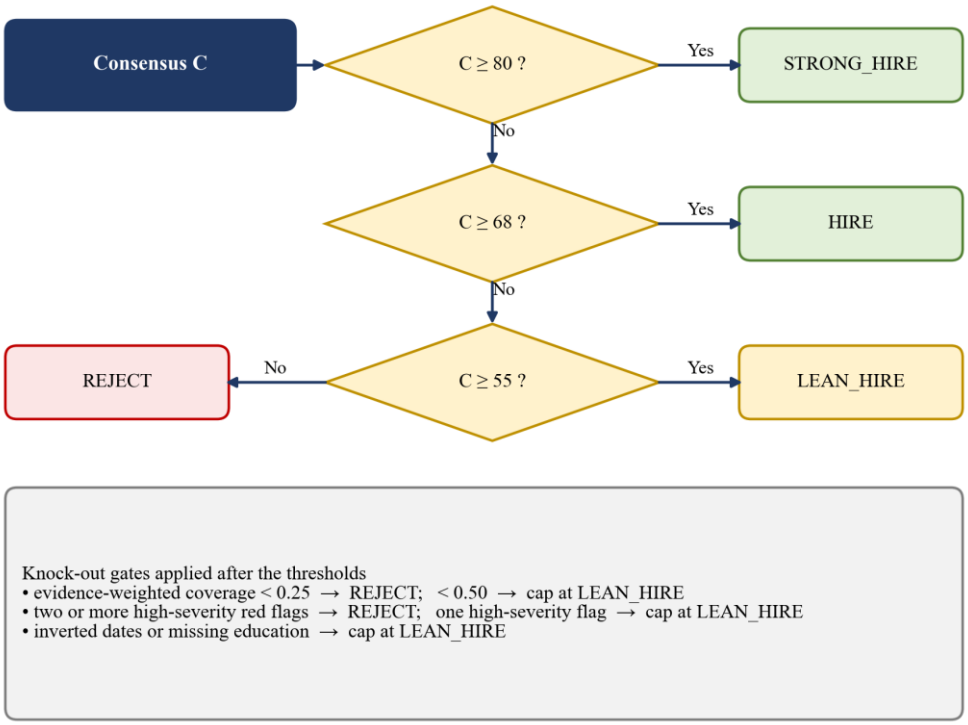


Figure 6.1: Verdict thresholds and knock-out gates of the council.

Listing 6.4: Knock-out gates applied to the verdict (abridged from agentic_council.py)

```
# Knock-out gates on required skills (least subjective part of the decision)
if f.required:
    if f.evid_coverage < 0.25:
        verdict = Verdict.REJECT
    elif f.evid_coverage < 0.5 and verdict in (Verdict.STRONG_HIRE, Verdict.HIRE):
        verdict = Verdict.LEAN_HIRE
red_flags = self._red_flags(f, tech, culture, audit, seniority)
high = [x for x in red_flags if x["severity"] == "high"]
if len(high) >= 2 and verdict != Verdict.REJECT:
    verdict = Verdict.REJECT
```

```
elif len(high) == 1 and verdict in (Verdict.STRONG_HIRE, Verdict.HIRE):
    verdict = Verdict.LEAN_HIRE
if f.inverted_dates and verdict in (Verdict.STRONG_HIRE, Verdict.HIRE):
    verdict = Verdict.LEAN_HIRE
if not f.has_education and verdict in (Verdict.STRONG_HIRE, Verdict.HIRE):
    verdict = Verdict.LEAN_HIRE
```

The overlap used by the concurrency rule is measured in months on the month index:

$$o_{ij} = max(0, min(e_i, e_j) - max(s_i, s_j)), o_{ij} \geq$$
$$6 \text{ months and different countries } \Rightarrow \text{ high} - \text{severity flag} \quad (6.10)$$

Finally, the council reports its own uncertainty. The confidence starts from 100 and is reduced by 1.2 times the standard deviation of the four agent scores, and by fixed penalties for missing information:

$$\text{conf} = min(99, max(20, 100 - 1.2\sigma - \sum \text{penalties})) \quad (6.11)$$

The penalties are 8 for a missing timeline, 15 for no parsed skills, 15 for inverted dates, 10 for missing education, 20 for a concurrent-employment conflict, 10 for fewer than 150 words and 5 for a job without parsed required skills. An agent whose score differs from the consensus by more than 20 points is reported as dissenting. A human review is requested when the auditor raises a high-severity flag, when the integrity score is below 70, when the confidence is below 55, or when the consensus lies within 2 points of a verdict boundary. The council is therefore a decision-support device and not an autonomous decision-maker.

Module 9: GitHub Authenticity Inspector

The inspector validates the programming languages that a resume claims against the public GitHub footprint of the candidate. It finds the handle in the parsed data or in patterns such as *github.com/user* in the text, and queries the public REST interface (user, repositories, languages per repository and public events) with the standard library only. A token is used when it is configured, and rate limits are handled: a short wait is honoured, and a longer limit returns a structured *rate limited* report instead of raising an error. Results are cached for 15 minutes. Forked repositories are excluded from the language evidence because forked code is not authored work. Each claimed language is classified as verified, weak or unverified, and the authenticity score combines the language, activity and repository-substance scores:

$$A = 0.55s_{\text{lang}} + 0.25s_{\text{act}} + 0.20s_{\text{sub}}, s_{\text{sub}} = 0.25s_{\text{stars}} + 0.10s_{\text{forks}} + 0.30s_{\text{mat}} +$$
$$0.15s_{\text{orig}} + 0.20s_{\text{age}} \quad (6.12)$$

If no language is claimed, the score is (0.25 s_{act} + 0.20 s_{sub})/0.45. When there is nothing public to judge, the verdict is *insufficient public data* and no score is given, because the absence of a public profile is not evidence of dishonesty. The report is used by the Tech Lead agent as additional evidence of repository maturity. The idea of using repository evidence is related to earlier work that matched developer profiles with job advertisements, but here it audits resume claims.

Module 10: Hybrid Talent Search

The talent search answers natural-language questions about the resume pool, for example *Engineers with 2+ years in Python and PyTorch*. Every resume is divided into chunks tagged with their section (skills, experience, projects, summary, education or certifications). A query plan extracts the experience requirement and the competencies of the query and expands the concepts with synonyms. Each chunk is scored by the fusion of Equation 3.7 with alpha = 0.55 and with BM25 parameters k_1 = 1.5 and b = 0.75. The sparse index uses an inverted index, and the BM25 score is scaled by a fraction of its theoretical maximum so that it lies between 0 and 1. The fused chunk score is multiplied by a section factor (1.0 for skills, experience and projects, 0.85 for the summary and 0.7 for education). The candidate-level scores are:

$$\text{sem} = 0.6h_{\text{best}} + 0.4h_{\text{top3}}, \text{rel} = (0.40\text{sem} + 0.60\text{cov})(0.65 + 0.35min(1, y/y_q)) \quad (6.13)$$

where:

- h_{best}, h_{top3} the best fused chunk score of the candidate and the mean of its best three chunk scores;
- *cov* competency coverage, the weighted sum of the strength of the evidence found for each competency divided by the total weight (a competency proved in experience or projects has strength 1.0, in the skills list 0.7, and by related terms only 0.25 to 0.45);
- y, y_q years of experience of the candidate and the experience required by the query.

Every result carries matched, partial and missing competencies with the section that proves each one, and highlighted evidence snippets. The index is cached by a content fingerprint, so repeated queries on the same pool are fast.

Module 11: Interview-Question Generator

The interview generator prepares a questionnaire for the panel from the profile of the candidate. It draws technical questions from a knowledge bank of skill-specific questions for the matched skills; project-specific questions from the technologies used in the projects of the candidate; probing questions for the missing skills (for instance, how quickly the candidate could learn the skill); questions suited to the level of experience (fresher or experienced); and behavioural questions. Every question has a difficulty (easy, medium or hard) and a statement of what the interviewer should look for in the answer. The questions of the council about gaps and red flags are appended as interview focus points.

The web dashboard contains its own version of the generator, which builds a larger kit for the selected candidate. The kit starts with the verification question that is stored with the candidate and a follow-up on the same evidence. It adds an architecture question and, for the role family of the candidate (machine learning, DevOps and reliability, data platform, backend, full-stack, business intelligence or AI safety), two system-design questions, a hands-on debugging exercise and two questions on quality and security. Up to three deep-dive questions are chosen for the matched skills from a bank of about thirty-five skills, with a generic template for the other skills, and a gap question or a mastery question follows, depending on whether skills are missing. Further questions are generated from the audit findings of this candidate: the red flags (an experience deficit, unverified or stuffed skills, missing quantified impact, inconsistent dates, missing required skills), a medium or high GitHub risk and a low ATS score. The kit ends with incident forensics, three behavioural questions and one leadership question, which are chosen from a rotating bank by a hash of the candidate identifier so that the kit is stable for a candidate but differs between candidates, and two questions on motivation and logistics. On the 162 candidates of the three pools the kit contains between 19 and 24 questions, 20.7 on average, compared with the single stored question per candidate before. The page offers filter chips by category and a button that copies the whole kit.

Module 12: Recruiter Chatbot

The recruiter chatbot lets the recruiter ask questions about the screened pool in words. It supports prompts such as the number of candidates, which candidates have the required skills, who has red flags and why, a comparison of two named candidates, an outreach e-mail for a named candidate and the compensation band. The answers are

composed from retrieved and scored evidence (the leaderboard, the council output and the talent-search results) and not from free text generation, in line with the principle of retrieval-grounded generation. In the default configuration the text of the answers is template-based and needs no language model. The conversation can be exported as a Markdown report.

The web dashboard contains its own chatbot engine, which answers only from the loaded pool and the active role and keeps the context of the conversation. Intents are recognised with whole-word patterns that are tried in a fixed order of specificity, so that a question about salary is not mistaken for a request for the top candidates. The engine understands questions about a single candidate (profile, the reason for the verdict, whether to hire, skills, GitHub, salary and interview questions), comparisons, filters by verdict, years of experience, skill, flag type and category, rankings and shortlists, the role and the compensation band, the meaning of the scores, and three kinds of e-mail draft (outreach, interview invitation and decline). Every answer carries a small context record with the intent, the candidates discussed and the list that was shown, and the earlier messages are passed back with the next question. This lets the engine resolve pronouns, the first, second or third item of the last list, and the instruction *show more*. Lists are shown five at a time, and asking for the same list again moves on to the next page. A name that belongs to two candidates, as in the 100-resume pool, is answered with a question that shows the rank numbers, and the wording varies deterministically so that the same question does not always receive the same sentence. Clickable follow-up questions are offered below the last answer. Candidates with missing fields, such as uploaded resumes, are shown with *n/a* instead of an error.

Module 13: REST API and Persistence

The FastAPI application of version 4.0.0 creates the database tables at start-up, adds a processing-time header to every response and configures cross-origin access from the list of allowed origins. The routers for jobs, resumes, screenings and audits delegate to service classes. The resume service computes the SHA-256 hash of an uploaded file and returns the stored resume if the hash is already known, so a duplicate is never stored twice. The screening service loads the job and the selected resumes, applies the blind-screening filter when requested, scores every candidate with the hybrid ranker and the quality analyser, produces the explanation, ranks the candidates, stores the session and its results, and writes an audit-log entry. A later request records the decision of the recruiter

on a result together with a time stamp. The relational database is SQLite when no other URL is configured (a PostgreSQL URL is also supported), and the schema is created by an Alembic migration; on the deployed host the relational routes use a temporary SQLite file.

The same application contains the account service of the deployed system. Its routers for authentication and candidates use a MongoDB database through PyMongo. The service hashes passwords with PBKDF2-HMAC-SHA256 (210,000 iterations and a random 16-byte salt) and compares the digests in constant time. For an unknown e-mail address it still performs a hash computation, so that the response time does not reveal which addresses are registered, and it returns the same message for an unknown address and for a wrong password. A counter of failed sign-ins locks the account for 15 minutes after five failures. A successful sign-in creates a random 32-byte token; only the SHA-256 digest of the token is stored in the sessions collection, together with an expiry time that a time-to-live index uses to delete sessions after seven days. Sign-in with Google sends the ID token to the endpoint /auth/google, which asks Google to validate it and then checks the audience, the issuer, the expiry, the verified-e-mail claim and the gmail.com domain before the user is created or linked. The candidate endpoint returns a pool from the candidates collection, and the start-up routine fills empty pools from the bundled seed files.

Module 14: Web Dashboard

The web dashboard is a single-page React 19 application built with Vite and styled with Tailwind CSS. After a sign-in, the recruiter sees the three-step workspace and, after the screening, the seven pages of the navigation. The sign-in card supports e-mail and password accounts, which are stored in the stand-alone configuration as per-user salted PBKDF2-HMAC-SHA256 digests with 200,000 iterations in the browser and are created and checked by the API in the connected configuration, and an optional Google button that is restricted to Gmail accounts and verified by the API. The credential fields block copy, cut and paste but accept the autofill and the strong-password suggestion of a password manager, they are not pre-filled after a logout, and the application asks Google to forget the previously selected account at sign-out. The application extracts text from PDF and DOCX files in the browser and ships a browser-side scorer and a bundled set of sample candidates, so that it runs fully offline; with environment variables it can instead call the API. The Streamlit application provides the same pages for local use. The

dashboard is deployed on Vercel, the API on Render and the data in MongoDB Atlas, and the deployed site uses the connected configuration. The logic of the interview generator and of the chatbot is covered by 29 tests, which run with the Node.js test runner on all 162 candidates.

6.3 Difficulties Encountered and Strategies Used to Tackle Them

During the development of the system, several technical considerations affected the design. They are described below with the strategy that was adopted.

Dates that defeat text similarity

A stress test with adversarial resumes showed that an inverted date range, such as *Jul 2026 to Jan 2024*, and two overlapping full-time jobs in different countries were not detected by the first version of the pipeline, because similarity scoring treats a date as an ordinary token. The inverted-date resume was scored on a shorter history and still received a hire verdict, and the concurrent-employment resume received a strong-hire verdict without any flag.

Strategy: the parser was extended with month-aware timeline parsing, and the council received explicit gates for inverted dates, concurrent full-time employment and missing education. The gates cap the verdict instead of changing the weighted average, so a high mean cannot hide a fatal flaw. The probe is a regression case study and not a held-out detection rate.

Keyword stuffing

A resume that lists many fashionable tools without any evidence obtained the same keyword score as a genuine candidate with four years of experience (47.1 percent in both cases).

Strategy: evidence-weighted coverage (Equation 3.10) was introduced, together with an unevidenced-skill ratio and the red flags *UNVERIFIED_SKILL_CLAIMS* and *SKILL_STUFFING*. The ablation experiment shows that this mechanism changes two verdicts.

Optional dependencies and offline operation

Dense embeddings, language-model narratives and the GitHub service all need libraries, model files or a network connection that may not be present on a demonstration machine or on a free hosting tier.

Strategy: every optional component has a deterministic local fallback (TF-IDF for embeddings, rule-based text for the narrative, a structured report for GitHub failures), and the interface reports which backend is active. The lean requirements file of the API omits the dense model, so the service fits in 512 MB of memory.

Rate limits and unreliable external services

The public GitHub interface limits the number of requests, and events are only available for the last 90 days.

Strategy: the inspector honours short waits, returns structured rate-limited or partial reports, caches responses for 15 minutes, excludes forks, and reports insufficient data instead of a guessed score. Its behaviour was verified with a simulated transport in the automated tests.

Bias that is hidden inside optional components

The deterministic components do not use the name of the candidate, so the name-swap test shows no change. This does not prove that dense embeddings or a language model would be invariant, because audits show bias exactly in those components.

Strategy: blind screening removes identifiers at the data layer before any optional component sees the text, and an auditor agent scans the text of other agents for biased language. Testing with dense embeddings and a language model remains future work.

Calibration of verdicts

On a 100-resume synthetic benchmark the council never issued a strong-hire verdict and rejected only 11 of 35 reject profiles, which shows that the thresholds are conservative at the top and permissive at the bottom.

Strategy: the limitation is reported openly, and the decisions are routed to human review when the consensus is close to a threshold. Recalibration against recruiter decisions is listed as a necessary step before operational use.

Deployment constraints

A free API service sleeps after a period of inactivity (the first request then takes up to about 50 seconds), the free MongoDB Atlas tier offers 512 MB and has to accept connections from any address because the host has no fixed address, and the Streamlit hosting resets its local database when it restarts. The relational routes of the API use a

temporary SQLite file on the deployed host, so only the accounts, the sessions and the candidate pools are persistent.

Strategy: the documentation states these constraints, the API reads its configuration from environment variables, and the web interface can run without the API in its stand-alone mode. The database password is kept only in the environment of the host, and the responses of the account endpoints do not reveal whether an address is registered.

Account security

A public sign-in form can be attacked by guessing passwords, by finding out which e-mail addresses are registered, and by stealing a copy of the database. A first version of the web interface kept the accounts in the browser only, which cannot protect against any of these.

Strategy: the accounts were moved to the server. Passwords are hashed with a salt and 210,000 PBKDF2 iterations, a wrong password and an unknown address give the same answer, five failures lock the account for 15 minutes, and sessions are random tokens of which only the digest is stored and which expire after seven days. Sign-in with Google is verified on the server instead of being trusted from the browser. The behaviour is checked by 18 automated tests.

Duplicate candidate names

In the 100-resume benchmark 50 names are used twice, for two different candidates with different roles. A chatbot that looks a candidate up by name would silently answer for the wrong person, or compare a candidate with himself or herself.

Strategy: the chatbot groups the candidates that share a name, asks which one is meant, shows the rank numbers and the roles, and accepts a rank number or an expression such as *the first one* as the answer.

Pseudocode of the Screening Pipeline

Listing 6.5: Pseudocode of the overall screening pipeline

```
FUNCTION screen(job_text_or_role, resumes, blind_mode):
    jd     <- JDParser.parse(job_text_or_role)         # required, preferred, min years
    cands  <- [ResumeParser.parse(file) for file in resumes]
    IF blind_mode: cands <- [BiasFilter.anonymize(c, i) for i, c in enumerate(cands)]
    FOR EACH c IN cands:
        quality <- ResumeQualityAnalyzer.analyze(c, jd.required_skills)
    ranked <- HybridRanker.rank_all(cands, jd, quality)      # 7 weighted scores
    FOR EACH item IN ranked:
        item.explanation <- ExplainableAI.generate(item, jd)  # reasons, strengths,
warnings
        item.council     <- Council.evaluate(item.candidate, jd)
```

```
        # four scorecards -> debate -> consensus C -> thresholds -> gates -> flags
    SAVE session, results, audit log
    RETURN ranked WITH verdicts, red flags, confidence, review triggers
```

The actual function calls differ slightly between the Streamlit application, the React interface and the API, but the sequence of operations is the one shown above. The behaviour of every module was confirmed by execution and testing.

CHAPTER 7

TESTING AND EVALUATION

Software testing is an important phase in the development of the AI Powered Resume Screening and Ranking System. It verifies that the implemented components behave as specified, finds defects, and validates that the modules work correctly together. For a screening system, testing has an additional purpose: it must show that the decisions follow the documented rules, that the system does not depend on protected attributes, and that it reports its limits instead of hiding them. The automated suite consists of 78 tests: 49 written with the pytest framework for the Python core, the API and the Streamlit interface, and 29 written with the Node.js test runner for the interview generator and the chatbot of the web interface. In the final run on the development machine all 49 Python tests passed in 16.25 seconds and all 29 JavaScript tests passed in 0.3 seconds.

7.1 Unit Testing

Unit testing is the process of testing individual components or functions of the software independently. The objective is to verify that each module performs its intended function. In this project, unit tests cover the resume parser, the skill ontology, the hybrid ranker and the explanation module, the council, the GitHub inspector, the talent search, the chatbot, the database, the authentication flow, the MongoDB-backed account and candidate API, and the interview generator and chatbot of the web interface. Table 7.1 lists ten representative test cases from the automated suite, chosen to cover the core screening behaviour; the complete suite is summarised by test area in Table 7.2.

Table 7.1: Representative Unit and Component Test Cases

Sl. No.	Test description	Expected result	Actual result
1	Resume parser: parse a sample resume given as text (name, e-mail, phone, location, links, skills, education, experience, projects)	Fields are extracted; B.E. is mapped to B.E. / B.Tech; experience of at least 1.5 years; at least one project	Passed
2	Ontology: match required skills of a job with a resume that has Flask instead of FastAPI	Matched, missing (AWS) and partial (FastAPI via Flask) skills are reported; match above 60 percent	Passed

Sl. No.	Test description	Expected result	Actual result
3	Hybrid ranker and explanation: score a strong candidate and explain the score	Overall score at least 75; skill score at least 80; experience score at least 85; a recommended tier with strengths and reasons	Passed
4	Council: protected attributes in a resume	Date of birth or age is reported by the auditor but is not used in scoring	Passed
5	Council: candidate with missing required skills	The verdict is REJECT	Passed
6	Demo suite: gold-standard resume	Rank 1, match of at least 85 percent, STRONG_HIRE, no flags	Passed
7	Demo suite: buzzword resume	Match below 45 percent, REJECT, UNVERIFIED_SKILL_CLAIMS and an experience flag	Passed
8	Demo suite: lateral partial match	LEAN_HIRE; FastAPI is a partial match; PyTorch is reported as missing	Passed
9	Demo suite: corrupt resume	ATS score below 35 and DATE_INCONSISTENCY flag	Passed
10	Interview kit (web interface): size	Each of the 162 candidates of the three pools gets 15 to 30 questions; the mean is between 19 and 24	Passed

Table 7.2 summarises the complete automated test run, grouped by test area.

Table 7.2: Automated Test Results by Test Area

Test area	Test files	Tests	Result
Resume parsing and skill ontology	test_parser.py, test_ontology.py	4	Passed
Hybrid ranker, explanation and four-agent council	test_hybrid_ranker.py, test_agentic_council.py	7	Passed
GitHub inspector and hybrid talent search	test_github_inspector.py, test_talent_rag.py	8	Passed
Recruiter chatbot and recruitment database	test_chatbot.py, test_database.py	2	Passed
Authentication flow and application start-up	test_auth_flow.py, test_ui_smoke.py	6	Passed
Demonstration resumes end to end	test_demo_suite.py	4	Passed
Accounts, sessions, Google sign-in and candidate pools (MongoDB)	test_mongo_api.py	18	Passed
Web interface: interview kit and chatbot	interview.test.mjs, chatbot.test.mjs	29	Passed
Total	**49 Python and 29 JavaScript tests**	**78**	**78 passed**

The 49 Python tests ran in 16.25 seconds and the 29 JavaScript tests in 0.3 seconds. The account tests use an in-memory MongoDB, and the Google tests replace the call to Google with a simulated answer, so that no network access and no secret is needed.

Objectives of Unit Testing

The main objectives of unit testing are:

- verify the correctness of individual functions and classes;
- make sure that each component works independently of the user interface;
- find errors in parsing, matching and scoring logic early;
- verify the handling of missing, invalid or unusual input, such as an empty query or a failed external service;
- validate that fairness-related behaviour (flags for protected attributes, redaction of personal data) works as specified; and
- improve the reliability and maintainability of the system through repeatable regression tests.

Detailed Explanation of Test Case 1

The resume parser is the entry point of the pipeline. If it extracts a field incorrectly, every later score is affected, so Test Case 1 verifies the parser with a complete sample resume. The sample contains a header with name and contact details, a professional summary, a skills section, an education section with a CGPA, a work experience section with a date range that ends with *Present*, and a project section.

Objective: verify that the parser separates the sections of a resume and extracts the required fields correctly.

The test case involves the following steps:

1. Prepare the sample resume as a text document.
2. Pass the bytes of the document and its file name to the parser.
3. Check the name, e-mail address, telephone number and location.
4. Check that the skills include Python and Machine Learning after normalisation.
5. Check that the degree is mapped to B.E. / B.Tech.
6. Check that the years of experience computed from the date range are at least 1.5.

7. Check that at least one project is detected.

7.2 Integration Testing

Integration testing verifies the interaction between modules of the system. It ensures that the modules communicate correctly and that information flows through the pipeline as intended. The following aspects are considered:

- **Parser and ontology:** skills extracted from the text of a resume are normalised to canonical names and are matched against the required skills of a job.
- **Ontology, ranker and explanation:** the match percentage of the ontology feeds the technical score of the ranker, and the breakdown of the ranker feeds the explanation module.
- **Features and agents:** the common feature builder passes the same evidence-weighted coverage, timeline and term counts to the four agents, and the auditor reads the text written by the others.
- **Council and verdict gates:** missing required skills, red flags, inverted dates and missing education change the verdict as specified.
- **GitHub report and Tech Lead:** the report of the inspector is picked up by the Tech Lead agent as additional evidence of repository maturity, while failures of the inspector do not stop the council.
- **Search and chatbot:** the chunk index of the talent search is reused by the queries of the chatbot, and the chatbot composes its answers from the ranked results.
- **Authentication and interface:** the sign-in and sign-up flow controls which tabs are visible, and the interface boots with the ranked candidates in its session.
- **Accounts and database:** the account endpoints, the session store and the seeding of the candidate pools work together on a MongoDB database, and the Google endpoint combines the check of the token with the user store.
- **Interview generator, chatbot and pool data:** the logic of the web interface is exercised on the real files of all three candidate pools.

The account and candidate endpoints of the API are covered by automated tests. The relational endpoints for jobs, resumes, screenings and audit logs are exercised manually through the automatically generated documentation page; automated tests for them have not yet been written, and they are listed among the future enhancements.

7.3 System Testing

System testing evaluates the integrated system to verify whether the implemented functionality works together as expected. It focuses on the complete screening of a pool of resumes, the correctness of the verdicts, the explanation of the decisions and the behaviour of the interface. Four purpose-written resumes (the demonstration suite) are used to probe specific behaviours of the complete pipeline. Each is run against the Machine Learning Engineer role with 17 required skills and a two-year minimum. The following aspects are considered during system testing:

- **Ranking:** the gold-standard resume must rank first and receive STRONG_HIRE without flags.
- **Keyword stuffing:** the buzzword resume must be rejected, with the flags for the experience deficit and the unverified claims.
- **Related skills:** the lateral resume must receive partial credit for FastAPI through Flask, report PyTorch as missing and receive LEAN_HIRE.
- **Uncertainty:** the corrupt resume must have a low ATS score, an inconsistency flag and a request for human review instead of a confident verdict.
- **Interface:** the application must start with all tabs and a ranked pool.

Table 7.3 lists the results of the four demonstration resumes obtained from the complete pipeline.

Table 7.3: System Test Results of the Demonstration Resumes

Test case	Expected behaviour	Match (%)	ATS score	Verdict	Status
TC1: Gold standard	Ranked first, strong hire, no flags	88.1	100	STRONG_HIRE	Passed
TC2: Buzzword exaggeration	Rejected with experience and unverified-claim flags	41.6	56	REJECT	Passed
TC3: Lateral partial match	FastAPI credited through Flask; PyTorch reported missing	61.3	92	LEAN_HIRE	Passed

Test case	Expected behaviour	Match (%)	ATS score	Verdict	Status
TC4: Corrupt ATS format	Low ATS score, date-inconsistency flag, human review	34.0	23	REJECT (human review)	Passed

System testing was performed in a controlled environment with the offline TF-IDF configuration. The results are produced by running the real pipeline and are not hard-coded; the scores can differ by a small amount when the dense model is installed.

The deployed system was also checked end to end, partly by hand and partly with a few requests. The health endpoint of the API answers 200 and reports the database as connected; the three candidate pools return 12, 50 and 100 candidates; a request from the origin of the web site receives the cross-origin permission header; a login with an unknown address receives the generic 401 message; the built web site contains the address of the API and the Google client identifier; and a sign-up and a sign-in were carried out on the live site by the team, after which the new accounts were visible in the users collection of the database. No load test was made.

The sign-in and sign-up forms were checked in a browser against the built application. Scripted paste, copy and cut events were blocked on the four credential fields, and a real keyboard copy and paste left the password field empty. The sign-in fields unlock when they are clicked, a value that is filled programmatically, as a password manager does, is accepted, and the sign-up password field keeps the new-password hint that triggers the strong-password suggestion. The Interview Assistant page shows the filter chips and about twenty questions for a candidate, and the chatbot page shows follow-up suggestions, answers a follow-up question about the candidate discussed last, and scrolls to the newest message.

7.4 User Acceptance Testing

User acceptance testing evaluates whether the system satisfies the requirements from the point of view of its intended user, a recruiter. It focuses on the clarity of the interface, the usefulness of the explanations and the correctness of the results. For this system, the acceptance checks were carried out by the project team with the demonstration script of the project, which walks through the sign-in, the loading of a pool, the leaderboard, the dossier with the debate, the blind-screening switch, the comparison, the interview kit, the chatbot and the export of a report. The following activities were checked:

- **Workflow:** a new user can reach the leaderboard in three steps without instructions.
- **Leaderboard:** the ranking, the score bars, the evidence lines, the skill chips and the number of flags are understandable.
- **Dossier:** the four scorecards, the level and the compensation band, and the replayable debate explain why a verdict was given.
- **Blind mode:** names become neutral identifiers throughout the interface when blind screening is switched on.
- **Interview kit:** the questions are specific to the selected candidate, are grouped in categories and can be filtered by category.
- **Chatbot:** a follow-up question such as why? refers to the candidate discussed last, a long list can be continued with show more, and a conversation can be exported as a Markdown file.
- **Error handling:** a missing optional component or a failed external call produces a clear message and does not stop the screening.

A formal usability study with external recruiters has not been conducted, and whether the views help recruiters to decide better or faster remains future work. The system is considered acceptable for demonstration because the implemented functions operate as intended, the displayed information is understandable and the results of the performed tests satisfy the defined criteria.

CHAPTER 8

RESULTS

This chapter presents the execution and the results of the implemented system. The first part shows screenshots of the running web interface that were captured from the real application with the demonstration pool of twelve resumes loaded. The second part reports the experiments that were carried out to answer five research questions (RQ – Research Question). All resumes used in the experiments are synthetic.

8.1 Snapshots

This section presents screenshots that illustrate the functions of the web interface. The screenshots were captured from the running application and are not mock-ups.

Workspace and Candidate Intake

After signing in, the recruiter selects the role and loads the candidates. Figure 8.1 shows the second step of the workspace after the 12-resume core suite has been loaded: the pool size is shown at the top right, the recruiter can upload their own PDF, DOCX or TXT files (up to 100 in the browser) or load one of the benchmark pools, and the screening engine is started with a single action. In the deployed configuration the benchmark pools are fetched from the candidates collection of the database. The scores of these pools were computed beforehand by the Python pipeline, so the staged progress messages of the interface only illustrate the stages; only resumes uploaded by the recruiter are parsed and scored in the browser.

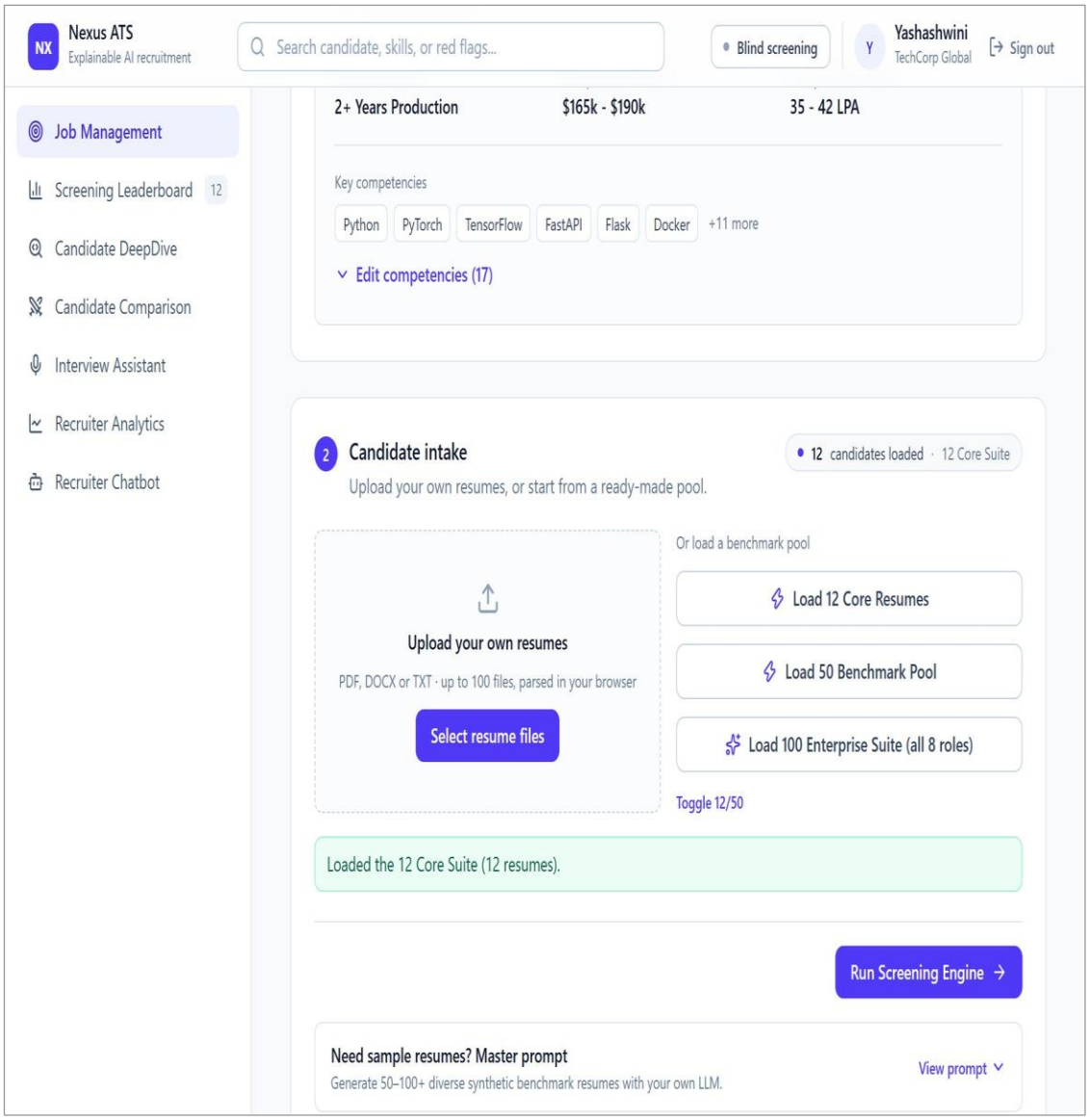


Figure 8.1: Candidate intake with the core suite of twelve resumes loaded.

Screening Leaderboard

The leaderboard is shown as soon as the screening has finished. Its header displays the telemetry of the run: the number of candidates screened (12) with the average match of 52.2 percent, the top match of 88.1 percent, the model backend (TF-IDF with BM25, local council, local backend) and the active role with its 17 required skills. A natural-language search with prompt shortcuts is placed above the ranked cards. Figure 8.2 shows the top of the leaderboard.

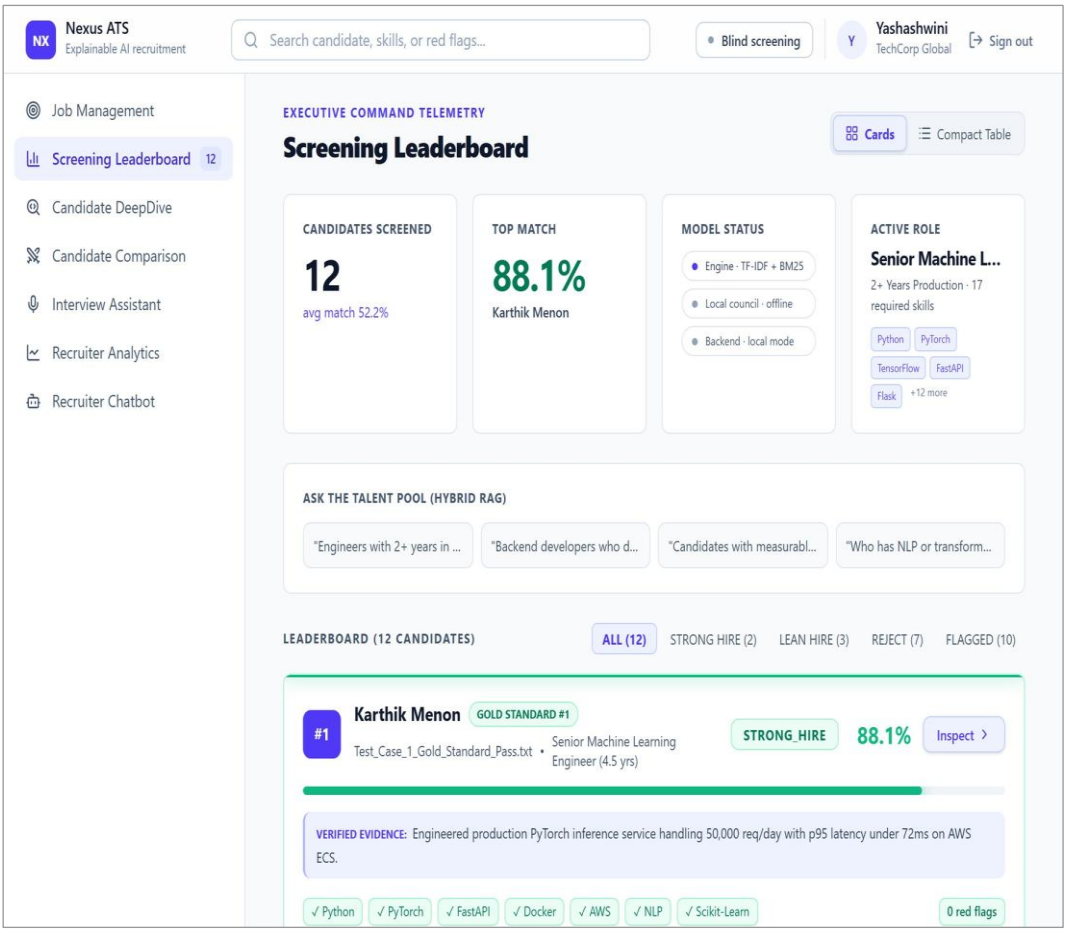


Figure 8.2: Screening leaderboard: run telemetry, talent-pool search and the first candidate cards.

Each card contains the rank, the file name and the role of the candidate, the verdict, the match percentage with a score bar, a line of verified evidence from the resume, the matched skills (with a check mark), the partially matched skills (for instance *FastAPI via Flask: 0.5 credit*), the missing skills (with a cross), and the number of red flags. Figure 8.3 shows the cards of the lateral and the weaker candidates, where the flag *REQUIRED_SKILLS_MISSING* is raised.

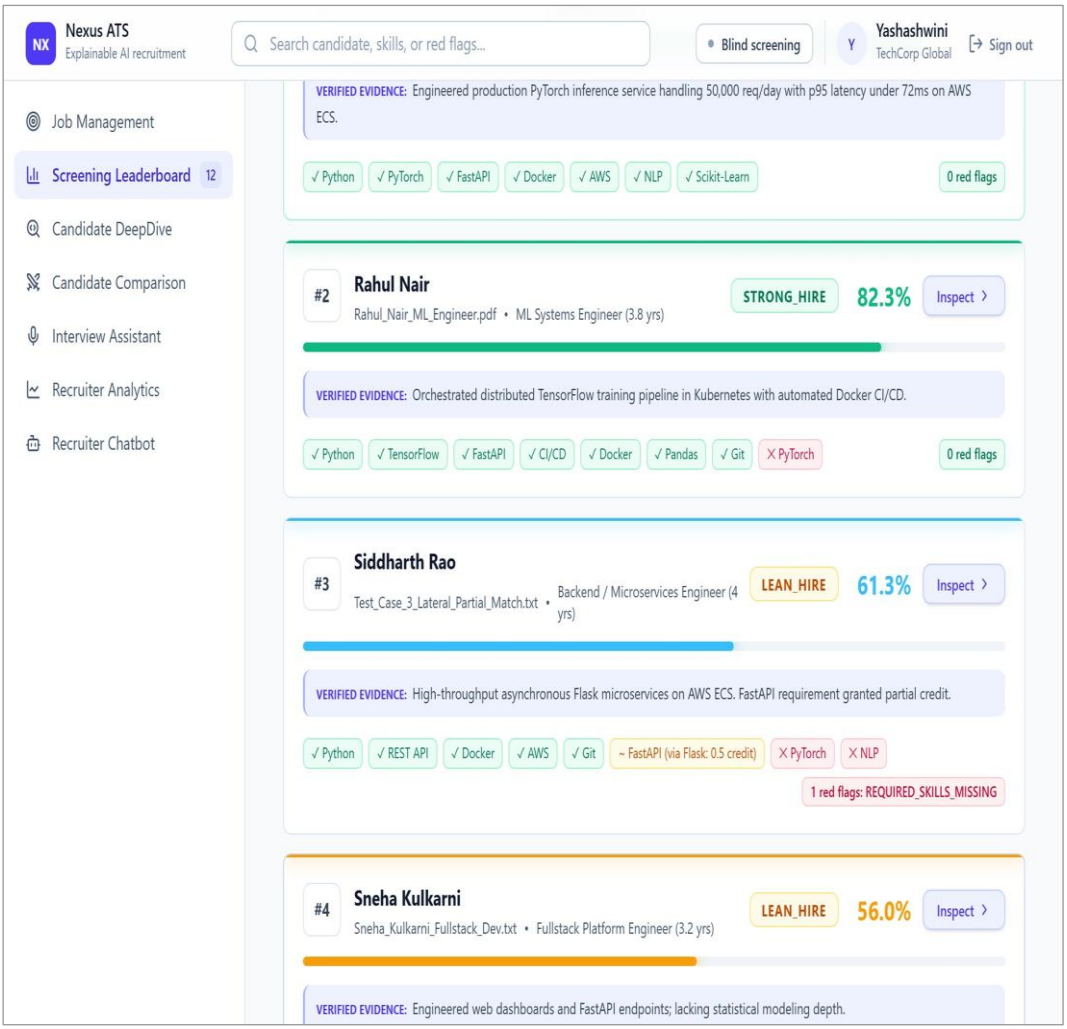


Figure 8.3: Leaderboard cards with evidence lines, skill chips and red-flag counts.

The cards can be switched to a compact table with the rank, the match, the ATS format score, the council consensus and the red flags, which is useful for a quick comparison of many candidates (Figure 8.4).

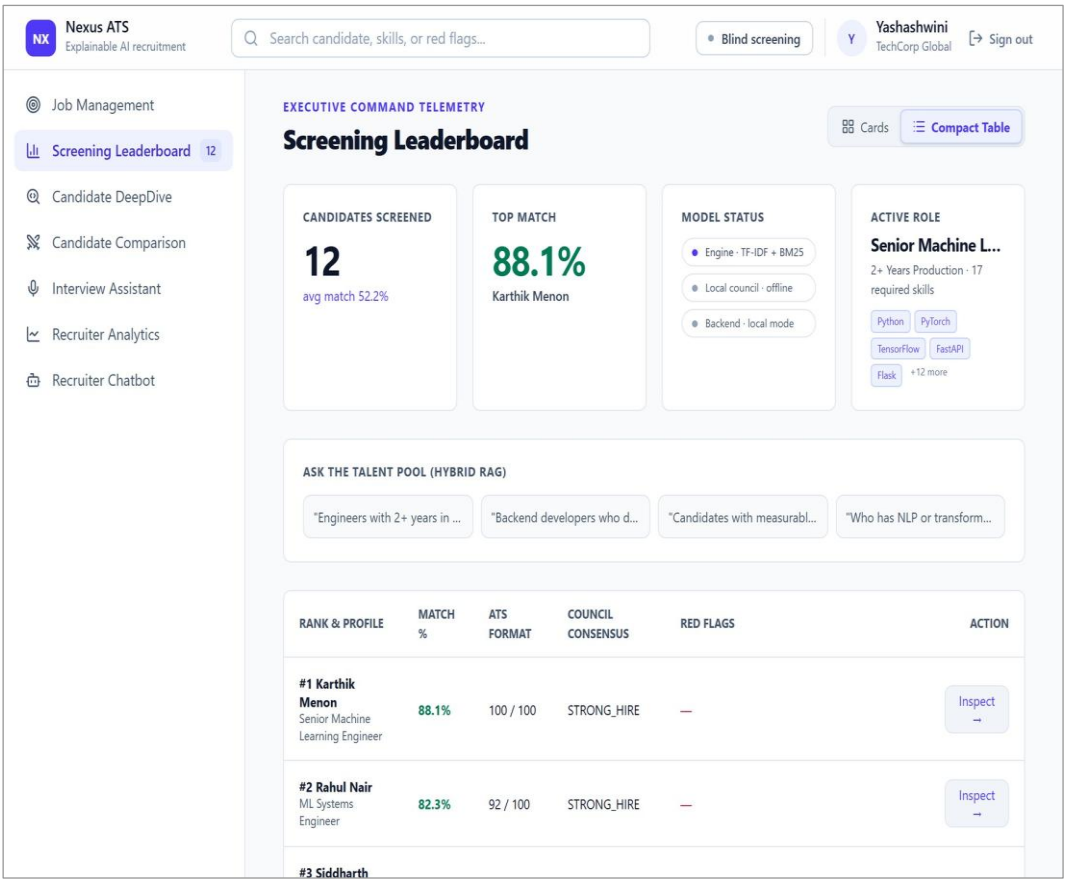


Figure 8.4: Compact table view of the leaderboard.

Candidate DeepDive and Council Dossier

The dossier of a candidate shows the hybrid score and the council consensus, the scorecards of the four agents with their weights (Tech Lead 35 percent, Culture and Retention 20 percent, Fair-Hiring Auditor 25 percent, and Seniority and Compensation 20 percent), the estimated level with the indicative compensation band, and the cross-examination debate, which can be replayed turn by turn. For the top candidate, the Tech Lead scores 89, Culture and Retention 78, the auditor 99 and the Seniority agent assigns level L5 (Figure 8.5).

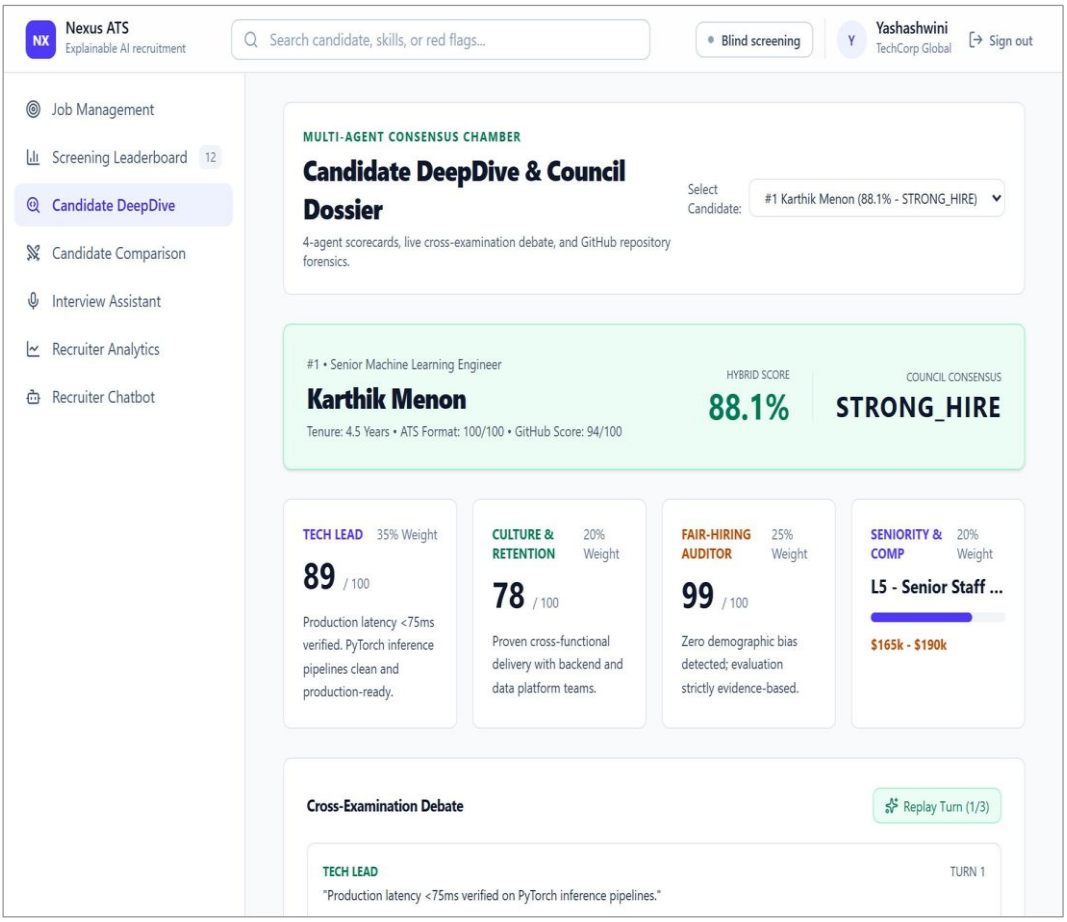


Figure 8.5: Candidate dossier: scorecards of the four council agents and the debate.

Candidate Comparison

The comparison page puts two selected candidates side by side. For each dimension it shows the two values and the advantage: overall match, ATS readability, council consensus, GitHub authenticity and the salary range. In Figure 8.6 the first candidate leads the second by 5.8 percentage points.

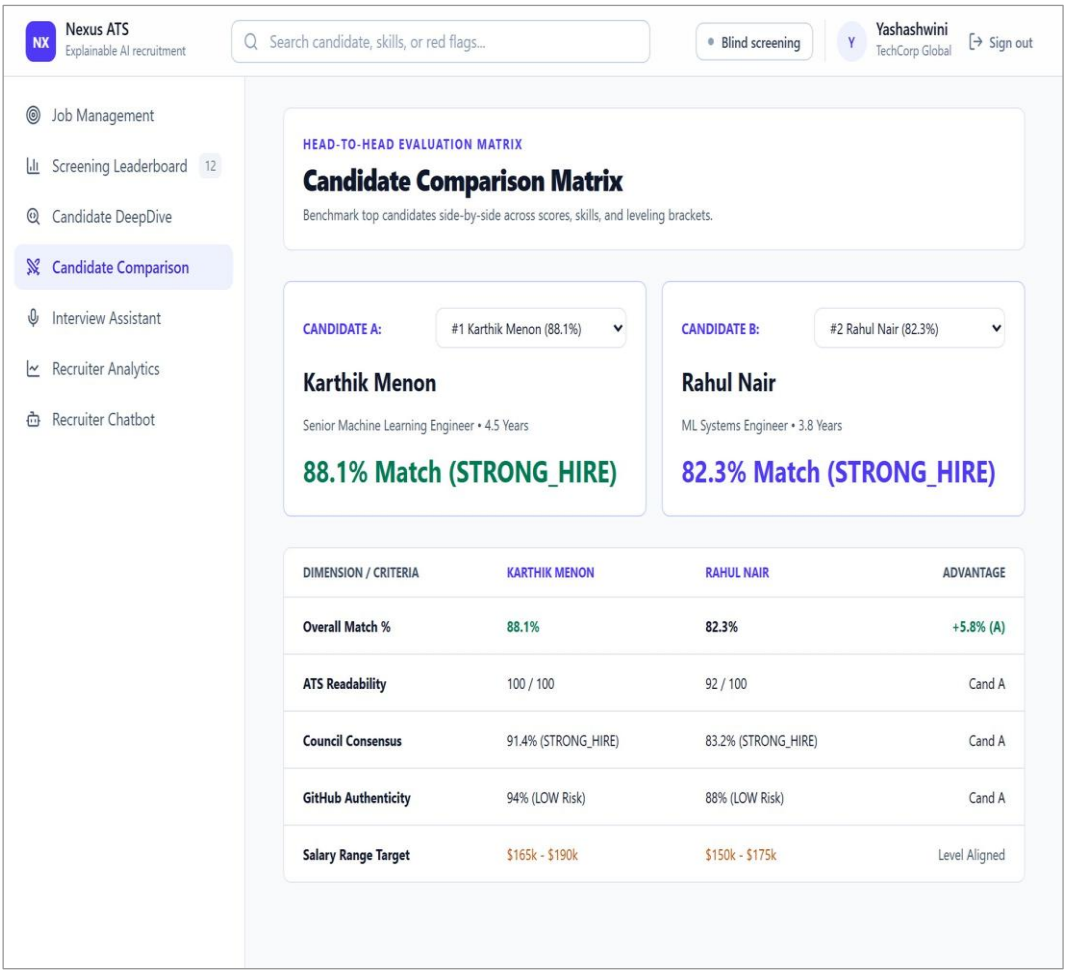


Figure 8.6: Head-to-head comparison matrix of two candidates.

Interview Assistant

The interview assistant prepares a tailored questionnaire for the selected candidate. It combines the question stored with the candidate with questions generated from the role family, the matched skills, the detected gaps, the red flags and the GitHub risk of the candidate. The kit has between 19 and 24 questions for each of the 162 candidates of the three pools (20.7 on average), where the stored data held a single question. Every question has a coloured category badge (for instance Technical Verification, System Design, Red-Flag Verification or Behavioral and Collaboration) and a rubric that states what the interviewer should look for in the answer. Chips above the list filter the kit by category, with the number of questions in each category, and a button copies the whole kit as text (Figure 8.7).

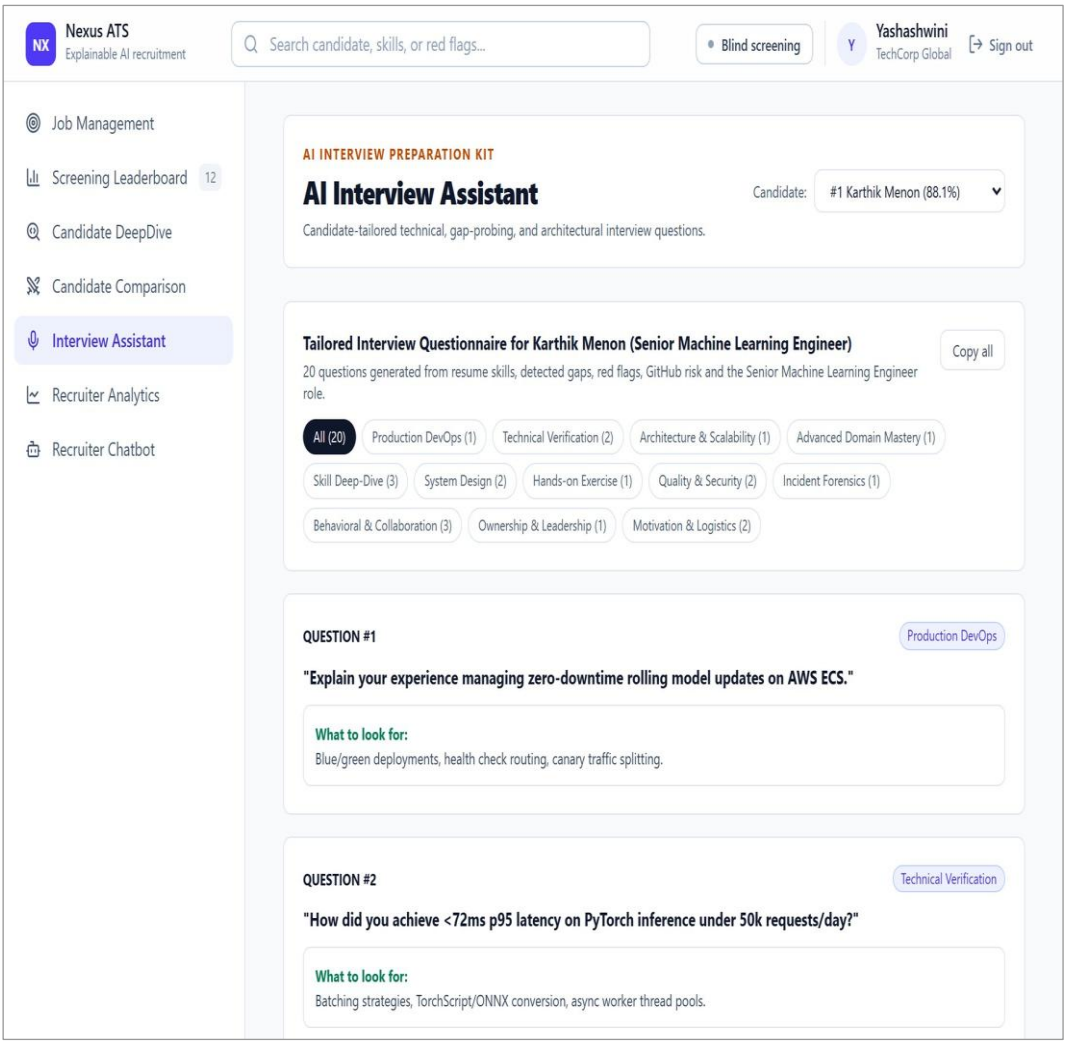


Figure 8.7: Interview assistant: about twenty candidate-specific questions with category filters and evaluation rubrics.

Recruiter Analytics

The analytics page summarises the pool: the pool conversion rate (2 of 12 resumes are strong hires), the median score of 48.8 percent, the red-flag intercept rate of 58.3 percent (7 of 12 resumes are flagged), the ATS parse integrity of 91.7 percent, and the frequency of the required skills across the applicant pool (Python in 92 percent of the candidates, Docker in 50 percent, FastAPI in 42 percent, AWS in 33 percent and verified PyTorch in 17 percent), as shown in Figure 8.8.

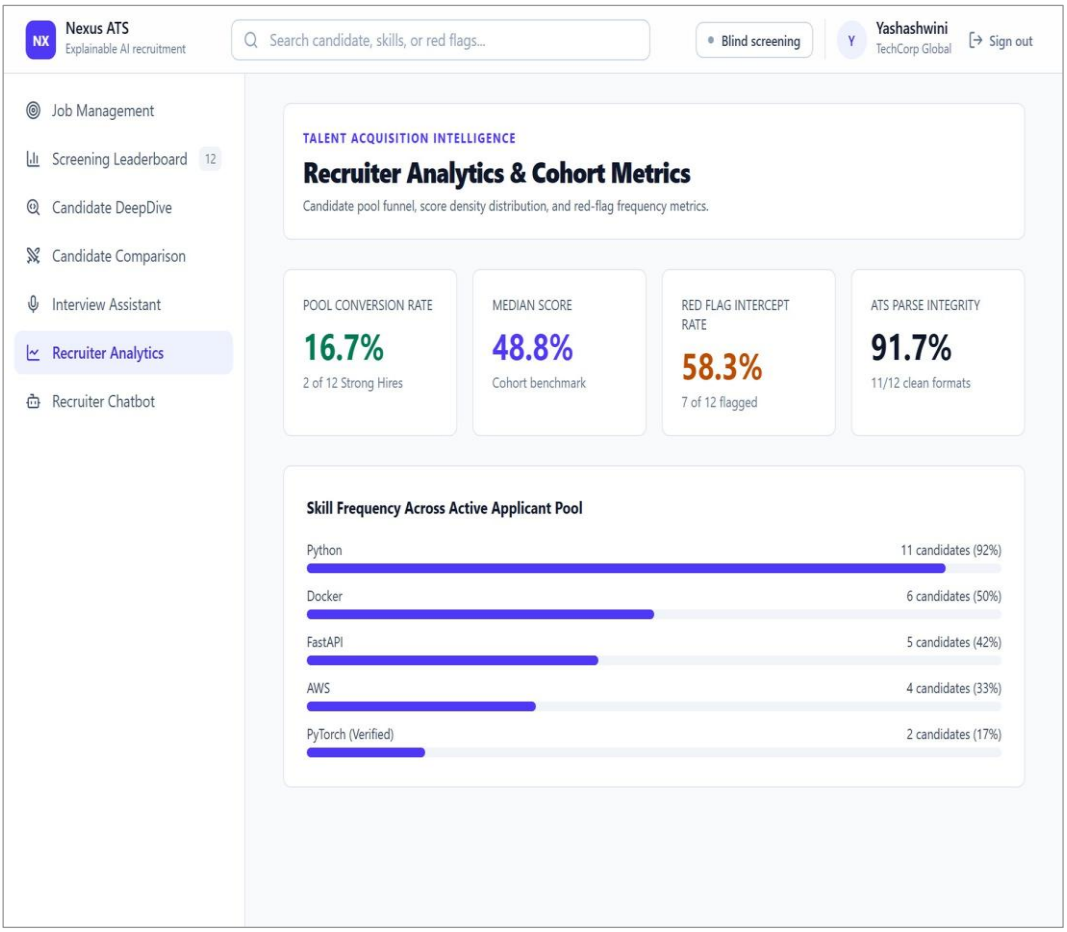


Figure 8.8: Recruiter analytics and cohort metrics.

Recruiter Chatbot

The recruiter chatbot answers questions in words from the screened pool. Figure 8.9 shows the answer to the question *Who has red flags and why?*: the audit summary lists each flagged candidate with the rank, the match, the verdict, the triggered flags, the finding of the Fair Auditor and the GitHub forensics.

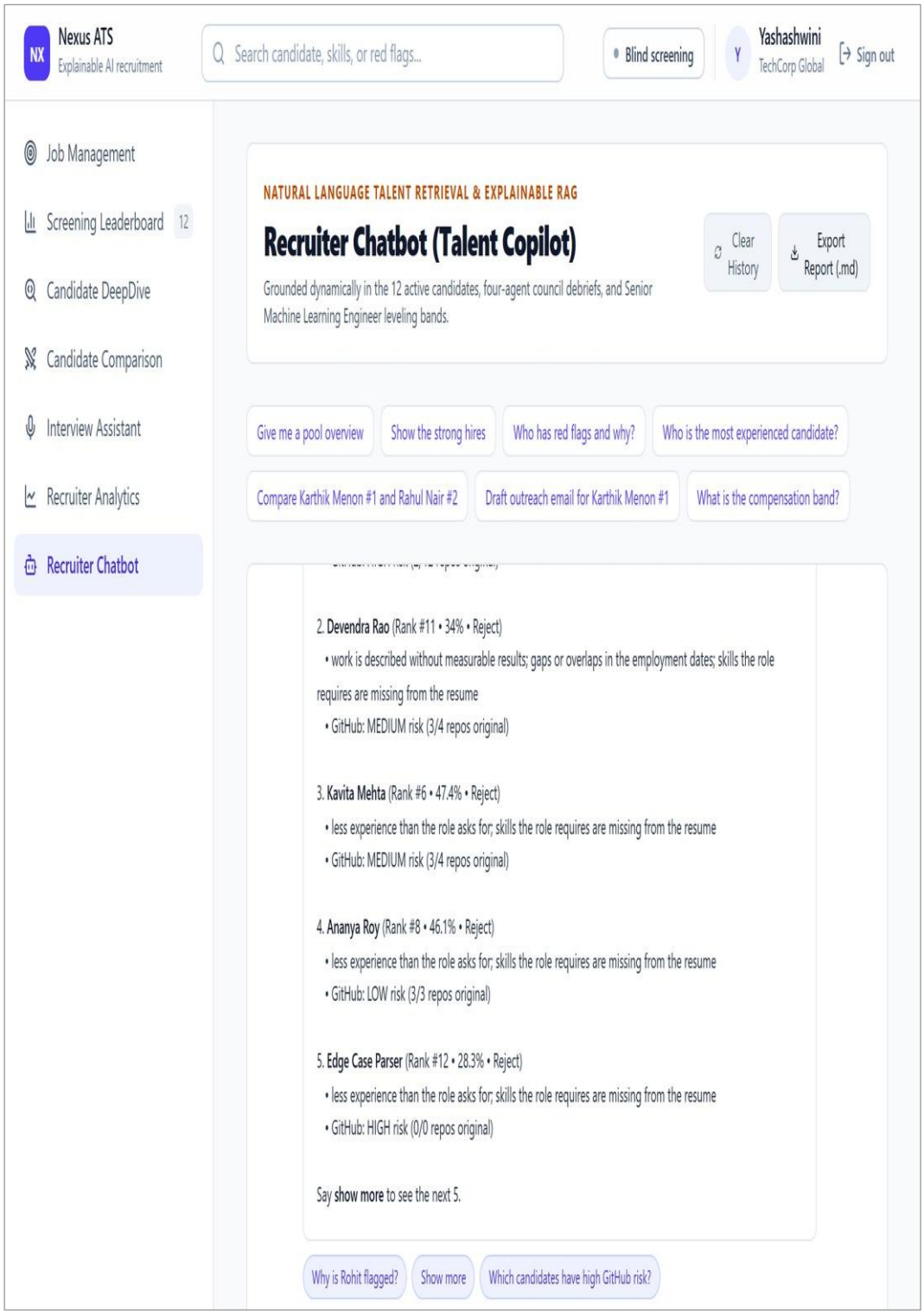


Figure 8.9: Recruiter chatbot answering a red-flag audit question.

The chatbot keeps the context of the conversation. After a question about a candidate, the follow-up *why?* explains the verdict with the council scorecards, *should I hire her?* gives a recommendation with the arguments for and against, and *interview questions for her* returns five questions from the kit. Long lists are shown five at a time and continue with *show more*, a name that belongs to two candidates is answered with a

question that shows their rank numbers, and clickable follow-up questions appear below each answer (Figure 8.10). The whole conversation can be exported as a Markdown file.

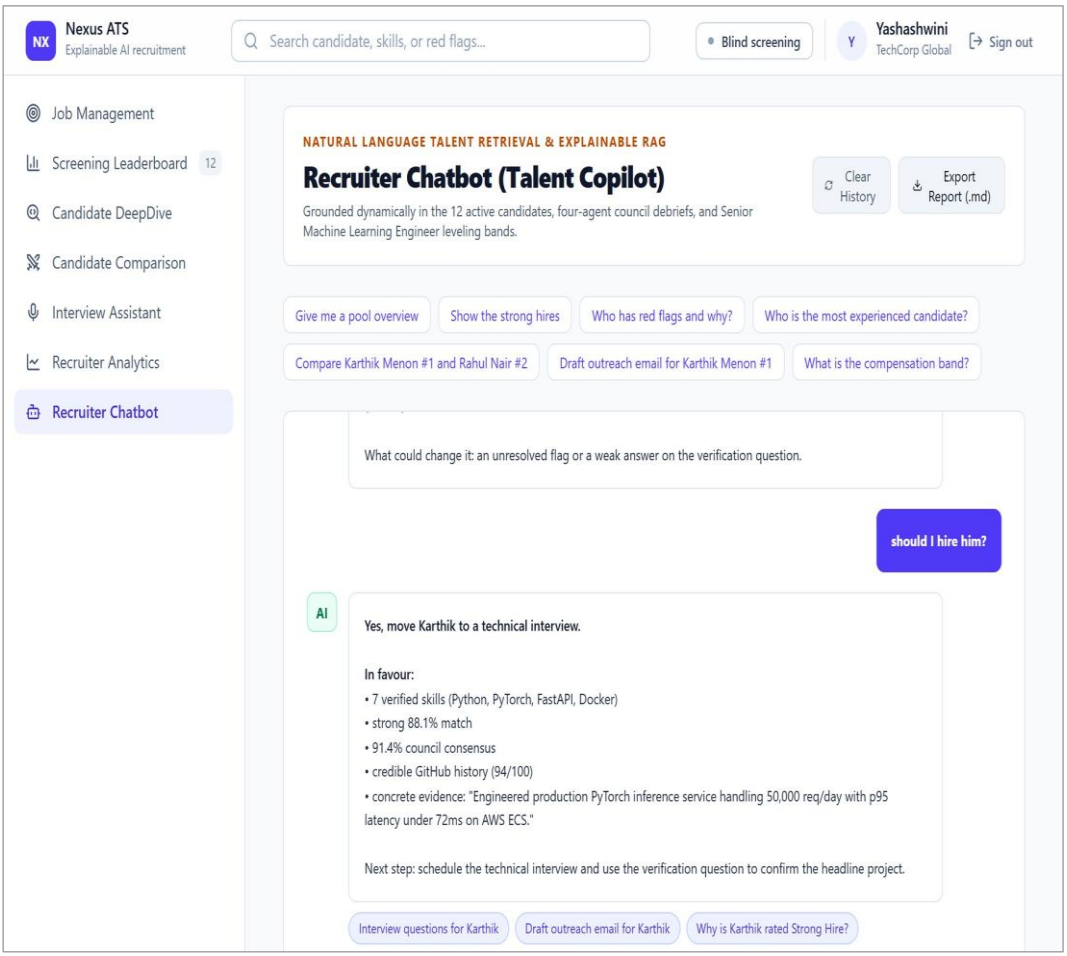


Figure 8.10: Recruiter chatbot answering follow-up questions about the candidate discussed last, with clickable suggestions.

Blind Screening

When the blind-screening switch in the top bar is turned on, the identities in the whole interface are replaced by neutral identifiers (*Candidate #001*, *Candidate #002*, and so on) and the cards show *identity hidden*. The scores, the verdicts and the ranking remain unchanged, which is the behaviour that the name-swap experiment verifies. Figure 8.11 shows the leaderboard with blind screening on.

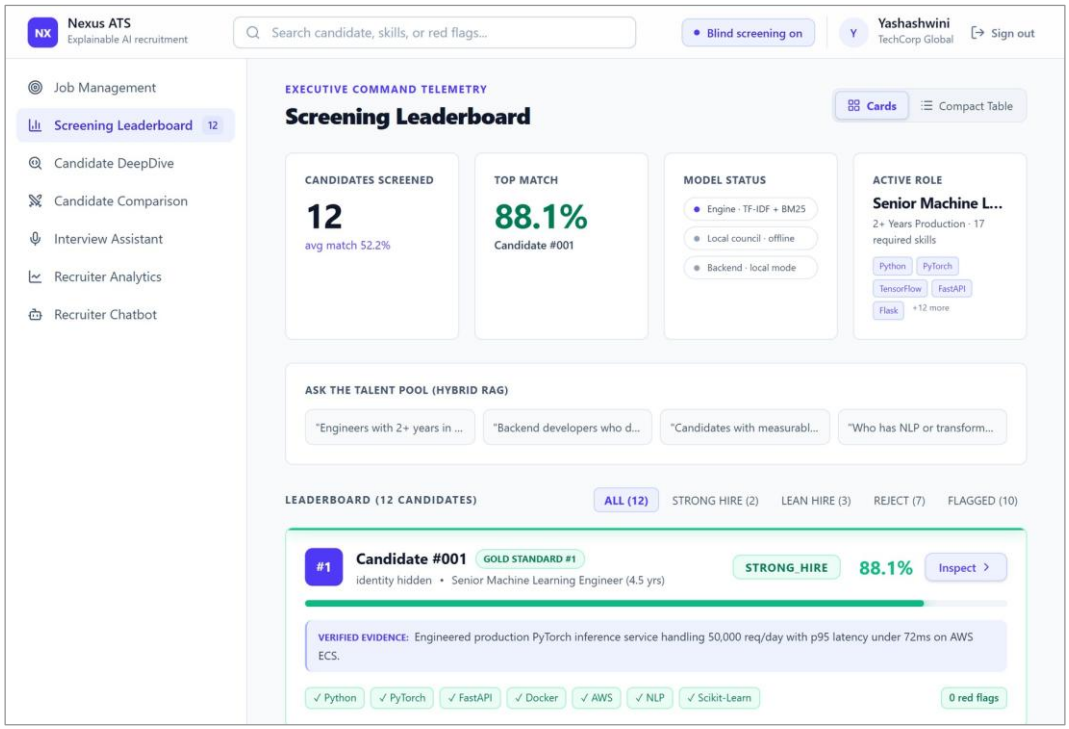


Figure 8.11: Leaderboard with blind screening switched on: identities are hidden.

8.2 Experimental Setup and Research Questions

The evaluation has three parts. The **pilot study** uses twelve resumes and one job (Machine Learning Engineer, 17 required skills, minimum two years of experience): eight synthetic resumes that ship with the project plus four purpose-written test cases that probe specific behaviours. The **benchmark study** uses 100 synthetic resumes over the eight role families, generated with language-model assistance under four archetypes in fixed proportions: 20 strong hires, 45 lean hires, 25 buzzword exaggerators and 10 chronology or formatting anomalies (the last two form the 35 reject profiles). The **adversarial probe** runs the first 20 benchmark resumes with additional targeted anomalies through the earlier and the upgraded pipelines. All values were measured in the offline configuration (TF-IDF semantics, no language model) with Python 3.11.9 on a laptop CPU, in a single process. The baseline is a keyword score modelled on the Phase 1 ATS: the percentage of required skills whose name appears anywhere in the resume text.

The experiments answer the following research questions. Throughout this chapter, RQ stands for Research Question, and RQ1 to RQ5 refer to the questions listed below.

- **RQ1 – Research Question 1:** Does evidence-weighted screening separate genuine candidates from keyword-stuffed candidates better than the Phase 1 keyword baseline?

- **RQ2 – Research Question 2:** Are the scores and verdicts invariant to the name of the candidate?
- **RQ3 – Research Question 3:** Are the rankings and verdicts robust to changes in the weights chosen by the developers?
- **RQ4 – Research Question 4:** What is the processing cost of screening one resume?
- **RQ5 – Research Question 5:** Are temporal and structural anomalies in resumes, such as inverted dates or concurrent full-time employment, intercepted by explicit gates?

8.3 Processing Latency (RQ4)

Table 8.1 and Figure 8.12 report the cost of each stage on the 12-resume pool. A full screening takes 29.58 ms per candidate, which is about 33.8 candidates per second on a single CPU thread. Parsing dominates the cost with 61 percent; the hybrid ranking is the cheapest stage. The timings depend on the hardware and exclude dense-embedding inference, which was not installed, and the rendering cost of the browser client. The cost grows linearly with the number of resumes.

Table 8.1: Per-Resume Processing Latency (12-Resume Pool, TF-IDF, Single Thread)

Stage	Latency (ms)	Share (%)
Resume parsing	18.01	60.9
Hybrid scoring and ranking	3.78	12.8
Multi-agent consensus	7.79	26.3
End-to-end	29.58	100

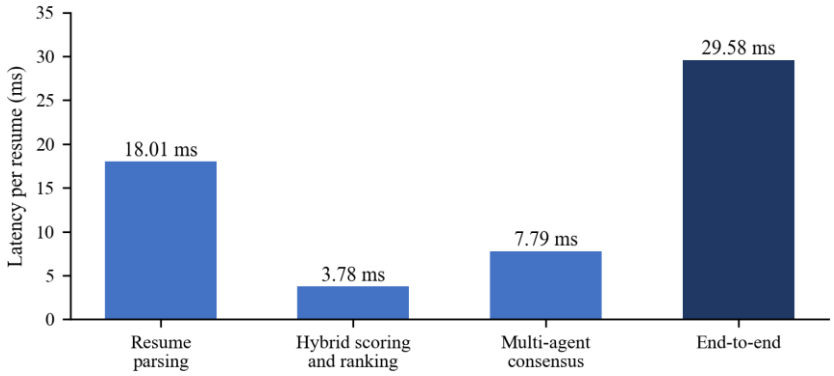


Figure 8.12: Per-stage processing time per resume.

8.4 Ranking Divergence from the Keyword Baseline (RQ1)

Table 8.2 lists the keyword-baseline and hybrid scores of the four test cases of the pilot pool, and Figure 8.13 compares the two scores for all twelve pilot resumes. The two rankings agree on the extremes, with a Kendall rank correlation of 0.667, but they differ where the design intends (Figure 8.13). The keyword baseline scores TC2, a resume with 0.3 years of experience and a long list of fashionable tools, at 47.1 percent, identical to TC3, a genuine four-year engineer, and ranks it fifth, whereas the proposed ranker places it tenth and separates TC2 from TC3 by 19.7 points (41.6 against 61.3). A value of 0.667 indicates substantial disagreement between the rankings; it does not by itself show which ranking is correct. The evidence for correctness here is the case analysis, and with a single pool of twelve synthetic resumes the result shows a mechanism and not a statistical effect.

Table 8.2: Keyword Baseline and Proposed Hybrid Scores of the Four Test Cases

Test case	Keyword baseline (%)	Proposed hybrid (%)	Council consensus	Verdict
TC1: Gold standard	100.0	88.1	91.4	STRONG_HIRE
TC2: Buzzword exaggeration	47.1	41.6	50.3	REJECT
TC3: Lateral partial match	47.1	61.3	76.0	LEAN_HIRE
TC4: Corrupt ATS format	0.0	34.0	38.5	REJECT

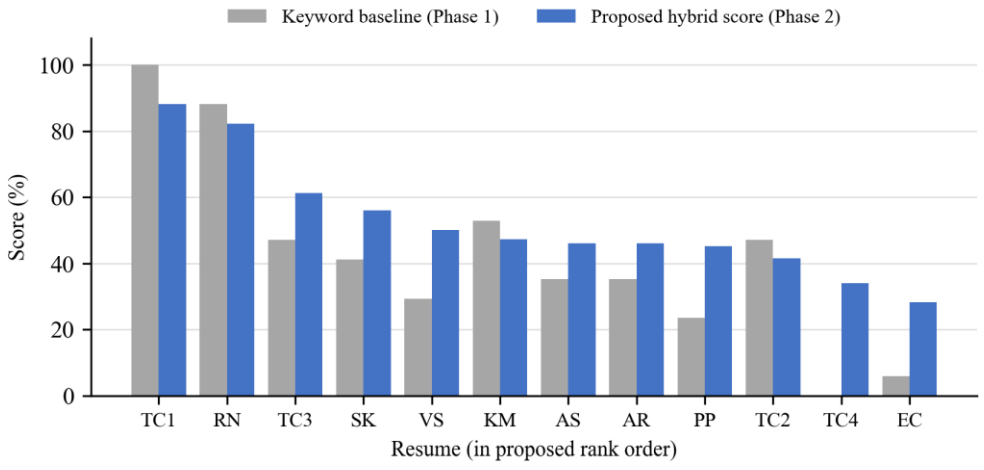


Figure 8.13: Keyword baseline versus proposed hybrid score for the twelve pilot resumes.

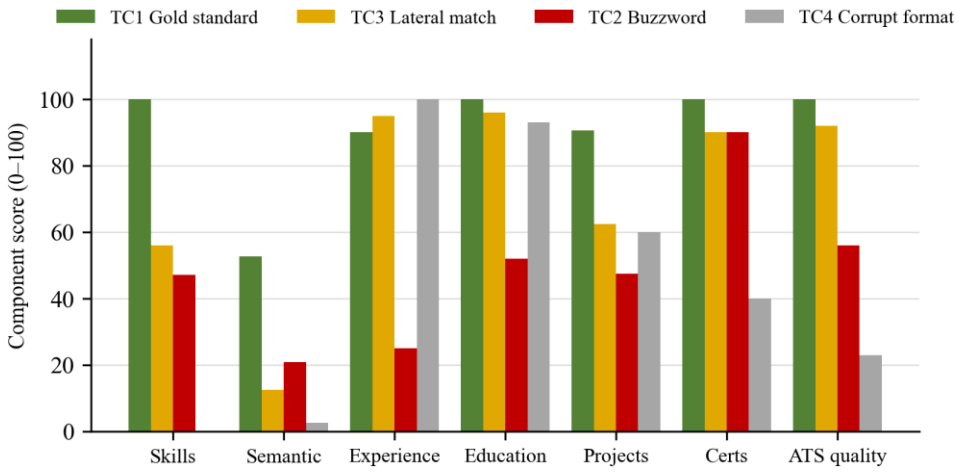


Figure 8.14: Component scores of the four demonstration resumes.

8.5 Evidence Mechanism

Table 8.3 shows why. TC2 lists 8 required skills but evidences only 1, which gives an unevidenced ratio of 0.90 and an evidence-weighted coverage of 0.27 (TC3: 0.47, TC1: 0.97). Its council consensus of 50.3 falls below the 55-point threshold for LEAN_HIRE, and the resulting REJECT carries the flags for the experience deficit, the unverified skill claims and skill stuffing. TC3 keeps LEAN_HIRE because FastAPI is credited at 0.5 through Flask, while PyTorch and the deep-learning stack are reported as missing, which is what a recruiter should hear about a lateral candidate. TC4 illustrates the handling of uncertainty: with no parsed skills and inverted dates, the ATS quality score is 23 out of 100, the confidence falls to 32.8 and the council requests a human review instead of issuing a confident verdict.

Table 8.3: Council Evidence Metrics and Flags of the Four Test Cases

Case	Listed / evid.	Coverage c_{ev}	Unevid. ratio	Quant. results	Verdict	Main flags
TC1	17 / 16	0.97	0.09	9	STRONG_HIRE	None
TC2	8 / 1	0.27	0.90	1	REJECT	Experience Deficit, Unverified Skill Claims, Skill Stuffing
TC3	8 / 8	0.47	0.07	5	LEAN_HIRE	Required Skills Missing
TC4	0 / 0	0.00	0.00	0	REJECT (human review)	No Quantified Impact, Date Inconsistency

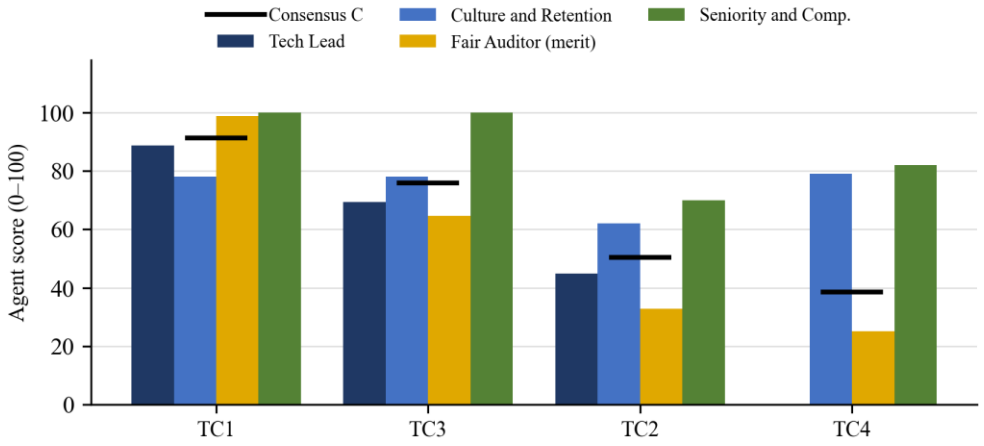


Figure 8.15: Scores of the four council agents and the consensus for the demonstration resumes.

8.6 Ablation of Evidence Weighting and Gates

The council was re-run with evidence weighting disabled (coverage computed from the listed skills only, with no unevidenced-skill penalty) and, separately, with the red-flag gates disabled (Table 8.4, Figure 8.16). Without evidence weighting every consensus score rises by 0.7 to 4.9 points. TC2 rises from 50.3 (REJECT) to 55.2 and is promoted to LEAN_HIRE just above the 55-point boundary, and Vikram Singh, a DevOps resume that lists several required machine-learning skills, is promoted in the same way; the other ten verdicts are unchanged. Evidence weighting is a deliberate trade-off: it penalises resumes whose skills appear only in lists, which can hurt terse but honest resumes, which is why such cases are routed to human review. Disabling the red-flag gates changed no verdict on this pool, because the flagged candidates already fall below the threshold; the gates are a safety net that this small pool does not exercise, and no empirical benefit is claimed for them here.

Table 8.4: Ablation: Effect of Evidence Weighting and Red-Flag Gates on the Pilot Pool

Configuration	Effect on the council consensus	Verdicts changed (of 12)
Full council (default)	Reference values (TC2: 50.3, REJECT)	–
Evidence weighting disabled	Every consensus rises by 0.7 to 4.9 points (TC2: 55.2)	2 (TC2 and Vikram Singh: REJECT to LEAN_HIRE)
Red-flag gates disabled	No change; the gates act on the verdict, not on the score	0

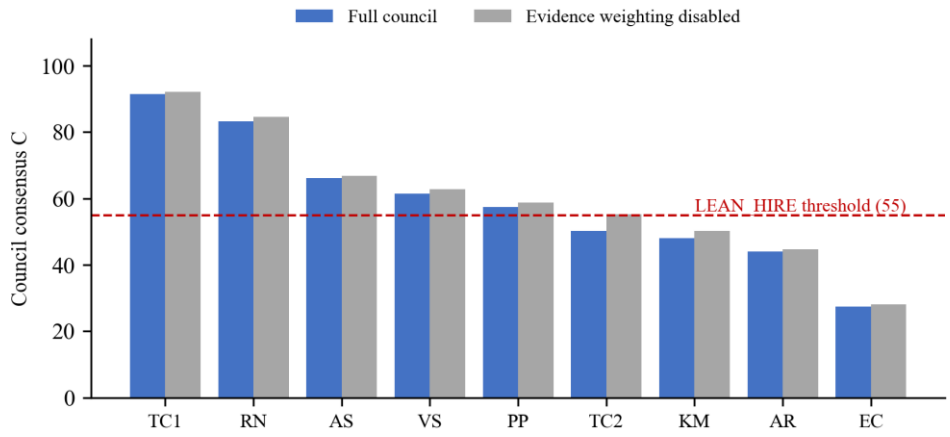


Figure 8.16: Council consensus with and without evidence weighting.

8.7 Demographic Robustness: Counterfactual Name Swap (RQ2)

Following the design of the audit study of Bertrand and Mullainathan, each of the 12 resumes was re-scored with four names used in that study (Emily Walsh, Greg Baker, Lakisha Washington and Jamal Jackson), substituted in both the name field and the text, with and without blind mode. This gives 12 × 4 × 2 = 96 runs. In every run the hybrid score and the council consensus were identical to those of the unmodified resume (Table 8.5). This is expected for a deterministic pipeline in which the name is never a scoring feature, so the invariance holds in the standard mode as well as in the blind mode. It should not be over-read: it demonstrates invariance for the TF-IDF backend and the rule-based council, not for dense embeddings or a language-model narrative, which are exactly the components where bias was observed in the literature. Blind mode is therefore a defence in depth, and testing it with the dense model and a language model is future work.

Table 8.5: Name-Swap Experiment: Maximum Score Change over All Substitutions (Points)

Mode	Hybrid score Δ	Council Δ	Runs
Standard	0.00	0.00	48
Blind	0.00	0.00	48

8.8 Sensitivity to the Heuristic Weights (RQ3)

Because the weights were chosen by the developers, each ranker weight was perturbed by a random factor between 0.8 and 1.2 (renormalised, 200 Monte Carlo draws) and each council weight likewise (50 draws on each of the 12 resumes). Table 8.6 shows

that the rankings remained close to the default (mean Kendall τ = 0.969, minimum 0.879); the top candidate and the top-3 set were unchanged in all 200 draws, and the council verdict of every resume was unchanged in all 600 evaluations. The conclusions of the ranking analysis therefore do not depend on exact weight values within this range of plus or minus 20 percent, although larger changes and learned weights remain untested.

Table 8.6: Monte Carlo Weight-Perturbation Analysis (±20 Percent)

Quantity	Value
Ranker draws	200
Mean Kendall τ against the default ranking	0.969
Minimum Kendall τ	0.879
Top-1 candidate stability	100.0%
Top-3 set stability	100.0%
Council verdict stability (600 evaluations)	100.0%

8.9 Talent-Search Retrieval Quality

Eight natural-language queries (for example *Engineers with 3+ years of Python and Git*) were issued. The resumes whose parsed skills contain every queried skill and whose experience meets the stated minimum were taken as relevant. BM25 only, dense (TF-IDF fallback) only and the hybrid fusion at alpha = 0.55 all reached a precision at 3 of 91.7 percent and a recall at 3 of 100.0 percent (Table 8.7). On a 12-resume pool the task is saturated, so this test confirms the correctness of the search but cannot show an advantage of the fusion. Evaluating dense and hybrid retrieval with the dense model on a larger pool with paraphrased queries is future work, and no superiority claim is made.

Table 8.7: Retrieval Quality (8 Queries, 12 Resumes)

Retriever	Precision@3	Recall@3
BM25 only	91.7%	100.0%
Dense only (TF-IDF fallback)	91.7%	100.0%
Hybrid, α = 0.55	91.7%	100.0%

8.10 Scaled 100-Resume Multi-Role Benchmark

Table 8.8 gives the verdict distribution by role; 94 of the 100 resumes carry at least one red flag, and the most frequent flags were REQUIRED_SKILLS_MISSING (70), NO_QUANTIFIED_IMPACT (67), EXPERIENCE_DEFICIT (40) and LEVEL_MISMATCH (12). Agreement with the generation label is defined strictly: a

strong-hire profile should receive STRONG_HIRE, a lean-hire profile HIRE or LEAN_HIRE, and a reject profile REJECT. Table 8.9 shows the confusion matrix. The exact agreement is 46 of 100 (46.0 percent). The gross errors that matter most for a decision-support tool did not occur: no strong-hire profile was rejected and no reject profile received STRONG_HIRE.

Table 8.8: 100-Resume Benchmark by Role (S = Strong, L = Lean or Hire, R = Reject)

Role	n	Labels (S / L / R)	Verdicts (S / L / R)	Flagged
Senior ML engineer	13	3 / 6 / 4	0 / 11 / 2	10
Cloud DevOps and SRE lead	13	3 / 5 / 5	0 / 11 / 2	13
Full-stack AI engineer	13	3 / 5 / 5	0 / 12 / 1	10
Data platform engineer	13	3 / 5 / 5	0 / 13 / 0	13
Backend systems architect	12	2 / 6 / 4	0 / 11 / 1	12
AI safety and alignment	12	2 / 6 / 4	0 / 4 / 8	12
BI and analytics lead	12	2 / 6 / 4	0 / 6 / 6	12
Python / FastAPI developer	12	2 / 6 / 4	0 / 11 / 1	12
All roles	100	20 / 45 / 35	0 / 79 / 21	94

Table 8.9: Confusion Matrix: Generation Label Versus System Verdict

Generation label	STRONG_HIRE	HIRE / LEAN_HIRE	REJECT	Mean consensus C
Strong (20)	0	20	0	71.2
Lean (45)	0	35	10	60.1
Reject (35)	0	24	11	62.0

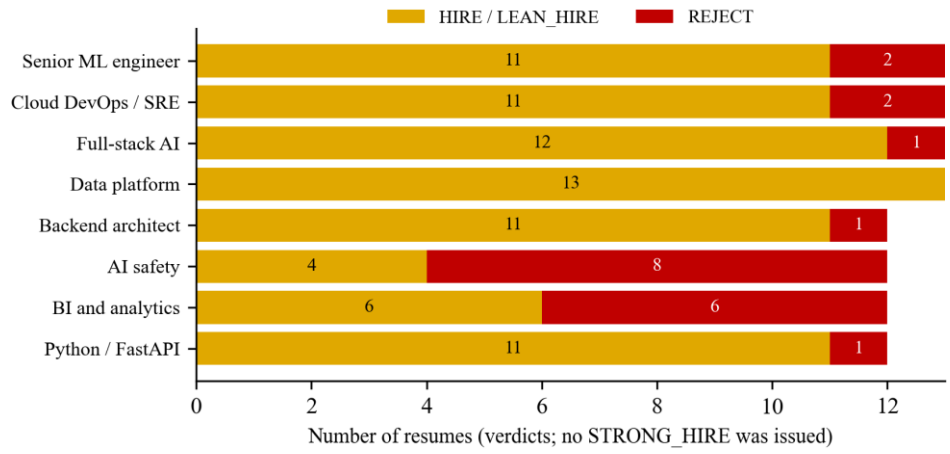


Figure 8.17: Verdicts of the 100-resume benchmark by role.

The results also expose weaknesses that are not hidden. The system never issued STRONG_HIRE on this benchmark (strong-hire recall 0 of 20), which is attributed to the

level gate of the seniority agent, since senior-level experience was not fully evidenced in the text of the resumes. The mean consensus of the reject profiles (62.0) is slightly higher than that of the lean-hire profiles (60.1), so the score alone does not order these groups, and the reject recall is only 11 of 35. Because the labels were assigned by the generator itself, agreement with them measures consistency with the specification and not hiring accuracy, and no accuracy figure is reported beyond the confusion matrix.

8.11 Adversarial Edge-Case Stress Test (RQ5)

The first 20 benchmark resumes were run through the earlier pipeline (TF-IDF backend, rule-based council, no temporal gates). The four strong profiles all received STRONG_HIRE and all five buzzword profiles were rejected with the flags for experience deficit and unverified claims. The anomaly probes, however, were not caught (Table 8.10). Candidate 019 lists a senior role as *Jul 2026 to Jan 2024*; the old parser discarded the inverted interval, so the resume was scored on a 2.0-year history and still received HIRE (72.5). Candidate 013 lists a full-time on-site role in San Jose from March 2023 concurrent with a full-time role in Bengaluru from January 2022 to August 2025, an overlap of 29 months; it received STRONG_HIRE (89.7) with no flag. Under the upgraded pipeline, the gates cap 019, 013 and C1 at LEAN_HIRE, and C2 drops from STRONG_HIRE to HIRE.

Table 8.10: Adversarial Probes: Verdicts Before and After the New Gates (Consensus in Parentheses)

ID	Defect	Before	After	Flag
019	Inverted dates, Jul 2026 to Jan 2024	HIRE (72.5)	LEAN_HIRE (73.5)	DATE_INCONSISTENCY
013	Concurrent full-time roles, San Jose and Bengaluru	STRONG_HIRE (89.7)	LEAN_HIRE (89.0)	CONCURRENT_EMPLOYMENT_CONFLICT
C1	Inverted dates in a strong profile	HIRE (78.6)	LEAN_HIRE (73.2)	DATE_INCONSISTENCY
C2	No education, phone or LinkedIn	STRONG_HIRE (92.6)	HIRE (78.6)	MISSING_EDUCATION

This probe is a regression case study and not a held-out detection rate: the gates were written after these failures were observed, so catching the same resumes shows that the fix works, not that the gates generalise to unseen fraud patterns. The exercise also shows why the semantic component cannot see these attacks: an inverted date range or a second employer is invisible to TF-IDF and to embedding similarity, and only an explicit timeline model exposes them.

8.12 Discussion of the Research Questions

The experiments answer the research questions as follows:

- **RQ1 – Research Question 1:** Yes, within the pilot study. The evidence mechanism separates the keyword-stuffed resume from a genuine resume that the keyword baseline scores identically (47.1 percent), and ranks it tenth of twelve. This is shown by a case analysis on a small synthetic pool and not by a statistical test.
- **RQ2 – Research Question 2:** Yes, for the deterministic components. Scores and verdicts did not change in 96 name-substitution runs; invariance of the optional dense embeddings and language-model narrative was not tested.
- **RQ3 – Research Question 3:** Yes, within ±20 percent. The rankings (mean Kendall τ of 0.969) and all verdicts were stable under random weight changes of up to 20 percent; larger changes and learned weights remain untested.
- **RQ4 – Research Question 4:** A complete screening costs about 30 milliseconds per resume in the offline configuration, which is about 34 resumes per second on one CPU thread.
- **RQ5 – Research Question 5:** Yes, for the probes tested. The explicit timeline gates intercept the inverted-date and concurrent-employment probes that similarity scoring cannot see, but the probes were used to design the gates, so generalisation to unseen anomalies is not shown.

The main weaknesses are the circularity of synthetic data, the conservative calibration of the verdicts (no strong hire in the 100-resume benchmark and only 11 of 35 reject profiles rejected), the heuristic weights and thresholds, the limited fairness scope of blind screening (direct identifiers only), the lack of a usability study, and the free-tier hosting of the deployed service, which sleeps when idle and has not been load-tested. They are stated here so that the results are not over-read.

Beyond the experiments, the system now runs as a deployed multi-user service, with the web interface on Vercel, the API on Render and the accounts, sessions and candidate pools in MongoDB Atlas. Its behaviour is checked by 78 automated tests and by manual checks of the live site. These checks show that the service works as designed; they do not measure its performance under load or its usability for recruiters.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The AI Powered Resume Screening and Ranking System shows that resume screening can be fast, explainable and fair at the same time. It replaces the keyword score of the first phase by a decision process that treats every skill as a claim that needs evidence. A skill that is demonstrated in experience or project text counts fully, a skill that is only listed counts half, and a related skill counts half through an explicit, auditable rule. These credits feed a seven-factor hybrid ranker and a four-agent council, so that the final verdict can be traced to the features, rules and disagreements that produced it.

The system was implemented as a Python core library, a FastAPI service with a relational data model, a Streamlit workspace and a React web interface. The web interface and the API are deployed as a multi-user service with server-side accounts (salted PBKDF2 hashes, expiring sessions and verified Google sign-in) and a managed MongoDB Atlas database. The core includes a resume parser with an OCR fallback, a 79-skill ontology, a hybrid ranker, a resume quality analyser, a blind-screening filter, a council with knock-out gates, red flags, confidence and human-review triggers, a GitHub authenticity inspector, a hybrid talent search, an interview-question generator and a retrieval-grounded recruiter chatbot. A suite of 78 automated tests covers these components, the account service and the logic of the web interface, and all of them pass.

The evaluation on synthetic resume pools gave the following results. A complete screening takes about 30 milliseconds per resume in the offline configuration. The keyword-stuffed test resume that a keyword baseline scores at 47.1 percent, the same as a genuine four-year engineer, is ranked tenth of twelve and rejected, while the strong test candidate is ranked first with 88.1 percent. The scores and the verdicts did not change in 96 name-substitution runs, and the rankings were stable under random changes of the weights by 20 percent (mean Kendall rank correlation 0.969). Explicit timeline gates intercepted resumes with inverted dates and concurrent full-time employment that similarity scoring could not see.

The evaluation also showed the limits of the work. All data are synthetic and were written with knowledge of the rules, so the results show consistency and mechanism and not accuracy on real hiring outcomes. On a 100-resume benchmark the council was conservative at the top (no strong-hire verdict) and permissive at the bottom (11 of 35 reject profiles rejected). The weights and thresholds are chosen by the developers, blind screening removes only direct identifiers, no usability study has been carried out, and the deployed service runs on free cloud tiers that have not been load-tested. The system should therefore be used as a decision-support tool with a human in the loop, and not as an autonomous decision-maker. Overall, the project shows that explainable, evidence-based and bias-aware screening is practical, transparent and fast enough for everyday use.

9.2 Future Enhancements

The following enhancements can improve the system.

- **Evaluation on real data:** a labelled evaluation on uncurated applicant pools, with recruiter agreement and ranking metrics such as nDCG as ground truth, is the decisive test of the system. Held-out adversarial resumes written independently of the gate rules should be added.
- **Learned and calibrated weights:** the weights, the thresholds and the verdict boundaries can be learned and calibrated from recruiter feedback, so that the council issues strong-hire verdicts where recruiters would.
- **Group-fairness measurement:** metrics such as the adverse-impact ratio and FA*IR-style ranking constraints should be measured on demographically labelled data, and bias tests should be extended to the dense embeddings and to language-model narratives.
- **Richer evidence:** live validation of the GitHub inspector on real candidate profiles, additional evidence sources such as LinkedIn and portfolios, and validation of certificates with their issuers.
- **Larger ontology and multilingual resumes:** extension of the skill ontology beyond 79 skills, possibly by mapping to a public taxonomy, and support for resumes in more languages.
- **Retrieval evaluation:** evaluation of the dense and hybrid search with the sentence-embedding model on larger pools with paraphrased queries, and the use of a cross-encoder for fine re-ranking.

- **API tests, stored history and e-mail:** automated tests for the relational REST endpoints (the account and candidate endpoints are already tested), persistence of screening runs and recruiter decisions for each account in the managed database (the accounts, the sessions and the candidate pools are already stored in MongoDB Atlas), and delivery of interview invitations by e-mail through the API, which the web interface is already prepared to use through configuration.
- **Usability study and production deployment:** a usability study with practising recruiters, a load test of the deployed service, and the step from the present free-tier deployment to a production platform with role-based access, candidate notice and consent, data-retention limits and regular fairness audits.

BIBLIOGRAPHY

[1] J. S. Black and P. van Esch, "AI-enabled recruiting: What is it and how should a manager use it?," Business Horizons, vol. 63, no. 2, pp. 215–226, 2020. doi: 10.1016/j.bushor.2019.12.001.

[2] A. K. Upadhyay and K. Khandelwal, "Applying artificial intelligence: implications for recruitment," Strategic HR Review, vol. 17, no. 5, pp. 255–258, 2018. doi: 10.1108/SHR-07-2018-0051.

[3] P. van Esch, J. S. Black, and J. Ferolie, "Marketing AI recruitment: The next phase in job application and selection," Computers in Human Behavior, vol. 90, pp. 215–222, 2019. doi: 10.1016/j.chb.2018.09.009.

[4] P. Tambe, P. Cappelli, and V. Yakubovich, "Artificial Intelligence in Human Resources Management: Challenges and a Path Forward," California Management Review, vol. 61, no. 4, pp. 15–42, 2019. doi: 10.1177/0008125619867910.

[5] M. Langer, C. J. König, and M. Papathanasiou, "Highly automated job interviews: Acceptance under the influence of stakes," International Journal of Selection and Assessment, vol. 27, no. 3, pp. 217–234, 2019. doi: 10.1111/ijsa.12246.

[6] C. Qin, H. Zhu, T. Xu, C. Zhu, L. Jiang, E. Chen, and H. Xiong, "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in Proc. 41st Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2018, pp. 25–34. doi: 10.1145/3209978.3210025.

[7] V. Bhatia, P. Rawat, A. Kumar, and R. R. Shah, "End-to-End Resume Parsing and Finding Candidates for a Job Description using BERT," arXiv:1910.03089, 2019.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.

[9] J. Dastin, "Amazon scraps secret AI recruiting tool that showed bias against women," Reuters, Oct. 10, 2018.

[10] K. Wilson and A. Caliskan, "Gender, Race, and Intersectional Bias in Resume Screening via Language Model Retrieval," in Proc. AAAI/ACM Conf. AI, Ethics, and Society (AIES), 2024, arXiv:2407.20371.

[11] L. Armstrong, A. Liu, S. MacNeil, and D. Metaxa, "The Silicon Ceiling: Auditing GPT's Race and Gender Biases in Hiring," arXiv:2405.04412, 2024.

[12] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," Official Journal of the European Union, Jun. 2024.

[13] M. Bertrand and S. Mullainathan, "Are Emily and Greg More Employable Than Lakisha and Jamal? A Field Experiment on Labor Market Discrimination," American Economic Review, vol. 94, no. 4, pp. 991–1013, 2004. doi: 10.1257/0002828042002561.

[14] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation)," Official Journal of the European Union, Apr. 2016.

[15] C. Qin, L. Zhang, Y. Cheng, R. Zha, D. Shen, Q. Zhang, X. Chen, Y. Sun, C. Zhu, H. Zhu, and H. Xiong, "A Comprehensive Survey of Artificial Intelligence Techniques for Talent Analytics," arXiv:2307.03195, 2023.

[16] C. de Ruijt and S. Bhulai, "Job Recommender Systems: A Review," arXiv:2111.13576, 2021.

[17] C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li, "Person-Job Fit: Adapting the Right Talent for the Right Job with Joint Representation Learning," arXiv:1810.04040, 2018.

[18] R. Le, W. Hu, Y. Song, T. Zhang, D. Zhao, and R. Yan, "Towards Effective and Interpretable Person-Job Fitting," in Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM), 2019, pp. 1883–1892. doi: 10.1145/3357384.3357949.

[19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2019, pp. 4171–4186.

[20] K. F. F. Jiechieu and N. Tsopze, "Skills prediction based on multi-label resume classification using CNN with model predictions explanation," Neural Computing and Applications, vol. 33, no. 10, pp. 5069–5087, 2021. doi: 10.1007/s00521-020-05302-x.

[21] A. Bhola, K. Halder, A. Prasad, and M.-Y. Kan, "Retrieving Skills from Job Descriptions: A Language Model Based Extreme Multi-label Classification Framework," in Proc. 28th Int. Conf. Computational Linguistics (COLING), 2020, pp. 5832–5842. doi: 10.18653/v1/2020.coling-main.513.

[22] J.-J. Decorte, J. Van Hautte, T. Demeester, and C. Develder, "JobBERT: Understanding Job Titles through Skills," arXiv:2109.09605, 2021.

[23] M. Zhang, K. N. Jensen, S. D. Sonniks, and B. Plank, "SkillSpan: Hard and Soft Skill Extraction from English Job Postings," arXiv:2204.12811, 2022.

[24] B. Clavié and G. Soulié, "Large Language Models as Batteries-Included Zero-Shot ESCO Skills Matchers," arXiv:2307.03539, 2023.

[25] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 5776–5788.

[26] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[27] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/1500000019.

[28] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[29] C. Hauff and G. Gousios, "Matching GitHub Developer Profiles to Job Advertisements," in Proc. 12th IEEE/ACM Working Conf. Mining Software Repositories (MSR), 2015, pp. 362–366. doi: 10.1109/MSR.2015.41.

[30] L. Chen, R. Ma, A. Hannák, and C. Wilson, "Investigating the Impact of Gender on Rank in Resume Search Engines," in Proc. CHI Conf. Human Factors in Computing Systems, 2018, pp. 1–14. doi: 10.1145/3173574.3174225.